



아이디어를 서비스로 바꾸는 Codex 사용법

폴더 생성부터 배포·결제·SEO까지
혼자 완성하는 실전 가이드

필립 지음

아이디어를 서비스로 바꾸는 Codex 사용법

폴더 생성부터 GitHub·배포·결제·SEO·자동화까지



필립 지음

초판 제작본 · 2026

이 책은 저자의 실제 제품 제작과 운영 경험을 바탕으로 합니다. 특정 수익을 보장하지 않으며, 법률·세무·플랫폼 정책은 실행 시점의 최신 기준을 별도로 확인해야 합니다.

이 책이 독자에게 약속하는 것

이 책은 IT를 전혀 모르는 비개발자가 ChatGPT와 Codex를 개발팀으로 삼아 서비스 폴더를 만들고, GitHub에 기록하고, Vercel에 배포하고, 회원·결제·검색·지도·자동화를 연결하는 전 과정을 다룹니다.

마리에카드, 아파트구구와 다니엘의 바이블의 실제 저장소와 운영 경험을 바탕으로 로그인, 데이터, 배포, 속도, 자동발행, 앱 심사와 구독 문제를 고친 과정까지 설명합니다.

코드를 외우는 책이 아니라 Codex에 일을 맡기고 결과를 검증하며 혼자 서비스를 책임지는 법을 익히는 책입니다.

추천 독자

- IT 용어와 코드가 낯설어 시작하지 못한 사람
- 직장을 유지하며 사이드 프로젝트를 시작하려는 사람
- Codex로 만든 코드를 배포와 운영까지 이어가고 싶은 사람
- 회원·결제·SEO·자동화를 가진 실제 서비스를 만들고 싶은 사람

목차

1부. IT를 몰라도 시작할 수 있는 최소 지식	9
1장. 개발자가 없어도 서비스를 만들 수 있는 시대	10
2장. 웹사이트와 웹서비스는 무엇이 다른가	13
3장. 프론트엔드·백엔드·서버·데이터베이스를 가계에 비유하기	15
4장. 폴더·파일·터미널·코드가 실제로 의미하는 것	18
5장. API와 환경변수, 키와 토큰을 이해하기	20
6장. 무료 도구와 유료 도구를 선택하는 기준	22
2부. ChatGPT와 Codex를 나의 개발팀으로 만들기	23
7장. ChatGPT와 Codex의 역할을 나누는 법	25
8장. Codex가 일할 프로젝트 폴더 만들기	27
9장. 좋은 요청의 네 요소: 목표·맥락·제약·완료 조건	29
10장. 계획하고, 만들고, 테스트하고, 검토하는 한 사이클	32
11장. AGENTS.md로 같은 실수를 반복하지 않게 하기	35
12장. 권한·승인·비밀정보를 안전하게 다루기	38
13장. 오류 화면을 Codex와 함께 해결하는 법	41
3부. 첫 서비스를 내 컴퓨터에서 실행하기	43
14장. 한 페이지 기획서와 데이터 설계	45
15장. Next.js 프로젝트를 만들고 실행하기	47
16장. 화면을 페이지와 컴포넌트로 나누기	49
17장. 입력·계산·저장·결과의 기본 흐름 만들기	52
18장. 모바일 화면과 접근성까지 확인하기	54

19장. 테스트와 빌드가 왜 필요한가	56
4부. GitHub에 기록하고 인터넷에 배포하기	57
20장. Git과 GitHub는 같은 것이 아니다	59
21장. 커밋·푸시·풀·브랜치·PR을 완전히 이해하기	61
22장. Codex에게 안전하게 커밋과 푸시를 맡기기	64
23장. 배포란 내 컴퓨터의 결과물을 인터넷에 올리는 일	67
24장. Vercel과 GitHub Pages의 차이	69
25장. Vercel 배포와 도메인·환경변수 연결	71
5부. 회원과 데이터를 가진 서비스 만들기	73
26장. Supabase가 해주는 네 가지 일	75
27장. 테이블과 행, 열, 관계를 초보자 눈높이로 이해하기	78
28장. 회원가입과 Google·Kakao 로그인 붙이기	80
29장. 이미지 업로드와 Storage 사용하기	82
30장. 관리자 페이지와 권한 설계	84
31장. 개인정보와 보안을 나중으로 미루지 않기	86
6부. 외부 서비스 연결하기	87
32장. 결제 붙이기: 상품·주문·승인·웹훅	89
33장. Google·Naver·Kakao 지도를 선택하고 연결하기	92
34장. 카카오톡 공유와 채널 상담 연결하기	94
35장. Jira로 해야 할 일을 관리하기	96
7부. 검색과 수익 만들기	98
36장. SEO는 검색엔진을 속이는 기술이 아니다	100
37장. Google Search Console 연결하기	102

39장. sitemap·robots·canonical·noindex 이해하기	106
40장. Google AdSense를 붙이기 전에 준비할 것	109
8부. Threads 홍보와 자동화	110
41장. 홍보 글을 제품의 데이터에서 만들기	112
42장. Threads API로 자동발행 봇 만들기	114
43장. 예약발행·중복방지·실패복구 설계	117
44장. Codex로 운영 자동화를 계속 개선하기	119
9부. 혼자 운영하고 성장시키기	121
45장. 로그·모니터링·백업·장애 대응	123
46장. 비용이 갑자기 커지지 않게 하는 법	125
47장. 마리에카드를 만든 실제 순서	127
48장. 아파트구구를 키운 실제 순서	130
49장. 다니엘의 바이블로 배운 앱·구독·AI 운영	132
50장. 첫 30일 실행 계획	135
부록. 바로 복사해 쓰는 Codex 실전 도구	138
부록 1. 새 서비스 시작 프롬프트	139
부록 2. 기능 구현 프롬프트	140
부록 3. 버그 수정 프롬프트	141
부록 4. 배포 전 프롬프트	142
부록 5. 출시 전 체크리스트	143
부록 6. 초보자 용어집	144
부록 7. 공식 문서와 업데이트 원칙	147
부록 8. 하나의 예약 서비스로 보는 50단계	149
부록 9. 장별 완료 증거 기록표	151

부록 10. 자주 만나는 오류 실습 8가지	152
부록 11. 판본과 업데이트 확인표	153
경험편. 아이디어를 결제로 바꾸는 법	154
프롤로그	155
만들고 싶은 사람과 결제받는 사람의 차이	156
1부. 돈을 지불할 문제 찾기	158
1장. 사이드 프로젝트는 아이디어가 아니라 불편에서 시작한다	159
2장. 경쟁 서비스를 직접 구매해 보라	162
3장. 고객의 말보다 고객이 이미 하고 있는 행동을 본다	165
4장. 완전히 새로운 아이디어가 아니어도 되는 이유	168
2부. 비개발자가 서비스를 만드는 방법	171
5장. 개발자보다 먼저 찾아야 할 것	172
6장. 개발자 한 명과 반년 동안 MVP 만들기	174
7장. 기능을 더하는 것보다 빼는 것이 어려웠다	176
8장. 고객이 스스로 입력하고 결제하게 만들기	178
9장. 디자인과 개발 사이에서 반드시 문서로 남길 것	181
3부. 첫 고객과 첫 결제 만들기	183
10장. 출시했다고 고객이 오지는 않는다	184
11장. 결혼 준비 커뮤니티에서 고객을 만나다	186
12장. 후기 한 줄이 광고보다 강했던 이유	188
13장. 고객 문의에서 다음 기능을 찾는 법	191
14장. 자체 결제에서 스마트스토어로 이동한 이유	193

4부. 작은 서비스를 사업으로 바꾸기 196

15장. B2C에서 웨딩플래너용 B2B로 확장하기	197
16장. 관리자 페이지가 매출보다 먼저 해결한 것	200
17장. 자동화해도 사람이 해야 하는 일은 남는다	202
18장. 직장과 서비스를 동시에 운영하는 현실	204
19장. 아무것도 하지 않아도 팔릴 때 더 위험하다	206

5부. 계속할 것인가, 팔 것인가 209

20장. 잘 팔리던 서비스를 매각한 이유	210
21장. 공동창업자와 역할·수익을 나눌 때 생기는 문제	213
22장. 서비스의 가치를 설명하는 숫자들	215
23장. 매각 전에 정리해야 할 운영 자산	218
24장. 첫 서비스가 다음 도전에 남긴 것	220

부록 223

부록 1. 문제 발견 기록지	224
부록 2. 경쟁 서비스 분석표	225
부록 3. 고객 인터뷰 질문지	226
부록 4. 1페이지 MVP 기획서	227
부록 5. 개발자 협업 체크리스트	228
부록 6. 출시 전 점검표	229
부록 7. 첫 30일 고객 확보 계획	230
부록 8. 서비스 운영-매각 체크리스트	232



1부. IT를 몰라도 시작할 수 있는 최소 지식

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

1장. 개발자가 없어도 서비스를 만들 수 있는 시대



과거에는 서비스를 만들려면 기획자, 디자이너, 프론트엔드 개발자, 백엔드 개발자와 서버 담당자가 필요하다고 생각했다. 나 역시 모바일 청첩장 서비스를 처음 만들 때 개발자와 함께 반년 동안 MVP를 만들었다. 화면을 설명하고 결과를 기다린 뒤 수정 요청을 전달하는 방식이었다.

지금은 한 사람이 Codex에 목표를 설명하고, Codex가 프로젝트를 조사하고 파일을 만들고 명령을 실행하고 테스트하도록 할 수 있다. 그렇다고 개발 지식이 완전히 필요 없어지는 것은 아니다. 직접 코드를 암기할 필요는 줄었지만 무엇을 만들고 있는지, 무엇이 위험한지, 결과가 맞는지 판단하는 책임은 여전히 사람에게 있다.

비개발자의 장점도 있다. 고객이 어떤 말에서 막히는지, 어떤 화면이 불편한지, 어떤 기능에 돈을 내는지를 더 가까이 볼 수 있다. AI가 구현 비용을 낮추면 문제를 고르고 우선순위를 정하는 능력이 더 중요해진다.

이 책에서 말하는 혼자 개발한다는 것은 모든 코드를 손으로 타이핑한다는 뜻이 아니다. 다음 다섯 가지를 혼자 책임진다는 뜻이다.

- 1 어떤 문제를 해결할지 결정한다.

- 2 Codex가 일할 수 있도록 목표와 자료를 준다.
- 3 만들어진 결과를 직접 사용해 본다.
- 4 테스트와 로그로 문제를 확인한다.
- 5 배포와 운영의 최종 결정을 내린다.

첫 번째 원칙

Codex는 일을 대신할 수 있지만 서비스의 책임까지 대신하지는 않는다.

이번 장의 Codex 실행 카드

목표: 내 아이디어에서 고객이 돈을 낼 문제 하나를 고른다. 먼저 내 메모와 고객 경험을 확인해줘. 코드를 만들지 말고 문제와 가설만 정리한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 고객·문제·현재 해결법·첫 검증 행동이 한 페이지로 정리된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 누가 언제 어떤 불편을 겪었는지 실제 사례로 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “알아서 돈 되는 앱 전부 만들어줘”

직접 확인: 내가 직접 만날 수 있는 고객인가; 문제를 한 문장으로 말할 수 있는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 내 아이디어에서 고객이 돈을 낼 문제 하나를 고른다. 완료되면 고객·문제·현재 해결법·첫 검증 행동이 한 페이

지로 정리된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

2장. 웹사이트와 웹서비스는 무엇이 다른가

웹사이트는 정보를 보여주는 데 중심이 있다. 회사 소개, 포트폴리오, 행사 안내 페이지가 대표적이다. 방문자가 내용을 읽고 링크를 누르는 정도라면 정적 사이트로도 충분하다.

웹서비스는 사용자의 입력을 받고 결과를 바꾸거나 데이터를 저장한다. 마리에카드에서 사용자는 신랑·신부 이름과 날짜를 입력하고 사진을 올린다. 아파트구구에서는 소득과 자산을 입력하고 구매 가능 금액을 계산한다. 로그인, 결제, 저장, 검색, 알림이 들어가면 웹서비스의 성격이 강해진다.

둘의 차이를 알아야 배포 도구를 고를 수 있다. 단순 소개 페이지라면 GitHub Pages도 가능하다. 하지만 서버에서 결제를 승인하거나 로그인한 사용자의 데이터를 안전하게 읽어야 한다면 서버 기능이 있는 Vercel과 데이터베이스가 필요하다.

스스로 구분하는 질문

- 사용자마다 다른 결과가 필요한가?
- 입력한 내용을 나중에도 다시 불러와야 하는가?
- 로그인이나 결제가 필요한가?
- 비밀 키를 숨긴 채 외부 API를 호출해야 하는가?
- 정해진 시각에 자동으로 일을 실행해야 하는가?

하나라도 그렇다면 단순 페이지가 아니라 서비스 구조를 생각해야 한다.

이번 장의 Codex 실행 카드

목표: 내 아이디어가 정적 웹사이트인지 데이터가 필요한 웹서비스인지 구분한다. 먼저 원하는 화면과 사용자 행동 목록을 확인해줘. 로그인·저장·결제가 정말 필요한지 과장하지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 필요 기능과 배포 방식이 웹사이트·웹서비스로 구분된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 사용자가 입력하고 다시 돌아왔을 때 기억해야 할 내용을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “일단 서버와 데이터베이스까지 전부 붙여줘”

직접 확인: 사용자마다 다른 결과가 있는가; 비밀 키가 필요한가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 내 아이디어가 정적 웹사이트인지 데이터가 필요한 웹서비스인지 구분한다. 완료되면 필요 기능과 배포 방식이 웹사이트·웹서비스로 구분된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

3장. 프론트엔드·백엔드·서버·데이터베이스를 가게에 비유하기



처음 IT 용어를 배우면 각각이 모두 별도 기술처럼 보인다. 작은 가게를 떠올리면 쉽다.

용어	가게의 비유	서비스에서 하는 일
프론트엔드	손님이 보는 매장	화면, 버튼, 입력창, 결과
백엔드	직원이 일하는 주방	계산, 검증, 권한 확인, 외부 서비스 호출
서버	주방이 실제로 돌아가는 장소와 설비	요청을 받아 코드를 실행하고 응답
데이터베이스	장부와 참고 목록	회원, 상품, 주문, 작성한 내용 저장

용어	가계의 비유	서비스에서 하는 일
스토리지	물건을 보관하는 창고	사진, 영상, PDF 같은 큰 파일 저장
API	주문서와 전달 창구	서로 다른 서비스가 약속된 형식으로 대화

마리에카드의 편집 화면은 프론트엔드다. 발행할 권한과 쿠폰을 확인하는 코드는 백엔드다. Vercel은 이 코드를 인터넷에서 실행해 주고, Supabase는 회원과 초대장 내용을 저장한다.

아파트구구에서 계산 화면은 프론트엔드다. 국토교통부 데이터나 지도 위치를 가져오는 부분은 백엔드 API다. 자주 쓰는 결과를 빠르게 돌려주기 위한 캐시는 임시 장부와 같다.

서버는 꼭 방 한 칸을 차지한 컴퓨터를 뜻하지 않는다. Vercel처럼 요청이 들어올 때 필요한 코드를 잠깐 실행하는 서버리스 환경도 있다. 서버리스는 서버가 없다는 뜻이 아니라 서버를 직접 설치하고 관리하지 않는다는 뜻이다.

이번 장의 Codex 실행 카드

목표: 내 서비스의 프론트엔드·백엔드·서버·DB·외부 API를 그림으로 나눈다. 먼저 1페이지 기획서와 핵심 사용자 흐름을 확인해줘. 실제로 필요 없는 구성요소를 만들지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 각 구성요소의 역할과 데이터 이동이 쉬운 말로 설명된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 입력·계산·저장·외부 연동이 어디에서 일어날지 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “최신 아키텍처로 알아서 복잡하게 만들어줘”

직접 확인: 비밀번호가 브라우저에 가지 않는가; 데이터 저장 위치를 설명할 수 있는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 내 서비스의 프론트엔드·백엔드·서버·DB·외부 API를 그림으로 나눈다. 완료되면 각 구성요소의 역할과 데이터 이동이 쉬운 말로 설명된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

4장. 폴더·파일·터미널·코드가 실제로 의미하는 것

프로젝트는 특별한 상자가 아니다. 내 컴퓨터에 있는 하나의 폴더다. 그 안에 화면 파일, 설정 파일, 이미지와 설명 문서가 들어 있다.

`app/page.tsx` 같은 표기를 보면 겁먹기 쉽다. /는 폴더 안쪽으로 들어간다는 뜻이다. `app` 폴더 안의 `page.tsx` 파일이라고 읽으면 된다. 확장자 `.tsx`는 TypeScript와 화면 문법을 함께 쓰는 파일이라는 표시다.

터미널은 마우스 대신 글자로 컴퓨터에 명령하는 창이다. 다음 명령은 각각 이런 뜻이다.

```
pwd
ls
cd my-service
npm install
npm run dev
```

- `pwd`: 지금 어느 폴더에 있는지 보여준다.
- `ls`: 현재 폴더의 파일 목록을 보여준다.
- `cd my-service`: `my-service` 폴더로 이동한다.
- `npm install`: 프로젝트가 필요로 하는 도구를 내려받는다.
- `npm run dev`: 개발용 서비스를 내 컴퓨터에서 실행한다.

초보자가 모든 명령을 외울 필요는 없다. 대신 Codex가 명령을 실행하기 전에 무엇을 하려는지 설명하게 하고, 실행 뒤에는 결과를 요약하게 하면 된다.

Codex에게 묻는 문장

이 프로젝트의 폴더 구조를 IT 초보자에게 설명해줘. 각 주요 폴더가 서비스의 어느 부분인지 비유와 함께 알려줘. 아직 파일을 수정하지 마.

이번 장의 Codex 실행 카드

목표: 현재 프로젝트 폴더와 주요 파일을 이해한다. 먼저 현재 작업 폴더를 확인해줘. 파일을 수정하거나 삭제하지 않고 조사만 한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 폴더 지도와 실행 명령, 중요한 파일 다섯 개가 정리된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 모르는 확장자와 폴더 이름을 그대로 말한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “필요 없어 보이는 파일은 전부 정리해줘”

직접 확인: Codex가 확인한 작업 경로가 내 서비스 폴더와 같은가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 현재 프로젝트 폴더와 주요 파일을 이해한다. 완료되면 폴더 지도와 실행 명령, 중요한 파일 다섯 개가 정리된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

5장. API와 환경변수, 키와 토큰을 이해하기

API는 다른 서비스에 부탁을 보내는 약속이다. 지도 서비스에 주소를 보내 좌표를 받고, 결제 서비스에 주문 정보를 보내 결제를 승인받고, Threads에 글을 보내 게시하는 일이 모두 API 호출이다.

API 키는 누가 요청했는지 식별하는 번호다. 토큰은 일정한 권한을 가진 임시 출입증에 가깝다. 비밀 키는 주방 안쪽에서만 써야 하는 마스터키다. 브라우저 화면 코드에 비밀 키를 넣으면 방문자가 볼 수 있으므로 안 된다.

환경변수는 키와 설정을 코드 밖에 두는 방법이다. 보통 로컬에서는 `.env.local`, Vercel에서는 Settings의 Environment Variables에 저장한다.

```
NEXT_PUBLIC_SITE_URL=https://example.com
SUPABASE_SERVICE_ROLE_KEY=절대_공개하면_안_되는_값
```

`NEXT_PUBLIC`처럼 공개를 전제로 한 값과 서버에서만 써야 하는 비밀값을 구분해야 한다. 이름에 `PUBLIC`이 붙지 않았다고 자동으로 안전한 것은 아니다. 코드가 브라우저로 전달되는 위치에서 사용하면 노출될 수 있다.

절대 하지 않을 일

- 비밀 키가 들어 있는 `.env.local`을 GitHub에 올리지 않는다.
- 채팅이나 화면 캡처에 실제 토큰을 붙여 넣지 않는다.
- 테스트 키와 운영 키를 섞지 않는다.
- 유출된 키를 단순히 파일에서 지우고 끝내지 않는다. 즉시 폐기하고 새로 발급한다.

이번 장의 Codex 실행 카드

목표: 프로젝트의 API·키·토큰·환경변수를 안전하게 분류한다.
먼저 코드와 .env.example, 배포 문서를 확인해줘. 실제 비밀값은 출력하지 않고 이름과 설정 여부만 본다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 공개값·서버 비밀값·로컬·Preview·Production 표가 생긴다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 어느 외부 서비스 계정까지 만들었는지만 답하고 키 값은 보내지 않는다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “환경변수 값을 전부 화면에 보여줘”

직접 확인: .env*가 Git에서 제외되는가; 유출 키는 폐기했는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 프로젝트의 API·키·토큰·환경변수를 안전하게 분류한다. 완료되면 공개값·서버 비밀값·로컬·Preview·Production 표가 생긴다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

6장. 무료 도구와 유료 도구를 선택하는 기준

처음부터 모든 도구를 유료로 살 필요는 없다. 하지만 무료라는 이유만으로 운영 구조를 복잡하게 만들면 나중에 시간 비용이 더 커진다.

무료 플랜을 선택할 때는 네 가지를 본다.

- 1 사용량 한도를 넘으면 서비스가 멈추는가, 비용이 청구되는가?
- 2 상업적 사용이 허용되는가?
- 3 데이터를 내보내거나 다른 서비스로 옮길 수 있는가?
- 4 로그와 백업을 어느 정도 제공하는가?

소개 페이지는 GitHub Pages로 충분할 수 있지만 GitHub 공식 정책상 온라인 사업, 전자상거래나 SaaS를 주목적으로 한 무료 호스팅 용도로는 적합하지 않다. 회원·결제·서버 기능이 있는 서비스라면 Vercel 같은 애플리케이션 배포 환경을 선택한다.

Supabase 무료 플랜으로 첫 고객을 검증할 수 있지만 데이터와 로그인 권한을 설계하는 책임은 그대로 남는다. AdSense도 코드를 붙였다고 바로 수익이 생기는 것이 아니라 사이트 심사와 독창적인 콘텐츠가 먼저다.

2부. ChatGPT와 Codex를 나의 개발팀으로 만들기

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

이번 장의 Codex 실행 카드

목표: 첫 출시의 무료·유료 도구와 예상 한도를 정한다. 먼저 예상 사용자 수와 기능 목록을 확인해줘. 가격과 한도는 현재 공식 문서를 확인한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 도구별 선택 이유·무료 한도·초과 시 행동이 표로 정리된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 첫 달 사용자·이미지 용량·API 호출 횟수를 모르면 보수적으로 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천 받는다.

피할 요청: “무조건 평생 무료인 조합으로 만들어줘”

직접 확인: 상업 이용이 가능한가; 한도 초과 알림이 있는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 첫 출시의 무료·유료 도구와 예상 한도를 정한다. 완료되면 도구별 선택 이유·무료 한도·초과 시 행동이 표로 정리 된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

7장. ChatGPT와 Codex의 역할을 나누는 법

ChatGPT는 아이디어를 정리하고, 용어를 설명하고, 시장과 문서를 조사하고, 긴 대화를 통해 방향을 잡는 데 좋다. Codex는 실제 폴더에 접근해 파일을 찾고, 코드를 고치고, 명령과 테스트를 실행하고, 변경사항을 검토하는 데 좋다.

처음에는 다음처럼 나누면 쉽다.

단계	ChatGPT	Codex
문제 정의	고객과 문제 정리	기존 메모와 프로젝트에서 근거 찾기
기획	기능 우선순위와 문구	1페이지 기획서를 저장소 문서로 작성
개발	개념과 선택지 설명	파일 생성·수정·실행·테스트
디자인	화면 방향과 문구 검토	실제 화면 구현과 브라우저 확인
출시	체크리스트 작성	빌드·배포 전 검증·배포
운영	지표 해석과 의사결정	로그 조사·버그 수정·자동화

중요한 점은 Codex에게 코드 조각만 달라고 하지 않는 것이다. 실제 프로젝트를 열어 둔 상태에서 결과까지 맡긴다.

이 로그인 오류를 고치는 코드를 알려줘.

보다 다음 요청이 낫다.

현재 프로젝트에서 Google 로그인 후 대시보드로 돌아오지 못하는 원인을 재현하고 찾아줘. 관련 파일과 환경변수 이름을 확인 하되 비밀값은 출력하지 마. 수정한 뒤 lint, 테스트, build를 실행 하고 변경 내용과 남은 위험을 알려줘.

이번 장의 Codex 실행 카드

목표: ChatGPT와 Codex에 맡길 일을 나눈다. 먼저 현재 아이디어와 보유 자료를 확인해줘. 외부 게시·배포·결제 같은 행동은 승인 전 실행하지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 기획·조사·개발·검증·운영 담당표가 생긴다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 내가 직접 결정하고 싶은 부분과 맡기고 싶은 부분을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “네가 다 알아서 끝까지 해”

직접 확인: 중요한 결정의 최종 책임자가 나로 남아 있는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 ChatGPT와 Codex에 맡길 일을 나눈다. 완료되면 기획·조사·개발·검증·운영 담당표가 생긴다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

8장. Codex가 일할 프로젝트 폴더 만들기

가장 먼저 서비스 하나당 폴더 하나를 만든다. 바탕화면에 파일을 흩어 놓지 않는다.

macOS의 예시는 다음과 같다.

```
mkdir -p ~/Documents/GitHub/my-first-service  
cd ~/Documents/GitHub/my-first-service
```

Windows에서는 문서 폴더 아래에 **GitHub\my-first-service** 폴더를 만들어도 된다. 폴더 이름은 영문 소문자와 하이픈을 권한다. 공백이나 한글이 항상 문제를 일으키는 것은 아니지만 일부 명령과 배포 도구에서 경로를 다루기 어려워질 수 있다.

Codex 앱에서 이 폴더를 프로젝트로 연다. 이 순간부터 Codex의 작업 범위는 '내 컴퓨터 전체'가 아니라 이 서비스 폴더가 된다. 처음 요청은 개발이 아니라 조사로 시작한다.

이 폴더가 비어 있는지 확인해줘. 비어 있다면 아직 파일을 만들지 말고, 초보자가 Next.js 서비스 개발을 시작하기 위해 설치 여부를 확인해야 할 Git, Node.js, npm 항목과 확인 명령을 설명해줘.

Codex가 `node -v`, `npm -v`, `git --version`을 확인하도록 한다. 설치되지 않은 도구가 있다면 공식 설치 경로를 안내하게 한다. 암기한 오래된 버전을 고정하지 말고, 프로젝트를 만드는 시점의 공식 지원 버전을 확인한다.

폴더를 잘못 열었을 때 생기는 일

- 다른 프로젝트 파일을 수정할 수 있다.

- `npm install`이 엉뚱한 폴더에 실행된다.
- Git 커밋이 다른 저장소에 기록된다.
- Codex가 필요한 문맥을 찾지 못해 새 파일을 중복으로 만든다.

따라서 매 작업 시작 시 “현재 작업 폴더와 Git 상태를 먼저 확인해줘”라고 요청하는 습관이 좋다.

이번 장의 Codex 실행 카드

목표: 새 서비스 폴더를 만들고 개발 도구 설치 상태를 확인한다.
먼저 만들 폴더의 정확한 위치와 서비스 영문 이름을 확인해줘.
상위 폴더나 다른 프로젝트를 건드리지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 정확한 폴더에서 Node·npm·Git 상태가 확인된다. 마지막으로 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 운영체제와 원하는 폴더 위치를 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “내 컴퓨터에서 필요 없는 개발 파일을 전부 지워줘”

직접 확인: 터미널의 현재 경로가 맞는가; 기존 파일이 보존됐는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 새 서비스 폴더를 만들고 개발 도구 설치 상태를 확인한다. 완료되면 정확한 폴더에서 Node·npm·Git 상태가 확인된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

9장. 좋은 요청의 네 요소: 목표·맥락·제약·완료 조건



Codex 공식 가이드가 원하는 좋은 기본 구조는 목표, 맥락, 제약, 완료 조건이다. 어려운 개발 용어보다 이 네 가지가 더 중요하다.

목표

무엇이 달라져야 하는지 말한다.

방문자가 소득과 자산을 입력하면 아파트 구매 가능 금액을 계산하는 페이지를 만든다.

맥락

참고할 자료와 현재 상황을 말한다.

research.md에 계산 정책이 있고, design/ 폴더의 모바일 화면을 참고한다. 현재는 Next.js 초기 프로젝트만 있다.

제약

지켜야 할 것과 하지 말아야 할 것을 말한다.

계산식은 임의로 만들지 말고 문서 값만 사용한다. 외부 API는 아직 연결하지 않는다. 모바일을 먼저 설계한다.

완료 조건

어디까지 해야 끝인지 말한다.

입력값 검증, 결과 화면, 새로고침 후 초기화가 동작해야 한다. lint와 build가 통과하고 브라우저에서 모바일 화면을 확인해야 한다.

네 요소를 한 번에 완벽하게 쓰지 못해도 괜찮다. 모호한 생각이라면 먼저 질문하게 한다.

아직 아이디어가 흐릿하다. 바로 코딩하지 말고 고객, 문제, 입력, 결과, 저장 여부, 결제 시점에 대해 한 번에 하나씩 질문해줘. 답변이 끝나면 1페이지 기획서와 개발 순서를 제안해줘.

이번 장의 Codex 실행 카드

목표: 내 첫 기능 요청을 목표·맥락·제약·완료 조건으로 다시 쓴다.
먼저 기획서와 참고 화면을 확인해줘. 모호한 부분은 추측하지 말고 질문하게 한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 한 번에 구현 가능한 작업 요청 한 개가 완성된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 고객 행동과 반드시 지킬 정책을 쉬운 말로 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “예쁘고 완벽하게 알아서 만들어줘”

직접 확인: 완료 여부를 눈으로 확인할 문장이 있는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 내 첫 기능 요청을 목표·맥락·제약·완료 조건으로 다시 쓴다. 완료되면 한 번에 구현 가능한 작업 요청 한 개가 완성된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

10장. 계획하고, 만들고, 테스트하고, 검토하는 한 사이클

초보자가 가장 많이 하는 실수는 큰 요청을 한 뒤 결과 화면만 보고 끝내는 것이다. 실제 개발은 다음 네 단계가 한 묶음이다.

- 1 계획: 현재 구조를 조사하고 무엇을 바꿀지 정한다.
- 2 구현: 작은 범위의 파일을 수정한다.
- 3 검증: lint, 테스트, build와 브라우저 확인을 한다.
- 4 검토: 변경사항과 위험을 다시 읽는다.

복잡한 작업은 Codex의 Plan 모드를 사용하거나 “먼저 계획만 세워줘”라고 한다. 계획에서 내가 확인할 것은 코드 문법이 아니다.

- 고객이 원하는 흐름과 같은가?
- 한 번에 너무 많은 기능을 만들지 않는가?
- 기존 기능을 망가뜨릴 가능성이 있는가?
- 실제 서비스의 데이터나 결제에 영향을 주는가?
- 완료를 어떻게 확인할 것인가?

한 기능을 끝내는 표준 요청

먼저 현재 구조와 관련 파일을 조사해 구현 계획을 세워줘. 계획에는 수정 파일, 데이터 흐름, 예외 상황, 테스트 방법을 포함해줘. 계획을 검토한 뒤 구현하고, 관련 테스트와 전체 build를 실행해줘. 마지막에는 사용자가 직접 확인할 세 가지를 알려줘.

한 번에 홈페이지, 로그인, 결제, 관리자, SEO를 모두 만들어 달라고 하지 않는다. 홈페이지가 완성되고 검증되면 커밋한다. 그 다음 로그인으로 넘어간다. 작은 커밋은 문제가 생겼을 때 돌아갈 수 있는 안전망이다.

이번 장의 Codex 실행 카드

목표: 복잡한 기능을 계획·구현·테스트·검토 단계로 실행한다. 먼저 현재 코드와 기능 요구사항을 확인해줘. 계획을 먼저 보고 범위가 맞을 때 구현한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 수정 파일·테스트·브라우저 확인 결과가 남는다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 범위에 포함할 것과 다음으로 미룰 것을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “계획 필요 없으니 한 번에 다 고쳐”

직접 확인: 원래 기능이 유지되는가; 실패한 검사가 숨겨지지 않았는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 복잡한 기능을 계획·구현·테스트·검토 단계로 실행한다. 완료되면 수정 파일·테스트·브라우저 확인 결과가 남는다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

11장. AGENTS.md로 같은 실수를 반복하지 않게 하기

AGENTS.md는 Codex를 위한 프로젝트 사용설명서다. 저장소를 열 때 자동으로 참고할 수 있는 문서이며 폴더 구조, 실행 방법, 검증 명령, 작업 규칙을 적는다.

처음에는 다음 정도면 충분하다.

```
# My Service Agent Guide

## 서비스
- 비개발자가 운영하는 Next.js 서비스다.
- 모바일 화면을 우선한다.

## 주요 폴더
- app/: 페이지와 API
- components/: 재사용 화면
- docs/: 정책과 작업 기록

## 검증
- npm run lint
- npm test
- npm run build

## 규칙
- 실제 계산식은 docs/policy.md만 근거로 사용한다.
- .env 파일과 비밀 키를 출력하거나 커밋하지 않는다.
- 기존 사용자 데이터를 삭제하는 작업은 먼저 확인한다.
```

Codex CLI에서는 `/init` 명령으로 초안을 만들 수 있다. 자동 생성된 문서를 그대로 믿기보다 내 서비스에 맞게 고친다. Codex가 같은 실수를 두 번 했다면 “왜 반복됐는지 회고하고 AGENTS.md에 짧은 규칙을 추가해줘”라고 한다.

마리에카드에서는 UI 변경은 승인 후 진행하고, 기본 검증 순서를 `lint -> test -> build`로 정했다. 아파트구구에서는 배포 전 계산 회귀 테스트와 운영 데이터 경로 확

인이 중요하다. 다니엘의 바이블에서는 읽기 기록과 구독 권한, 앱 심사 규칙을 함께 확인해야 했다. 같은 Codex라도 프로젝트마다 다른 규칙이 필요한 이유다.

기능표보다 역할 지도를 먼저 만든다

AI에게 기능을 길게 나열하기 전에 누가 어떤 판단을 맡을지 정하면 작업이 덜 꼬인다. 혼자 만드는 프로젝트라도 다음 다섯 역할은 분리해서 생각한다.

역할	먼저 답할 질문	남길 결과
기획자	누구의 어떤 문제를 풀 것인가	한 페이지 문제 정의
조사자	어떤 근거와 제약을 확인해야 하는가	출처가 붙은 조사 메모
설계자	화면·데이터·권한이 어떻게 이어지는가	흐름도와 데이터 초안
구현자	이번 작업에서 무엇을 바꿀 것인가	작은 코드 변경과 커밋
검토자	무엇이 깨질 수 있고 어떻게 확인할 것인가	테스트 결과와 남은 위험

한 대화에서 다섯 역할을 차례로 수행할 수도 있고, 복잡한 작업에서는 별도 작업으로 나눌 수도 있다. 중요한 것은 모델의 수가 아니라 역할마다 입력과 완료 조건이 다른 사실이다. 조사자가 확인하지 않은 숫자를 구현자가 임의로 넣지 않고, 구현자가 만든 결과를 검토자가 같은 가정으로 눈감아 주지 않게 한다.

아이디어가 커지면 문서도 다섯 장으로 나눈다. 문제와 고객, 핵심 사용자 흐름, 데이터와 권한, 실행 계획, 완료·검수 기준이다. 이 문서들을 AGENTS.md가 가리키게 하면 긴 프롬프트를 매번 다시 쓰지 않아도 된다. 문서화는 개발 전에 치르는 비용이 아니라 다음 수정에서 문맥을 복구하는 장치다.

현재 기능 목록을 바로 구현하지 마. 먼저 기획자·조사자·설계자·구현자·검토자의 책임으로 나눠줘. 각 역할이 읽을 문서와 만들 결과를 정하고, 확인되지 않은 가정은 질문 목록으로 분리해줘. 내가 승인한 한 기능만 구현한 뒤 다른 역할의 관점으로 다시 검토해줘.

이번 장의 Codex 실행 카드

목표: 내 저장소용 AGENTS.md를 만든다. 먼저 README, package.json, 주요 문서와 실행 명령을 확인해줘. 짧고 실제로 지킬 규칙만 쓴다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 폴더 구조·검증 명령·보안·금지 규칙이 저장된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 반복됐던 실수와 꼭 실행할 테스트를 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “모든 상황을 처리하는 1000줄 규칙을 만들어줘”

직접 확인: 명령이 실제로 실행되는가; 오래된 규칙이 없는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 내 저장소용 AGENTS.md를 만든다. 완료되면 폴더 구조·검증 명령·보안·금지 규칙이 저장된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

12장. 권한·승인·비밀정보를 안전하게 다루기

Codex는 파일을 읽고 수정하며 명령을 실행할 수 있다. 이 힘이 편리한 만큼 작업 범위를 분명히 해야 한다.

초보자는 다음 원칙을 지킨다.

- 처음 보는 저장소에서는 읽기와 조사부터 시작한다.
- 삭제, 데이터베이스 변경, 운영 배포는 대상과 영향을 먼저 확인한다.
- 프로젝트 밖의 파일 접근은 왜 필요한지 확인한다.
- 비밀값 자체를 보여 달라고 하지 말고 “설정 여부와 변수 이름만 확인”하게 한다.
- `rm -rf`, 데이터 전체 삭제, 강제 Git 초기화처럼 되돌리기 어려운 명령은 승인하지 않는다.

삭제가 필요해도 기본값은 복구 가능하게 둔다. “필요 없는 파일을 정리해줘”보다 “삭제 후보와 사용처를 먼저 조사하고, 확실한 파일만 휴지통으로 이동해줘. 저장소 전체·사용자 폴더·운영 데이터는 건드리지 마”라고 요청한다. 대상이 변수나 와일드 카드로만 지정되어 있다면 실행 전에 정확한 파일 목록을 확인한다.

프로젝트가 커지면 역할을 나눌 수 있다. 한 작업은 요구사항과 위험을 검토하고, 다른 작업은 구현하며, 마지막 작업은 테스트와 보안을 검토한다. 중요한 것은 AI 작업자의 수가 아니라 각 역할의 변경 범위와 완료 조건을 문서로 고정하는 것이다.

샌드박스는 Codex가 접근할 수 있는 범위를 제한하는 안전 구역이고, 승인 모드는 어떤 행동 전에 사용자 확인을 받을지 정한다. 처음에는 기본 권한을 유지한다. 작업이 막힌다고 모든 권한을 한꺼번에 열지 않는다.

안전한 환경변수 점검 요청

이 기능에 필요한 환경변수 이름을 코드와 문서에서 찾아줘. 실제 값은 절대 출력하지 말고, 로컬·Preview·Production 각각 설정됐는지만 yes/no로 알려줘. .env*가 Git 추적 대상인지도 확인해줘.

운영 작업 전 확인 문장

이 작업이 운영 데이터, 결제, 사용자 계정에 영향을 주는지 먼저 설명해줘. 읽기 전용 확인을 먼저 하고, 쓰기 작업이 필요하다면 정확한 대상과 되돌리는 방법을 제시한 뒤 멈춰줘.

이번 장의 Codex 실행 카드

목표: Codex의 작업 권한과 위험 행동 기준을 정한다. 먼저 현재 폴더와 Git·DB·배포 연결 상태를 확인해줘. 삭제·운영 쓰기·외부 게시 전에는 정확한 대상을 확인한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 읽기·로컬 쓰기·운영 쓰기별 승인 기준이 문서화된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 운영 데이터와 결제가 있는지 솔직히 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “모든 권한을 열고 묻지 말고 실행해”

직접 확인: 되돌리기 어려운 행동 전에 멈추는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 Codex의 작업 권한과 위험 행동 기준을 정한다. 완료되면 읽기·로컬 쓰기·운영 쓰기별 승인 기준이 문서화된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

13장. 오류 화면을 Codex와 함께 해결하는 법

“안 돼”라는 말만으로는 사람도 Codex도 문제를 찾기 어렵다. 오류를 다음 다섯 항목으로 기록한다.

- 1 어디에서: 주소와 화면
- 2 무엇을 했을 때: 클릭과 입력 순서
- 3 기대한 결과: 원래 보여야 할 것
- 4 실제 결과: 화면 문구와 상태
- 5 증거: 오류 메시지, 콘솔, 네트워크 응답, 발생 시각

예를 들어 “로그인이 안 돼”보다 다음이 좋다.

<https://app.example.com/ko/create>에서 내용을 입력하고 ‘내보내기’를 눌러 Google 로그인을 완료했다. 원래 편집기로 돌아와 입력 내용이 복원되어야 하지만 빈 대시보드로 이동한다. 7월 18일 14:20 KST에 재현했다. 브라우저 주소에는 code=가 잠깐 보였다. 운영 데이터를 수정하지 말고 인증 callback, redirect, 임시 draft 복원 경로를 조사해줘.

Codex에게 가능한 원인 목록만 달라고 하지 말고 재현과 반증을 요청한다.



**세 가지 원인 가설을 세운 뒤 각 가설을 로그와 코드로 반증해줘.
확인되지 않은 추측과 확인된 사실을 구분해줘. 수정 뒤에는 같은
재현 절차와 관련 테스트로 검증해줘.**

화면 오류, 서버 오류, 데이터 오류를 분리하는 습관도 중요하다. 버튼이 안 보이는 것은 프론트엔드일 수 있고, 버튼을 눌렀는데 500 응답이 오는 것은 백엔드일 수 있다. 200 응답인데 잘못된 값이 오는 것은 데이터나 정책 문제일 수 있다.

3부. 첫 서비스를 내 컴퓨터에서 실행하기

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

이번 장의 Codex 실행 카드

목표: 오류를 재현하고 원인을 증거로 찾는다. 먼저 URL, 행동 순서, 기대·실제 결과, 시각, 오류 메시지를 확인해줘. 운영 데이터 수정 없이 읽기 전용 조사부터 한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 원인·수정·재현 테스트·남은 위험이 구분된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 모르면 추측하지 말고 화면과 오류 문구를 그대로 전달한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “에러 안 나게 예외처리로 전부 숨겨줘”

직접 확인: 같은 절차에서 문제가 사라졌는가; 다른 기능은 유지되는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 오류를 재현하고 원인을 증거로 찾는다. 완료되면 원인·수정·재현 테스트·남은 위험이 구분된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

14장. 한 페이지 기획서와 데이터 설계

코드를 만들기 전에 서비스 한 페이지를 먼저 쓴다.

항목	작성할 내용
고객	누가 쓰는가
문제	지금 무엇이 불편한가
입력	사용자가 무엇을 넣는가
결과	무엇을 얻는가
저장	무엇을 다시 불러와야 하는가
결제	언제 무엇에 돈을 내는가
운영	내가 반복해서 해야 하는 일은 무엇인가
첫 성공	30일 안에 어떤 행동이 나오면 계속할 것인가

마리에카드의 핵심 흐름은 정보 입력 -> 미리보기 -> 로그인 -> 발행 쿠폰 확인 -> 공개 링크다. 아파트구구의 핵심 흐름은 재무 정보 입력 -> 계산 -> 결과 -> 추천 정보 확인 -> 공유다.

데이터 설계는 어려운 표를 그리는 것부터 시작하지 않는다. 명사를 찾는다. 마리에카드에는 사용자, 초대장, 사진, 발행 쿠폰이 있다. 아파트구구에는 계산 입력, 계산 결과, 아파트, 거래가 있다. 각 명사에 필요한 항목을 적으면 테이블 초안이 된다.

Codex 기획 요청

아래 아이디어를 한 페이지 기획서로 정리해줘. 이어서 필요한 데이터의 명사와 각 항목을 표로 만들어줘. MVP에서는 로그인과 결제를 제외하고 로컬에서 입력과 결과만 검증한다. 아직 코드는 만들지 마.

이번 장의 Codex 실행 카드

목표: 아이디어를 1페이지 MVP 기획서와 데이터 초안으로 만든다. 먼저 고객 인터뷰와 문제 메모를 확인해줘. 첫 30일에 필요한 기능을 제외한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 고객·문제·입력·결과·저장·결제·운영·성공 기준이 정리된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 실제 고객의 말과 현재 행동으로 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “유명 서비스 기능을 전부 복제해줘”

직접 확인: 필수 기능이 세 개 이하인가; 첫 성공 기준이 행동인가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 아이디어를 1페이지 MVP 기획서와 데이터 초안으로 만든다. 완료되면 고객·문제·입력·결과·저장·결제·운영·성공 기준이 정리된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

15장. Next.js 프로젝트를 만들고 실행하기

이 책의 예시는 React 기반 웹서비스를 만들기 위한 Next.js를 사용한다. Next.js는 화면과 서버 기능을 한 프로젝트 안에서 만들 수 있고 Vercel과 연결하기 쉽다.

빈 폴더에서 Codex에 다음과 같이 요청한다.

현재 공식 문서를 확인해 안정적인 Next.js 프로젝트를 이 폴더에 만들어줘. TypeScript와 App Router를 사용하고, 스타일은 Tailwind CSS를 선택해줘. 설치 과정과 선택 이유를 초보자에게 설명하고, 생성 뒤 개발 서버를 실행해 첫 화면이 열리는지 확인해줘.

일반적으로 프로젝트가 만들어지면 다음 명령으로 실행한다.

```
npm run dev
```

브라우저에서 <http://localhost:3000>을 연다. localhost는 인터넷 주소가 아니라 지금 내 컴퓨터를 가리킨다. 개발 서버를 종료하면 이 주소도 열리지 않는다.

처음 생성된 파일을 모두 이해하려 하지 않는다. Codex에게 다음 네 곳부터 설명하게 한다.

- `package.json`: 필요한 도구와 실행 명령 목록
- `app/page.tsx`: 첫 화면
- `app/layout.tsx`: 모든 화면을 감싸는 공통 틀
- `public/`: 이미지 같은 공개 파일

첫 확인 체크리스트

- 설치 명령이 오류 없이 끝났는가?
- 개발 서버 주소가 열리는가?
- 파일 하나를 바꾸면 화면이 갱신되는가?
- `npm run build`가 통과하는가?

이번 장의 Codex 실행 카드

목표: 안정적인 Next.js 프로젝트를 만들고 로컬에서 실행한다. 먼저 빈 프로젝트 폴더와 현재 공식 문서를 확인해줘. 지원 버전을 확인하고 불필요한 라이브러리를 넣지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 개발 서버와 첫 화면, lint와 build가 동작한다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: TypeScript·모바일 우선 여부를 답하고 모르면 추천을 요청한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “아무 버전이나 설치하고 경고는 무시해”

직접 확인: localhost가 열리는가; 재시작해도 실행되는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 안정적인 Next.js 프로젝트를 만들고 로컬에서 실행한다. 완료되면 개발 서버와 첫 화면, lint와 build가 동작한다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

16장. 화면을 페이지와 컴포넌트로 나누기

페이지는 주소 하나에 대응하는 화면이다. /는 홈, /about은 소개, /dashboard는 대시보드처럼 생각하면 된다. 컴포넌트는 여러 페이지에서 다시 쓰거나 복잡한 화면을 작은 단위로 나눈 조각이다.

초보자는 처음부터 너무 잘게 나누지 않는다. 다음 기준으로 분리한다.

- 두 곳 이상에서 반복해서 쓴다.
- 하나의 독립된 역할과 이름이 있다.
- 파일이 너무 길어져 수정 위치를 찾기 어렵다.
- 별도로 테스트해야 할 계산이나 입력 흐름이다.

마리에카드에서는 초대장 편집기, 미리보기, 공개 초대장 렌더러를 나눴다. 아파트 구구에서는 입력 카드, 계산 결과, 추천 카드, 광고 단위를 나눴다.

화면 구현 요청

docs/product-brief.md를 기준으로 홈과 계산 페이지를 구현해줘. 먼저 페이지와 컴포넌트 구조를 제안하고, 과도하게 쪼개지 마. 모바일 390px를 우선하고 데스크톱에서도 자연스럽게 늘어나게 해줘. 기존 디자인 토큰과 컴포넌트를 먼저 재사용하고 새 색과 간격을 임의로 만들지 마. 로딩·빈 화면·오류·성공 상태를 포함해줘. 실제 정책에 없는 숫자와 후기는 만들지 마. 구현 뒤 390px와 1440px 브라우저에서 직접 확인하고 차이를 보고해 줘.

이미지나 참고 화면이 있다면 파일로 첨부하고 어떤 부분을 참고할지 말한다. “예쁘게”보다 “상단에는 한 문장 가치 제안, 첫 화면 안에 시작 버튼, 결과의 핵심 숫자는 가장 크게”처럼 관찰 가능한 기준을 준다.

원하는 분위기가 말로 잘 설명되지 않으면 무드보드를 함께 준다. 다만 “이 사이트처럼”으로 끝내지 말고 참고할 요소와 복사하지 않을 요소를 구분한다. 예를 들어 “큰 제목과 넓은 여백은 참고하되 색상·문구·페이지 구조는 우리 서비스에 맞게 새로 설계해줘”라고 적는다. 이 한 문장이 무분별한 복제와 화면별 스타일 흔들림을 줄인다.

디자인 결과가 흔해지는 가장 큰 이유는 AI가 창의적이지 않아서가 아니라 요청의 기본값이 너무 넓기 때문이다. 구현 전에 다음 세 가지를 먼저 정한다.

- 1 시각 방향: 무드보드나 참고 화면에서 가져올 분위기와 위계
- 2 공통 규칙: 색, 서체, 간격, 모서리, 그림자, 최대 너비
- 3 금지 규칙: 카드 남용 금지, 첫 화면의 보조 정보 제한, 화면당 핵심 행동 하나

랜딩페이지라면 섹션 목록만 주지 말고 이야기의 순서를 준다. 첫 화면의 약속 -> 사용자의 현재 불편 -> 해결 방식 -> 믿을 근거 -> 행동 요청으로 이어져야 한다. 구현 뒤에는 스크린샷을 다시 보고 “참고 화면을 닮았는가”보다 “우리 고객이 다음 행동을 이해하는가”를 검토한다.

이번 장의 Codex 실행 카드

목표: 기획서를 페이지와 적절한 컴포넌트 구조로 구현한다. 먼저 기획서와 디자인 참고을 확인해줘. 정책에 없는 숫자·후기·기능을 만들지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 모바일 핵심 화면이 구현되고 구조가 과도하게 쪼개지지 않는다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 페이지 주소와 첫 화면에서 해야 할 행동을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “무조건 컴포넌트를 최대한 많이 나눠줘”

직접 확인: 첫 행동이 잘 보이는가; 반복 요소만 재사용되는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 기획서를 페이지와 적절한 컴포넌트 구조로 구현한다. 완료되면 모바일 핵심 화면이 구현되고 구조가 과도하게 쪼개지지 않는다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

17장. 입력·계산·저장·결과의 기본 흐름 만들기

서비스의 가장 작은 흐름은 입력을 받고 규칙을 적용해 결과를 보여주는 것이다. 이 단계에서는 데이터베이스 없이 브라우저 안에서만 동작해도 된다.

예를 들어 예산 계산기라면 다음 순서다.

- 1 사용자 소득과 보유 자금을 입력한다.
- 2 빈 값, 음수, 너무 큰 값을 검사한다.
- 3 정책 문서의 계산식을 적용한다.
- 4 결과와 계산 근거를 보여준다.
- 5 입력을 바꾸면 결과가 다시 계산된다.

계산식은 화면 코드 안에 섞지 말고 별도 함수로 두는 편이 좋다. 그래야 화면 디자인을 바꿔도 계산식은 유지되고, 여러 입력값을 넣어 자동 테스트할 수 있다.

완료 조건 예시

소득 5,000만원, 보유 자금 2억원일 때 정책 문서의 예시 결과와 같아야 한다. 빈 입력은 0으로 계산하지 말고 안내를 보여준다. 계산 함수에는 정상·경계·오류 값 테스트를 추가한다. 계산 결과에는 사용자가 이해할 수 있는 근거 문장을 표시한다.

저장은 세 단계로 확장한다.

- 1단계: 화면을 닫으면 사라지는 상태

- 2단계: 같은 브라우저에 남는 localStorage
- 3단계: 로그인 후 다른 기기에서도 불러오는 데이터베이스

처음부터 3단계로 가지 않아도 된다. 고객이 입력과 결과를 원하는지 먼저 확인한다.

이번 장의 Codex 실행 카드

목표: 입력->검증->계산->결과 흐름을 만든다. 먼저 계산 정책과 예시 입력·정답을 확인해줘. 계산식을 화면과 분리하고 임의 값을 만들지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 정상·경계·오류 입력 테스트가 통과한다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 예시 입력 세 개와 기대 결과를 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “빈 값은 적당히 0으로 처리해”

직접 확인: 잘못된 입력 안내가 이해되는가; 계산 근거가 보이는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 입력->검증->계산->결과 흐름을 만든다. 완료되면 정상·경계·오류 입력 테스트가 통과한다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

18장. 모바일 화면과 접근성까지 확인하기

내 노트북에서 잘 보인다고 서비스가 완성된 것은 아니다. 실제 고객은 작은 휴대폰, 느린 네트워크, 큰 글자 설정, 키보드만 사용하는 환경에 있을 수 있다.

최소한 다음 크기를 확인한다.

- 모바일 390px 전후
- 작은 모바일 320px 전후
- 태블릿 768px 전후
- 데스크톱 1280px 이상

확인할 것은 단순히 줄바꿈이 아니다.

- 버튼이 손가락으로 누르기 충분한가?
- 입력창 이름이 보이고 오류 이유가 구체적인가?
- 색만으로 성공과 실패를 구분하지 않는가?
- 이미지에 대체 설명이 있는가?
- 키보드 Tab으로 이동할 수 있는가?
- 로딩 중인지 멈춘 것인지 알 수 있는가?

Codex가 브라우저를 사용할 수 있다면 실제 화면 캡처를 먼저 보게 한다.

개발 서버를 실행하고 390px, 768px, 1280px에서 홈과 계산 결과를 캡처해 확인해줘. 겹침, 가로 스크롤, 잘린 글자, 너무 작은 버튼, 낮은 대비를 찾아 수정하고 다시 캡처해 비교해줘.

이번 장의 Codex 실행 카드

목표: 모바일·태블릿·데스크톱과 접근성을 점검한다. 먼저 실행 중인 서비스와 핵심 URL을 확인해줘. 캡처를 먼저 보고 증거가 있는 문제만 고친다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 가로 스크롤·겹침·작은 버튼·대비·키보드 이동이 점검된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 주 고객의 기기와 가장 중요한 화면을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “전체 디자인을 더 트렌디하게 바꿔줘”

직접 확인: 390px에서 핵심 행동을 완료할 수 있는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 모바일·태블릿·데스크톱과 접근성을 점검한다. 완료되면 가로 스크롤·겹침·작은 버튼·대비·키보드 이동이 점검된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

19장. 테스트와 빌드가 왜 필요한가

npm run dev에서 화면이 보인다는 사실만으로 배포할 수 있는 것은 아니다.

- lint는 코드에서 흔한 실수와 규칙 위반을 찾는다.
- type check는 잘못된 데이터 형태를 찾는다.
- unit test는 계산 함수처럼 작은 규칙을 검증한다.
- integration test는 여러 부분이 함께 동작하는지 본다.
- E2E test는 실제 브라우저에서 사용자의 전체 흐름을 따라간다.
- build는 배포 가능한 결과물을 만들 수 있는지 확인한다.

마리에카드에서는 출시 전 lint, 132개의 테스트, build, 13개의 Playwright E2E를 통과시켰다. 공개 랜딩, 로그인 전 초안 복원, 첫 발행, 쿠폰 소비 같은 중요한 흐름은 화면을 한 번 본 것으로 끝내지 않았다.

기본 검증 요청

이번 변경과 관련된 테스트를 추가하거나 갱신해줘. lint, test, build를 순서대로 실행하고 실패하면 원인을 고친 뒤 다시 실행해줘. 테스트를 통과시키기 위해 기능 자체를 약화하지 마. 마지막에 실행한 명령과 결과를 표로 알려줘.

테스트가 많다고 무조건 안전한 것은 아니다. 잘못된 기대값을 테스트할 수도 있다. 실제 고객의 핵심 흐름과 돈·권한·데이터를 다루는 경로부터 테스트한다.

4부. GitHub에 기록하고 인터넷에 배포하기

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

이번 장의 Codex 실행 카드

목표: 핵심 기능에 맞는 테스트와 배포 build를 만든다. 먼저 package.json과 핵심 사용자 흐름을 확인해줘. 테스트 통과를 위해 기능을 약화하지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 lint·unit·필요한 E2E·build 결과가 표로 남는다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 돈·권한·데이터와 관련된 가장 중요한 흐름을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “테스트가 귀찮으니 전부 삭제해”

직접 확인: 실패 테스트가 0개인가; 실제 동작도 같은가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 핵심 기능에 맞는 테스트와 배포 build를 만든다. 완료되면 lint·unit·필요한 E2E·build 결과가 표로 남는다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

20장. Git과 GitHub는 같은 것이 아니다

Git은 내 컴퓨터에서 파일의 변경 역사를 관리하는 도구다. GitHub는 Git 저장소를 인터넷에 보관하고 다른 사람이나 배포 서비스와 연결하는 플랫폼이다.

사진에 비유하면 Git은 사진을 찍고 앨범의 순서를 관리하는 방식이고, GitHub는 그 앨범을 안전하게 보관하고 공유하는 온라인 공간이다.

저장소(repository)는 프로젝트 폴더와 변경 이력을 함께 담은 단위다. `git init`을 실행하면 현재 폴더 안에 숨겨진 `.git` 폴더가 생기고 Git이 변경을 추적하기 시작한다.

```
git init
git status
```

`git status`는 지금 바뀐 파일, 아직 추적하지 않는 파일, 다음 커밋에 들어갈 파일을 보여준다. 초보자가 가장 자주 확인할 명령이다.

GitHub 계정을 만들고 빈 저장소를 만든 뒤 로컬 저장소와 연결한다. 주소를 연결하는 이름으로 보통 `origin`을 쓴다.

```
git remote add origin https://github.com/사용자이름/저장소이름.git
git remote -v
```

`origin`은 인터넷 그 자체가 아니라 이 로컬 프로젝트가 바라보는 기본 원격 저장소의 별명이다.

이번 장의 Codex 실행 카드

목표: 현재 폴더를 Git 저장소로 안전하게 초기화하고 GitHub 연결을 준비한다. 먼저 현재 폴더와 Git 상태를 확인해줘. 기존 저장소라면 다시 init하지 않고 .env를 제외한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 저장소 상태·기본 브랜치·원격 연결 계획이 설명된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: GitHub 사용자명과 새 저장소 이름만 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “컴퓨터의 모든 프로젝트를 하나의 저장소로 만들어줘”

직접 확인: 저장소 루트가 정확한가; 비밀 파일이 추적되지 않는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 현재 폴더를 Git 저장소로 안전하게 초기화하고 GitHub 연결을 준비한다. 완료되면 저장소 상태·기본 브랜치·원격 연결 계획이 설명된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

21장. 커밋·푸시·풀·브랜치·PR을 완전히 이해하기



커밋

커밋은 변경사항의 사진 한 장이다. 파일을 저장하는 것과 다르다. 저장은 현재 파일을 바꾸고, 커밋은 선택한 변경사항과 설명을 Git 역사에 남긴다.

```
git add app/page.tsx
git commit -m "feat: 첫 계산 화면 추가"
```

`git add`는 다음 사진에 넣을 변경을 고르는 일이고, `git commit`은 실제 사진을 찍는 일이다.

푸시

푸시는 내 컴퓨터의 커밋을 GitHub로 올리는 일이다.

```
git push origin main
```

파일을 저장했다고 GitHub가 자동으로 바뀌지 않는다. 커밋만 해도 GitHub는 바뀌지 않는다. 푸시해야 원격 저장소에 반영된다.

풀

풀은 GitHub에 있는 새 커밋을 내 컴퓨터로 가져와 합치는 일이다. 여러 컴퓨터에서 일하거나 다른 사람이 수정했다면 작업 전에 확인한다.

```
git pull
```

브랜치

브랜치는 기존 서비스에 바로 영향을 주지 않고 별도의 작업 줄을 만드는 방법이다.

```
git switch -c feat/google-login
```

로그인 기능이 끝나면 main에 합친다. main은 보통 운영에 배포되는 기준 브랜치다.

Pull Request

PR은 “이 브랜치의 변경을 main에 합쳐도 되는지 검토해 달라”는 제안이다. 혼자 개발해도 PR을 쓰면 변경 설명, 테스트 결과와 미리보기 주소가 한곳에 남는다.

말	실제 의미	비유
save	파일의 현재 내용 저장	문서 저장
add	커밋에 넣을 변경 선택	사진에 담을 사람 모으기
commit	로컬 역사에 스냅샷 저장	사진 촬영
push	커밋을 GitHub에 전송	온라인 앨범 업로드
pull	GitHub 변경을 로컬로 가져오기	최신 앨범 내려받기
branch	독립된 작업 줄 만들기	원본 복사본에서 편집
PR	합치기 전 검토 요청	최종본 반영 제안

이번 장의 Codex 실행 카드

목표: 작은 변경을 add·commit·branch·push·PR 흐름으로 연습한다. 먼저 현재 Git 상태와 연습할 파일 하나를 확인해줘. main 직접 푸시보다 연습 브랜치를 사용한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 하나의 명확한 커밋과 PR용 설명이 만들어진다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 이번 변경에 포함할 파일을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “변경된 파일 전부 강제로 커밋하고 main에 푸시해”

직접 확인: 커밋 내용이 한 목적뿐인가; 원격 브랜치가 맞는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 작은 변경을 add·commit·branch·push·PR 흐름으로 연습한다. 완료되면 하나의 명확한 커밋과 PR용 설명이 만들어진다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

22장. Codex에게 안전하게 커밋과 푸시를 맡기기

Codex는 Git 상태와 변경 내용을 읽고 의미 있는 단위로 커밋할 수 있다. 하지만 “전부 커밋해”는 위험하다. 이전부터 있던 내 파일, .env, 임시 이미지까지 들어갈 수 있기 때문이다.

안전한 순서는 다음과 같다.

- 1 현재 브랜치와 `git status`를 확인한다.
- 2 이번 작업에서 Codex가 바꾼 파일과 기존 변경을 구분한다.
- 3 diff로 실제 변경을 검토한다.
- 4 `lint`, `test`, `build`를 실행한다.
- 5 관련 파일만 선택해 커밋한다.
- 6 푸시 대상 브랜치와 원격 저장소를 확인한다.
- 7 푸시한다.

권장 요청

현재 Git 상태를 확인해 이번 로그인 작업과 관련된 변경만 정리해줘. 내가 만들었던 기존 변경은 건드리지 마. 비밀번호와 .env가 포함되지 않았는지 검사하고 diff를 요약해줘. `lint`, `test`, `build`가 모두 통과하면 `feat: Google 로그인 흐름` 추가로 커밋해줘. 푸시는 아직 하지 마.

검토 후 이어서 말한다.

커밋 해시와 현재 브랜치, 연결된 원격 저장소를 확인한 뒤 이 브랜치만 push해줘. main에 직접 합치거나 Production 배포를 따로 실행하지 마.

커밋 해시는 각 커밋의 고유 번호다. 문제가 생겼을 때 “언제부터 잘못됐는지” 비교할 수 있다.

이번 장의 Codex 실행 카드

목표: 이번 작업만 검증해 안전하게 커밋하고 선택적으로 푸시한다. 먼저 `git status`, `diff`, 테스트 명령을 확인해줘. 기존 사용자 변경과 비밀번호를 포함하지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 검증된 관련 파일만 커밋되고 대상 브랜치가 확인된다. 마지막으로 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 푸시까지 허용할지와 브랜치 이름을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “문지 말고 커밋·푸시·배포까지 전부 해”

직접 확인: 커밋 해시·브랜치·원격 저장소가 정확한가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 이번 작업만 검증해 안전하게 커밋하고 선택적으로 푸시한다. 완료되면 검증된 관련 파일만 커밋되고 대상 브랜치가 확인된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

23장. 배포란 내 컴퓨터의 결과물을 인터넷에 올리는 일



로컬 개발 서버는 내 컴퓨터에서만 열린다. 배포는 소스 코드를 빌드해 인터넷에서 계속 접근할 수 있는 환경에 올리는 과정이다.

Vercel 기준으로는 보통 다음 일이 일어난다.

- 1 GitHub의 특정 커밋을 가져온다.
- 2 필요한 패키지를 설치한다.
- 3 `npm run build`를 실행한다.
- 4 정적 파일과 서버 함수를 배포한다.
- 5 고유한 URL을 만든다.
- 6 Production이면 실제 도메인에 연결한다.

배포에는 세 환경이 있다.

- Local: 내 컴퓨터에서 개발하는 환경
- Preview: 고객에게 공개하기 전 인터넷에서 확인하는 환경

- Production: 실제 고객이 사용하는 운영 환경

Preview에서 잘된다고 Production에서도 무조건 잘되는 것은 아니다. 환경변수, 도메인, 데이터베이스, OAuth callback 주소가 다를 수 있다.

이번 장의 Codex 실행 카드

목표: Local·Preview·Production 배포 흐름을 설계한다. 먼저 GitHub 저장소와 build 명령을 확인해줘. 운영 배포 전에 Preview를 통과한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 각 환경의 URL·데이터·환경변수 차이가 표로 정리된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 실제 고객용 도메인과 아직 테스트 중인 기능을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “로컬이 되니까 바로 운영 배포해”

직접 확인: Preview가 운영 데이터에 쓰지 않는가; build가 통과하는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 Local·Preview·Production 배포 흐름을 설계한다. 완료되면 각 환경의 URL·데이터·환경변수 차이가 표로 정리된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

24장. Vercel과 GitHub Pages의 차이

GitHub Pages는 저장소의 HTML, CSS, JavaScript 같은 정적 파일을 공개하는 서비스다. 포트폴리오, 문서, 단순 랜딩에 적합하다. 서버에서 비밀 키를 사용하는 코드나 일반적인 데이터베이스 백엔드를 직접 실행하는 용도는 아니다.

Vercel은 Next.js 같은 웹 애플리케이션을 빌드하고 정적 화면과 서버 함수를 함께 배포할 수 있다. GitHub에 푸시할 때마다 Preview를 만들고, Production 브랜치에 합치면 운영 배포를 만들 수 있다.

기준	GitHub Pages	Vercel
적합한 대상	문서·포트폴리오·정적 랜딩	Next.js 웹서비스·API
서버 코드	일반적인 서버측 언어 실행 불가	서버리스 함수 지원
비밀 환경변수	브라우저 정적 파일에 숨길 수 없음	서버 실행 환경에 설정 가능
Preview	별도 구성 필요	브랜치·PR별 자동 생성
상업 서비스	공식 사용 제한 확인 필요	플랜과 사용량 정책 내에서 가능
데이터베이스	외부 서비스 별도 연결	Supabase 등 외부 DB 연결

로그인, 결제, 비밀 API 키가 하나라도 있다면 이 책에서는 Vercel을 기본으로 선택한다.

이번 장의 Codex 실행 카드

목표: 내 서비스에 Vercel과 GitHub Pages 중 맞는 배포 방식을 고른다. 먼저 로그인·저장·결제·서버 API 요구를 확인해줘. 상업 이용과 현재 공식 제한을 확인한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 선택 이유와 포기하는 기능이 명확하다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 사용자 데이터와 비밀번호 키가 필요한지 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “무조건 무료인 GitHub Pages에 결제 서비스까지 올려줘”

직접 확인: 선택한 호스팅이 서버 기능과 약관을 지원하는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 내 서비스에 Vercel과 GitHub Pages 중 맞는 배포 방식을 고른다. 완료되면 선택 이유와 포기하는 기능이 명확하다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

25장. Vercel 배포와 도메인·환경변수 연결

첫 배포

- 1 GitHub에 저장소를 만든다.
- 2 로컬 커밋을 main에 푸시한다.
- 3 Vercel에서 New Project를 누른다.
- 4 GitHub 저장소를 Import한다.
- 5 Framework Preset이 Next.js인지 확인한다.
- 6 필요한 환경변수를 입력한다.
- 7 Deploy를 누른다.

연결 뒤에는 일반적으로 main 푸시가 Production 배포를 만들고 다른 브랜치 푸시는 Preview 배포를 만든다. Vercel 공식 문서에 따르면 Git 저장소의 커밋이나 PR이 새 배포를 자동으로 트리거할 수 있다.

환경변수

Vercel에서 환경변수는 Development, Preview, Production별로 다르게 설정할 수 있다. 값을 추가하거나 바꾼 뒤에는 기존 배포에 자동 적용되지 않으므로 새로 배포해야 한다.

도메인

처음에는 프로젝트이름.vercel.app 주소가 생긴다. 실제 도메인을 구매했다면 Vercel의 Domains에 추가하고 안내된 DNS 레코드를 도메인 업체에 설정한다. `www.example.com`과 `example.com` 중 하나를 대표 주소로 정하고 다른 주소는 대표 주소로 이동시킨다.

배포 실패 시 Codex 요청

Vercel 배포 로그의 첫 번째 실제 오류를 찾아 로컬에서 재현해 줘. 경고와 원인을 구분하고, 패키지 버전을 무작정 낮추지 마. 로컬 build를 통과시킨 뒤 변경사항과 Vercel에서 다시 확인할 환경변수 이름을 알려줘. 실제 비밀값은 출력하지 마.

5부. 회원과 데이터베이스 만들기

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

이번 장의 Codex 실행 카드

목표: GitHub 저장소를 Vercel에 연결하고 Preview를 배포한다. 먼저 GitHub 저장소·build·필요 환경변수 이름을 확인해줘. 비밀값을 코드나 로그에 노출하지 않고 Production은 승인 전 배포하지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 Preview URL에서 핵심 흐름과 build가 확인된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 환경변수의 실제 값은 Vercel 화면에 직접 입력하고 설정 여부만 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “내 시크릿 키를 코드에 넣어서 배포해”

직접 확인: Preview와 Production 변수가 구분되는가; 도메인 HTTPS가 정상인가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 GitHub 저장소를 Vercel에 연결하고 Preview를 배포한다. 완료되면 Preview URL에서 핵심 흐름과 build가 확인된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

26장. Supabase가 해주는 네 가지 일



Supabase 프로젝트를 만들면 중심에 PostgreSQL 데이터베이스가 생긴다. 그 위에 회원 인증(Auth), 파일 저장(Storage), 실시간 기능과 서버측 함수 같은 도구가 연결된다.

초보자는 네 가지만 기억하면 된다.

- 1 Database: 글과 숫자 같은 구조화된 데이터를 저장한다.
- 2 Auth: 회원가입, 로그인, 세션을 관리한다.
- 3 Storage: 사진과 파일을 저장한다.
- 4 API: 데이터에 접근할 수 있는 통로를 자동으로 제공한다.

마리에카드에서는 사용자와 초대장, 쿠폰을 데이터베이스에 두고 초대장 사진을 Storage에 둔다. Google 로그인은 Auth가 처리한다.

프로젝트 연결 순서

- 1 Supabase에서 새 프로젝트를 만든다.
- 2 Project URL과 공개용 키를 확인한다.

- 3 로컬 `.env.local`에 공개 설정을 넣는다.
- 4 서버 전용 키가 필요하다면 서버 코드에서만 사용한다.
- 5 Vercel의 Preview와 Production에도 각각 설정한다.
- 6 연결 확인 페이지나 테스트로 실제 읽기·쓰기를 검증한다.

Supabase의 `service_role` 키는 권한 검사를 우회할 수 있는 강한 키다. 브라우저로 전달하면 안 된다.

이번 장의 Codex 실행 카드

목표: Supabase 테스트 프로젝트와 앱을 연결한다. 먼저 데이터 초안과 Supabase 공식 문서를 확인해줘. `service_role` 키는 서버에서만 사용하고 운영 DB를 건드리지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 연결 상태와 Database·Auth·Storage 역할이 확인된다. 마지막으로는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 프로젝트 URL과 공개 키는 환경변수로 직접 설정했다고 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “모든 권한을 끄고 일단 연결해”

직접 확인: 브라우저 번들에 관리자 키가 없는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 Supabase 테스트 프로젝트와 앱을 연결한다. 완료되면 연결 상태와 Database·Auth·Storage 역할이 확인된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

27장. 테이블과 행, 열, 관계를 초보자 눈높이로 이해하기

스프레드시트를 떠올리면 쉽다. 테이블은 시트, 행은 한 건의 기록, 열은 항목이다.

invitations 테이블의 예시는 다음과 같다.

id	user_id	title	status	created_at
101	user-a	철완과 마리의 결혼식	draft	2026-07-01

user_id는 이 초대장이 어느 사용자에게 속하는지 연결한다. 이런 연결을 관계라고 한다.

상태값은 자유로운 문장보다 정해진 값으로 관리한다. 예를 들어 draft, published, expired다. 마리에카드에서는 공개 링크 노출과 쿠폰 요구 조건을 status === "published"로 통일하지 않았을 때 회귀가 생겼다. 같은 개념을 여러 방식으로 판단하면 오류가 생긴다.

RLS

Row Level Security는 각 행을 누가 읽고 쓸 수 있는지 정하는 데이터베이스 규칙이다. “로그인했으니 모든 초대장을 볼 수 있다”가 아니라 “로그인한 사용자는 user_id가 자신의 ID인 행만 볼 수 있다”처럼 제한한다.

Codex 요청

이 기획서의 데이터를 Supabase 테이블로 설계해줘. 각 테이블의 목적, 열, 데이터 형식, 필수 여부, 관계를 초보자용 표로 먼저 설명해줘. 사용자별 데이터에는 RLS가 필요하다. SQL migration을 만든 뒤 로컬 또는 테스트 프로젝트에서 검증할 방법을 제시해줘. 운영 DB에는 적용하지 마.

이번 장의 Codex 실행 카드

목표: Supabase 테이블·관계·RLS migration을 설계한다. 먼저 1페이지 기획서와 데이터 명사를 확인해줘. 사용자별 데이터는 기본 비공개이며 운영 적용 전 검토한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 SQL migration·관계도·정상/차단 테스트가 생긴다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 누가 어떤 데이터를 읽고 바꿀 수 있는지 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “RLS가 복잡하니 꺼줘”

직접 확인: A 사용자가 B 사용자 데이터를 볼 수 없는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 Supabase 테이블·관계·RLS migration을 설계한다. 완료되면 SQL migration·관계도·정상/차단 테스트가 생긴다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

28장. 회원가입과 Google·Kakao 로 그인 붙이기

로그인은 버튼 하나보다 흐름이 중요하다.

- 1 사용자가 로그인 버튼을 누른다.
- 2 Google이나 Kakao 로그인 화면으로 이동한다.
- 3 동의를 끝나면 미리 등록한 callback 주소로 돌아온다.
- 4 서비스가 전달받은 코드를 세션으로 바꾼다.
- 5 원래 하려던 화면으로 돌려보낸다.
- 6 새로고침해도 로그인 상태를 복원한다.

OAuth는 비밀번호를 우리 서비스가 직접 받지 않고 외부 로그인 제공자의 인증 결과를 사용하는 표준 흐름이다.

초보자가 자주 놓치는 것은 URL이다. 로컬 callback, Vercel Preview, 실제 도메인의 callback을 각각 등록해야 할 수 있다. Production 주소가 바뀌면 로그인 제공자 설정도 함께 확인한다.

마리에카드는 사용자가 먼저 초대장 내용을 입력하고 마지막 발행 순간에 로그인하는 login-later 구조를 사용한다. 텍스트는 localStorage, 파일은 IndexedDB에 임시 저장하고 로그인 뒤 복원했다. 회원 수를 늘리기 위해 처음부터 로그인시키는 것보다 핵심 가치를 먼저 경험하게 한 선택이다.

로그인 완료 조건

- 로그인 성공 후 원래 화면으로 돌아온다.
- 취소와 실패 메시지가 다르다.

- 새로고침해도 세션이 유지된다.
- 로그아웃하면 보호 화면에 접근할 수 없다.
- 다른 사용자의 데이터에 접근할 수 없다.
- callback URL과 환경변수가 로컬·Preview·Production에 맞다.

이번 장의 Codex 실행 카드

목표: Google 또는 Kakao OAuth 로그인을 안전하게 연결한다. 먼저 로컬·Preview·Production URL과 로그인 후 목적지를 확인해줘. callback과 취소·실패 흐름을 포함하고 비밀번호를 직접 저장하지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 로그인·새로고침·로그아웃·보호 경로 테스트가 통과한다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 어떤 로그인 제공자와 로그인 후 돌아갈 화면을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “callback은 아무 URL이나 허용해”

직접 확인: 다른 사용자의 데이터가 보이지 않는가; 원래 작업이 복원되는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 Google 또는 Kakao OAuth 로그인을 안전하게 연결한다. 완료되면 로그인·새로고침·로그아웃·보호 경로 테스트가 통과한다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

29장. 이미지 업로드와 Storage 사용 하기

사진을 데이터베이스 한 칸에 직접 넣기보다 Storage에 파일을 올리고 데이터베이스에는 파일 경로와 설명을 저장한다.

업로드에는 제한이 필요하다.

- 허용 확장자와 MIME type
- 최대 파일 크기
- 한 사용자의 파일 개수
- 공개 파일과 비공개 파일의 구분
- 삭제할 때 연결된 데이터 정리
- 파일 이름 충돌 방지

화면에서 `accept="image/*"`를 설정하는 것만으로는 부족하다. 서버에서도 다시 검사해야 한다. 브라우저 제한은 사용자가 우회할 수 있기 때문이다.

마리에카드에서는 로그인 전 선택한 파일도 실제 서버 업로드와 같은 확장자, MIME, 크기 제한을 통과시켰다. 임시 미리보기만 된다고 느슨하게 만들면 로그인 뒤 업로드 시점에 갑자기 실패한다.

요청 예시

초대장 이미지 업로드를 Supabase Storage로 구현해줘. 브라우저와 서버 양쪽에서 JPEG, PNG, WebP만 허용하고 파일당 10MB로 제한해줘. 사용자 ID 아래에 충돌 없는 경로로 저장하고, 다른 사용자가 수정하거나 삭제하지 못하게 정책을 설계해줘. 실패 시 사용자가 이해할 한국어 메시지를 보여주고 테스트를 추가해줘.

이번 장의 Codex 실행 카드

목표: Supabase Storage 이미지 업로드를 구현한다. 먼저 허용 형식·크기·사용자별 경로를 확인해줘. 브라우저와 서버에서 모두 검사하고 소유자 권한을 제한한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 업로드·미리보기·삭제·실패 테스트가 동작한다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 사진 종류와 최대 크기, 공개 여부를 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “모든 파일 형식과 크기를 허용해”

직접 확인: 다른 사용자가 파일을 덮어쓰거나 삭제할 수 없는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 Supabase Storage 이미지 업로드를 구현한다. 완료되면 업로드·미리보기·삭제·실패 테스트가 동작한다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

30장. 관리자 페이지와 권한 설계

관리자 페이지는 사용자 화면보다 늦게 예쁘게 만들어도 되지만 운영 기능은 일찍 필요하다. 문의가 왔을 때 사용자의 상태, 주문, 발행 결과를 안전하게 확인할 수 있어야 한다.

관리자 기능의 기본은 다음과 같다.

- 사용자와 콘텐츠 검색
- 상태 필터
- 상세 기록과 변경 이력
- 실패한 작업 재시도
- 쿠폰 또는 권한 발급
- 위험한 작업의 재확인

관리자 화면 주소를 숨기는 것은 보안이 아니다. 서버에서 관리자 권한을 다시 확인해야 한다. 사용자가 브라우저 요청을 직접 만들 수 있기 때문이다.

마리에카드 쿠폰 시스템에서는 생성, 삭제, 소비, 롤백, 거부 시도를 감사 로그에 남겼다. 이미 사용된 쿠폰 삭제는 **409 Conflict**로 막았다. 무엇을 할 수 있는지만큼 누가 언제 했는지를 기록하는 것이 중요하다.

이번 장의 Codex 실행 카드

목표: 운영에 필요한 최소 관리자 페이지를 만든다. 먼저 문의 대응에 필요한 검색·상태·이력을 확인해줘. 주소 숨김이 아니라 서버 권한으로 막고 위험 작업을 기록한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 관리자 인증·검색·필터·감사 로그가 검증된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 누가 관리자이며 어떤 작업만 허용할지 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “관리자 URL만 어렵게 만들면 보안은 됐어”

직접 확인: 일반 계정으로 관리자 API가 차단되는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 운영에 필요한 최소 관리자 페이지를 만든다. 완료되면 관리자 인증·검색·필터·감사 로그가 검증된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

31장. 개인정보와 보안을 나중으로 미루지 않기

서비스가 수집하는 정보가 적어도 개인정보 처리방침, 이용약관, 문의 경로가 필요할 수 있다. 법률 문서는 사업과 국가에 따라 달라지므로 AI 초안을 그대로 공개하지 말고 전문가 또는 관련 공식 안내를 확인한다.

기술적으로는 다음을 지킨다.

- 필요한 정보만 수집한다.
- 비밀번호를 직접 저장하지 않는다.
- 관리자 키를 브라우저에 보내지 않는다.
- 사용자별 데이터에 RLS를 적용한다.
- 계정 삭제 시 데이터와 파일이 어떻게 처리되는지 정한다.
- 로그에 토큰, 주민번호, 결제정보를 남기지 않는다.
- 의존성 보안 경고를 확인하되 자동 강제 업데이트로 서비스를 깨뜨리지 않는다.

마리에카드에서는 계정 삭제 때 작업공간 참조, Storage 파일, 인증 사용자를 함께 정리하는 흐름을 만들었다. “테이블 한 행 삭제”가 계정 삭제의 전부가 아니다.

6부. 외부 서비스 연결하기

경험을 실행 가능한 원칙으로 바꾸는 실천 파트

이번 장의 Codex 실행 카드

목표: 수집 데이터·삭제·보안·법적 문서 준비 항목을 점검한다. 먼저 DB schema, Storage, 로그, 약관 초안을 확인해줘. 법률 판단을 AI가 확정하지 않고 최신 공식·전문가 검토 항목을 표시한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 개인정보 흐름과 삭제 체크리스트가 생긴다. 마지막으로 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 수집 목적과 보유 기간을 모르면 미정이라고 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “다른 서비스 약관을 그대로 복사해”

직접 확인: 필요 없는 개인정보를 수집하지 않는가; 계정 삭제가 실제로 되는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 수집 데이터·삭제·보안·법적 문서 준비 항목을 점검한다. 완료되면 개인정보 흐름과 삭제 체크리스트가 생긴다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

32장. 결제 붙이기: 상품·주문·승인·웹훅



결제 화면을 띄우는 것과 돈을 안전하게 받는 것은 다르다. 기본 흐름은 다음과 같다.

- 1 서버가 상품 가격으로 주문을 만든다.
- 2 브라우저가 결제 위젯 또는 결제창을 연다.
- 3 사용자가 인증한다.
- 4 성공 주소로 돌아오면 서버가 금액과 주문을 다시 확인한다.
- 5 서버의 비밀 키로 결제를 승인한다.
- 6 결제 결과를 데이터베이스에 저장한다.
- 7 웹훅으로 취소, 환불, 지연된 상태 변경을 반영한다.

웹훅은 결제사가 우리 서버에 “결제 상태가 바뀌었다”고 보내는 서버 간 알림이다. 사용자가 성공 페이지를 닫아도 결제는 완료될 수 있으므로 화면 이동만 믿으면 안 된다.

한국 서비스라면 Toss Payments 결제위젯 같은 PG를 검토할 수 있다. 공식 문서의 테스트 키로 요청, redirect, 서버 승인까지 먼저 확인한다. 해외 결제나 구독이 필요하면 Stripe 같은 선택지도 있다. 사업자 계약, 지원 결제수단, 수수료, 정산, 환불과 세금 요건을 비교한다.

반드시 서버에서 확인할 것

- 주문 ID가 실제 존재하는가?
- 결제 금액이 서버의 상품 가격과 같은가?
- 이미 승인된 주문을 중복 승인하지 않는가?
- 결제 사용자와 주문 소유자가 같은가?
- 웹훅 서명이 진짜 결제사에서 온 것인지 검증되는가?

Codex 요청

Toss Payments 공식 최신 문서를 기준으로 테스트 결제부터 구현해줘. 클라이언트 키와 시크릿 키의 위치를 구분하고, 금액은 브라우저 값을 믿지 말고 서버 상품 정보로 검증해줘. 주문·결제 테이블, 성공·실패 화면, 승인 API, 웹훅, 중복 승인 방지와 테스트를 포함해줘. 운영 키는 사용하지 말고 실제 결제를 실행하지 마.

이번 장의 Codex 실행 카드

목표: 테스트 결제로 상품·주문·승인·웹훅 흐름을 구현한다. 먼저 현재 상품·가격·환불 정책과 결제사 공식 문서를 확인해줘. 운영 키와 실제 결제를 쓰지 않고 금액을 서버에서 검증한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 테스트 성공·취소·실패·중복·웹훅 검증이 통과한다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 일회성·구독 여부와 고정 가격을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “브라우저가 보낸 금액 그대로 승인해”

직접 확인: 중복 결제가 막히는가; 결제 성공과 주문 상태가 일치하는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 테스트 결제로 상품·주문·승인·웹훅 흐름을 구현한다. 완료되면 테스트 성공·취소·실패·중복·웹훅 검증이 통과한다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

33장. Google·Naver·Kakao 지도를 선택하고 연결하기

지도 기능은 크게 세 단계다.

- 1 주소를 좌표로 바꾸는 geocoding
- 2 지도를 화면에 표시하는 map
- 3 장소 검색, 길찾기, 외부 지도 열기

지도 제공자를 고를 때는 고객 지역, 데이터 품질, 가격과 할당량, 웹·iOS·Android 지원, 약관, 키 제한 방법을 본다.

- Google Maps: 전 세계 서비스와 풍부한 장소 데이터에 강점이 있다. 운영용 표준 API 키에는 결제 설정이 필요할 수 있고 웹사이트와 API 제한을 설정해야 한다.
- Naver Maps: 한국 주소와 네이버 생태계를 중심으로 검토할 수 있다.
- Kakao Maps: JavaScript, Android, iOS SDK와 로컬 검색을 함께 검토할 수 있다. 새 앱은 지도 기능 활성화와 플랫폼 키 설정이 필요하다.

동적인 지도가 항상 답은 아니다. 아파트구구는 64px 높이의 작은 위치 미리보기 요구에 Naver Static Map 이미지와 외부 지도 열기 CTA를 선택했다. 정적 export 와 iOS WebView 환경에서 무거운 JS 지도를 넣는 것보다 안정적이었다.

키 보안

브라우저에서 쓰는 지도 키는 완전히 숨길 수 없는 경우가 많다. 대신 허용 도메인, 앱 번들 ID, 호출 API를 제한한다. 서버 키는 서버에서만 사용한다.

구현 요청

이 서비스는 한국 아파트 주소의 위치 미리보기만 필요하다.
Google, Naver, Kakao의 동적 지도와 정적 지도 방식을 비용,
키 제한, 구현 복잡도, 모바일 성능으로 비교해줘. 공식 문서를 확
인하고 하나를 추천해줘. 선택 후 주소->좌표->미리보기->외부
지도 열기 흐름을 구현하고 실패 시 주소 텍스트는 유지해줘.

이번 장의 Codex 실행 카드

목표: Google·Naver·Kakao 지도 중 맞는 제공자를 고르고 위
치 흐름을 만든다. 먼저 대상 국가·주소·동적/정적 지도 요구를
확인해줘. 공식 가격·할당량을 확인하고 키에 도메인 제한을 건다.
현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해
줘. 완료 조건은 주소->좌표->지도->실패 fallback이 검증된다.
마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구
분해 알려줘.

Codex가 질문하면: 한국 중심인지 글로벌인지와 지도에서 할 행동을 답한다. 모르
는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “API 키 제한 없이 공개 코드에 넣어”

직접 확인: 잘못된 주소와 API 실패에도 핵심 화면이 유지되는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 Google·Naver·Kakao 지
도 중 맞는 제공자를 고르고 위치 흐름을 만든다. 완료되면 주소->좌표->지도->실패
fallback이 검증된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비
밀 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

34장. 카카오톡 공유와 채널 상담 연결하기

카카오톡 연동이라는 말에는 서로 다른 기능이 섞여 있다.

- 카카오톡 공유: 사용자가 친구나 대화방을 선택해 콘텐츠를 공유한다.
- 카카오톡 메시지: 권한과 조건에 따라 서비스 사용자에게 메시지를 보낸다.
- 카카오톡 채널: 사용자가 채널을 추가하거나 상담으로 이동한다.
- 카카오 로그인: 카카오 계정으로 로그인한다.

링크를 퍼뜨리는 목적이라면 카카오톡 공유부터 시작한다. Kakao Developers에서 앱을 만들고 웹 플랫폼 도메인을 등록하고 JavaScript 키로 SDK를 초기화한 뒤 Kakao.Share 모듈을 사용한다.

공유 메시지에는 제목, 설명, 이미지, 이동 링크가 필요하다. 공유된 페이지 자체에도 Open Graph 제목, 설명, 이미지가 있어야 카카오톡 외의 서비스에서도 미리보기가 자연스럽다.

확인 목록

- 모바일과 데스크톱에서 공유 버튼이 동작하는가?
- 공유 링크가 로그인 없이 열려야 하는 페이지인가?
- 삭제되거나 비공개인 콘텐츠가 미리보기로 노출되지 않는가?
- 운영 도메인이 Kakao Developers에 등록돼 있는가?
- 테스트 도메인과 운영 도메인을 구분했는가?

이번 장의 Codex 실행 카드

목표: 카카오톡 공유 또는 채널 상담 중 필요한 기능을 연결한다.
먼저 공유할 URL·제목·설명·이미지를 확인해줘. 공개 가능한 페이지지만 공유하고 운영 도메인을 등록한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 모바일·데스크톱 공유와 링크 미리보기가 확인된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 친구 공유인지 자동 메시지인지 상담 이동인지 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “사용자 동의 없이 친구들에게 자동 발송해”

직접 확인: 비공개 콘텐츠가 링크 미리보기에 노출되지 않는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 카카오톡 공유 또는 채널 상담 중 필요한 기능을 연결한다. 완료되면 모바일·데스크톱 공유와 링크 미리보기가 확인된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

35장. Jira로 해야 할 일을 관리하기

Jira는 해야 할 일, 담당자, 상태, 우선순위와 논의를 이슈 단위로 관리하는 도구다. 코드를 만드는 도구가 아니라 일이 사라지지 않게 하는 장부다.

혼자 개발할 때도 다음 상황에서 유용하다.

- 기능이 10개 이상으로 늘어났을 때
- 출시 전 해야 할 일이 여러 분야로 흩어졌을 때
- Codex 작업을 세션과 컴퓨터 사이에서 이어갈 때
- 버그의 재현 절차와 수정 커밋을 연결할 때

처음에는 복잡한 스프린트보다 To Do -> In Progress -> Review -> Done 네 상태면 충분하다.

좋은 이슈에는 다음이 있다.

- 문제 또는 목표
- 현재 동작과 기대 동작
- 범위에 포함되는 것과 제외되는 것
- 완료 조건
- 디자인·정책·로그 링크
- 검증 결과와 관련 커밋

Jira 이슈를 Codex 작업으로 바꾸는 요청



MAR-42 이슈를 읽고 현재 저장소와 비교해 구현 범위를 정리해
줘. 이슈의 완료 조건을 임의로 넓히지 말고, 필요한 하위 작업과
검증 명령을 제안해줘. 구현 후 커밋과 테스트 결과를 이슈에 남
길 요약으로 작성해줘.

7부. 검색과 수익 만들기

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

이번 장의 Codex 실행 카드

목표: 기능과 버그를 Jira 이슈로 관리하는 최소 흐름을 만든다. 먼저 현재 할 일 목록과 저장소 규칙을 확인해줘. 상태를 복잡하게 만들지 않고 완료 조건을 명확히 한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 To Do·In Progress·Review·Done과 이슈 템플릿이 생긴다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 이번 달 꼭 끝낼 결과와 보류할 일을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “생각나는 일을 전부 긴급 이슈로 만들어”

직접 확인: 각 이슈가 한 결과를 가지는가; 커밋과 검증이 연결되는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 기능과 버그를 Jira 이슈로 관리하는 최소 흐름을 만든다. 완료되면 To Do·In Progress·Review·Done과 이슈 템플릿이 생긴다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

36장. SEO는 검색엔진을 속이는 기술이 아니다

SEO는 Search Engine Optimization, 검색엔진 최적화다. Google 공식 가이드는 검색엔진이 콘텐츠를 이해하도록 돕고 사용자가 검색 결과에서 방문 여부를 판단하게 하는 일로 설명한다. 숨은 키워드를 반복하거나 모든 제목에 인기 검색어를 붙이는 기술이 아니다.

먼저 고객이 검색하는 문제를 페이지 하나의 주제로 만든다. 마리에카드는 홈 하나로 모든 검색어를 잡으려 하지 않고 모바일 청첩장, 청첩장 제작, 돌잔치 초대장, 초대장 템플릿 같은 의도별 랜딩을 만들었다. 각 페이지는 고유한 제목, 설명, 실제 내용과 내부 링크를 가졌다.

페이지 하나의 기본 구조

- 사용자가 검색할 질문과 같은 명확한 제목
- 첫 문단에서 바로 주는 답
- 실제 예시와 화면
- 관련 질문과 답변
- 다음 행동으로 이어지는 링크
- 페이지마다 다른 title과 description

SEO의 목적은 방문자 수만 늘리는 것이 아니다. 원하는 고객이 들어와 제품을 이해하고 다음 행동을 하게 하는 것이다.

이번 장의 Codex 실행 카드

목표: 고객 검색 의도에 맞는 SEO 페이지 계획을 만든다. 먼저 고객 질문·검색어·현재 공개 페이지를 확인해줘. 검색어 반복보다 실제 답과 고유 콘텐츠를 우선한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 페이지별 의도·title·description·내부 링크가 정리된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 고객이 검색창에 쓸 실제 문장을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “인기 키워드를 모든 페이지에 최대한 반복해”

직접 확인: 페이지가 검색어 없이도 사람에게 유용한가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 고객 검색 의도에 맞는 SEO 페이지 계획을 만든다. 완료되면 페이지별 의도·title·description·내부 링크가 정리된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

37장. Google Search Console 연결하기

Google Search Console은 Google이 사이트를 어떻게 발견하고 색인하며 검색 결과에 보여주는지 확인하는 도구다. 광고 도구가 아니며 등록했다고 상위에 올려 주는 것도 아니다.

연결 순서

- 1 운영 도메인과 HTTPS 배포를 준비한다.
- 2 Search Console에서 Domain 또는 URL-prefix 속성을 추가한다.
- 3 DNS TXT나 HTML 태그 등으로 소유권을 확인한다.
- 4 `https://도메인/sitemap.xml`을 제출한다.
- 5 URL Inspection으로 홈과 핵심 페이지를 검사한다.
- 6 색인 가능 여부와 Google이 본 canonical을 확인한다.
- 7 며칠 뒤 Pages와 Performance 보고서를 본다.

Next.js에서는 metadata에 Google verification 값을 넣을 수 있다. Vercel 환경 변수에 content 값만 넣고 새로 배포한다. DNS TXT로 확인했다면 코드용 verification 값이 필요하지 않을 수 있다.

Search Console에 “미색인”이 있다고 모두 오류는 아니다. 로그인, 관리자, 미리보기, 중복 URL은 의도적으로 제외해야 한다. 중요한 것은 공개하려는 페이지가 색인 후보인지와 제외 이유가 의도와 같은지다.

이번 장의 Codex 실행 카드

목표: Google Search Console 소유권·사이트맵·핵심 URL 검사를 준비한다. 먼저 운영 도메인·verification 방식·sitemap URL을 확인해줘. 실제 계정 로그인과 소유권 확인은 사용자가 수행한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 등록 순서와 배포 후 확인 URL이 체크리스트로 남는다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: DNS 접근 가능 여부와 대표 도메인을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “내 Google 계정에 알아서 로그인해서 전부 등록해”

직접 확인: 속성 URL과 canonical 도메인이 같은가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 Google Search Console 소유권·사이트맵·핵심 URL 검사를 준비한다. 완료되면 등록 순서와 배포 후 확인 URL이 체크리스트로 남는다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

38장. Naver Search Advisor 연결하기

Naver Search Advisor는 네이버 검색로봇이 사이트를 수집하고 이해하는 데 필요한 상태를 확인하는 도구다.

연결 순서

- 1 운영 사이트 주소를 추가한다.
- 2 메타 태그 또는 HTML 확인 파일로 소유권을 확인한다.
- 3 robots.txt와 sitemap.xml이 200 응답으로 열리는지 확인한다.
- 4 사이트맵을 제출한다.
- 5 URL 수집 요청과 진단 결과를 확인한다.
- 6 title, description, H1, 링크와 접근 제한 문제를 고친다.

마리에카드는 정적 HTML 확인 파일을 public/에 두어 <https://mariecard.com/확인파일.html>이 열리게 했다. 확인 파일 이름은 사이트마다 다르므로 다른 프로젝트의 값을 복사하면 안 된다.

네이버 공식 가이드에 따르면 robots.txt는 사이트 루트에 있어야 하고 일반 텍스트로 접근돼야 한다. 개인정보는 robots로 보호하지 않는다. robots는 검색로봇에 대한 안내이지 접근 통제가 아니다. 로그인과 서버 권한이 별도로 필요하다.

이번 장의 Codex 실행 카드

목표: Naver Search Advisor 소유권·robots·sitemap 등록을 준비한다. 먼저 운영 도메인과 Naver 확인 파일 또는 메타값을 확인해줘. 다른 사이트의 확인 파일을 복사하지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 확인 URL·robots·sitemap과 제출 순서가 검증된다. 마지막으로 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 메타 태그와 HTML 파일 중 선택한 방식을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

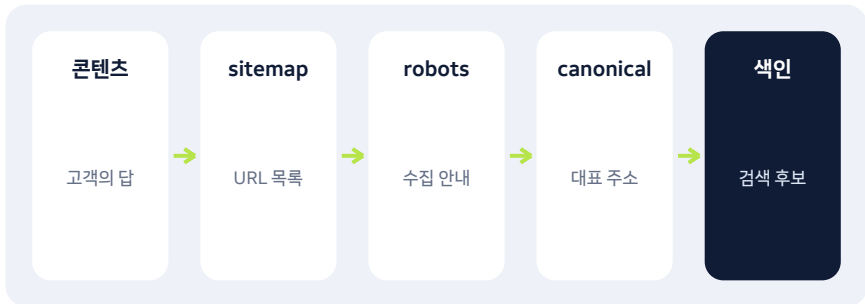
피할 요청: “인터넷에서 찾은 네이버 확인 파일을 넣어줘”

직접 확인: 확인 파일이 루트 URL에서 열리는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 Naver Search Advisor 소유권·robots·sitemap 등록을 준비한다. 완료되면 확인 URL·robots·sitemap과 제출 순서가 검증된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

39장. sitemap·robots·canonical·noindex 이해하기



네 단어는 서로 다른 일을 한다.

항목	역할	비유
sitemap.xml	수집할 주요 URL 목록 제공	건물 안내도
robots.txt	로봇이 방문할 경로 안내	출입 안내판
canonical	비슷한 URL 중 대표 주소 지정	본점 주소
noindex	이 페이지를 검색 결과에 넣지 말라고 요청	전시 제외 표시

robots.txt에서 막은 페이지에 noindex를 넣으면 검색로봇이 그 태그를 읽지 못할 수도 있다. 민감한 페이지는 인증으로 막고, 검색 노출 정책은 페이지 특성에 맞게 설계한다.

Codex SEO 점검 요청

운영 도메인을 기준으로 공개 페이지와 비공개 페이지를 분류해 줘. 각 공개 페이지의 title, description, canonical, Open Graph를 점검하고, 로그인·관리자·미리보기·사용자 개인 페이지에는 noindex 정책을 제안해줘. robots.txt와 sitemap.xml을 생성한 뒤 로컬 build와 실제 응답 형식을 검증해줘.

배포 직후 확인 주소

```
https://example.com/  
https://example.com/robots.txt  
https://example.com/sitemap.xml
```

이번 장의 Codex 실행 카드

목표: 공개·비공개 URL의 sitemap·robots·canonical·noindex 정책을 구현한다. 먼저 전체 라우트 목록과 대표 도메인을 확인해줘. robots를 개인정보 보호 수단으로 사용하지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 세 파일/메타 정책과 실제 HTTP 응답이 확인된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 검색에 보여야 할 페이지와 숨길 페이지를 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “검색이 잘되게 모든 페이지를 sitemap에 넣어”

직접 확인: 로그인·관리자·개인 페이지가 검색 후보에서 제외되는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 공개·비공개 URL의 `sitemap·robots·canonical·noindex` 정책을 구현한다. 완료되면 세 파일/메타 정책과 실제 HTTP 응답이 확인된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

40장. Google AdSense를 붙이기 전에 준비할 것

AdSense는 사이트에 광고를 표시하고 수익을 배분하는 Google 서비스다. 코드를 넣기 전에 계정, 결제 정보, 사이트 연결과 사이트 검토를 거친다. Google은 독창적이고 방문자에게 가치 있는 콘텐츠와 프로그램 정책 준수를 요구한다.

계산기 한 화면만 있고 설명, 운영 주체, 문의, 개인정보 처리방침이 없는 사이트는 승인과 장기 운영에 불리할 수 있다. 광고보다 먼저 제품과 콘텐츠를 만든다.

구현 원칙

- 버튼과 입력창 가까이에 광고를 두어 오클릭을 유도하지 않는다.
- 광고가 채워지지 않아도 큰 빈 공간이 남지 않게 한다.
- 광고 스크립트는 필요한 곳에서 한 번만 로드한다.
- 위치별 광고 단위를 나눠 성과를 본다.
- 네이티브 앱의 WebView에서는 정책과 SDK 요건을 별도 확인한다.
- 광고 로딩 실패가 핵심 기능을 막지 않게 한다.

아파트구구는 웹 콘텐츠 영역에만 AdSense를 표시하고 Capacitor iOS에서는 광고 UI와 네트워크 요청을 모두 차단했다. 모바일 하단 탭과 광고 사이의 간격, 미충족 광고 영역 접기, 1180px 이상 데스크톱 레일 광고를 별도 정책으로 관리했다.

8부. Threads 홍보와 자동화

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

이번 장의 Codex 실행 카드

목표: AdSense 신청 전 콘텐츠와 광고 배치 준비도를 점검한다. 먼저 공개 콘텐츠·정책 페이지·모바일 화면을 확인해줘. 승인과 수익을 보장하지 않고 오클릭을 유도하지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 신청 전 보완 목록과 안전한 광고 위치가 정리된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 원본 콘텐츠와 문의·약관 페이지의 존재를 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “수익이 높게 버튼 바로 옆에 광고를 붙여”

직접 확인: 광고 실패가 기능을 막지 않는가; 앱 정책이 분리되는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 AdSense 신청 전 콘텐츠와 광고 배치 준비도를 점검한다. 완료되면 신청 전 보완 목록과 안전한 광고 위치가 정

리된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

41장. 홍보 글을 제품의 데이터에서 만들기

매일 “우리 서비스를 써 주세요”라고 쓰면 팔로우할 이유가 줄어든다. 제품이 새롭게 만든 데이터, 고객 질문, 계산 결과, 사례를 콘텐츠로 바꾼다.

아파트구구에는 실거래, 신고가, 정책과 뉴스라는 반복 가능한 소재가 있었다. 자동화 전에 다음 구조를 정했다.

- 입력 데이터는 어디에서 오는가?
- 어떤 조건의 항목을 고르는가?
- 한 게시물에 몇 건을 보여주는가?
- 출처와 기준 시각을 어떻게 표시하는가?
- 같은 내용을 다시 쓰지 않는 기준은 무엇인가?
- 데이터가 오래됐을 때 게시할 것인가?

좋은 자동 글은 완전한 자유 생성보다 템플릿과 검증된 데이터의 조합이다.

[오늘의 서울 신고가 | 7월 19일]

강남구 A아파트 전용 84 가 최근 거래에서 이전 최고가를 경신했습니다.

기준 데이터: 국토교통부 실거래 공개자료

금융과 부동산 정보는 지연, 정정, 누락 가능성을 표시한다. AI가 숫자를 만들어내게 하지 않는다.

이번 장의 Codex 실행 카드

목표: 제품 데이터에서 반복 가능한 Threads 콘텐츠 규칙을 만든다. 먼저 데이터 출처·기준 시각·고객 질문을 확인해줘. 숫자를 생성하지 않고 오래된 데이터는 표시하거나 중단한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 선정 조건·템플릿·금지 표현·검수 예시가 생긴다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 데이터가 갱신되는 주기와 출처를 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “매일 자극적인 글을 시가 알아서 만들어”

직접 확인: 출처와 기준 시각이 있는가; 같은 문장이 반복되지 않는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 제품 데이터에서 반복 가능한 Threads 콘텐츠 규칙을 만든다. 완료되면 선정 조건·템플릿·금지 표현·검수 예시가 생긴다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

42장. Threads API로 자동발행 봇 만들기

화면을 자동으로 클릭하는 비공식 봇보다 Meta가 제공하는 Threads API를 사용한다. API와 권한 정책은 바뀔 수 있으므로 구현 시점의 공식 문서를 다시 확인한다.

기본 흐름은 다음과 같다.

- 1 Meta for Developers에서 앱을 만든다.
- 2 Threads 사용 사례를 추가한다.
- 3 callback URL과 필요한 권한을 설정한다.
- 4 OAuth로 사용자 동의를 받고 access token을 발급한다.
- 5 필요하면 장기 토큰으로 교환하고 만료를 관리한다.
- 6 사용자 ID를 확인한다.
- 7 글 내용으로 media container를 생성한다.
- 8 생성된 container를 publish한다.
- 9 게시물 ID와 시각을 저장한다.

아파트구구는 `THREADS_ACCESS_TOKEN`, 선택적인 `THREADS_USER_ID`, Cron 보호용 `CRON_SECRET`을 환경변수로 관리했다. 상태 확인, 수동 게시, 정책 글, 실거래 글을 각각 엔드포인트로 나눴다.

Codex 구현 요청

Meta Threads API의 현재 공식 문서를 확인해 텍스트 게시 자동화 기능을 구현해줘. OAuth URL 생성, callback 교환, 토큰 갱신, 사용자 ID 확인, container 생성, publish를 단계별 모듈로 나눠줘. 토큰은 서버 환경변수와 암호화된 저장소에서만 다루고 로그에 출력하지 마. 먼저 dry-run과 mock 테스트를 만들고 실제 계정 게시 전에는 멈춰줘.

실제 글을 외부에 게시하는 것은 되돌릴 수 있는 코드 수정과 다르다. 반드시 dry-run으로 본문을 확인한 뒤 실발행 권한을 준다.

이번 장의 Codex 실행 카드

목표: Meta Threads API 텍스트 자동발행을 dry-run부터 구현한다. 먼저 공식 최신 Threads 문서와 발행 계정을 확인해줘. 비공식 화면 봇을 쓰지 않고 토큰을 서버에서만 보관하며 실발행 전 멈춘다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 OAuth·토큰·container·publish의 mock과 dry-run이 통과한다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 본인 계정인지와 수동 승인 가능한 테스트 글을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “아이디와 비밀번호로 화면을 자동 클릭해 글을 올려”

직접 확인: 로그에 토큰이 없는가; 실발행 전에 본문을 볼 수 있는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 Meta Threads API 텍스트 자동발행을 dry-run부터 구현한다. 완료되면 OAuth·토큰·container·publish의

mock과 **dry-run**이 통과한다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

43장. 예약발행·중복방지·실패복구 설계



자동발행의 핵심은 글쓰기보다 운영 안전장치다.

예약발행

Vercel Cron이나 별도 서버의 스케줄러가 정해진 시각에 API를 호출한다. Vercel 설정은 UTC 기준일 수 있으므로 한국 시각으로 변환하고 실제 로그에서 확인한다.

중복방지

게시할 항목마다 고유 키를 만든다.

```
threads:record-high:20260719:서울:강남구
```

게시 전에 키가 있는지 확인하고, 성공한 뒤 저장한다. 서버리스의 /tmp 파일은 다음 실행에도 남는 영구 저장소가 아니므로 Redis나 데이터베이스 같은 공유 저장소를 사용한다.

실패복구

- API 요청 실패 시 게시 성공으로 기록하지 않는다.

- 게시 성공 뒤 저장 실패가 나면 게시물 ID로 중복 여부를 조사한다.
- 재시도 횟수와 간격을 제한한다.
- 토큰 만료와 권한 오류를 일반 네트워크 오류와 구분한다.
- 오래된 데이터라면 발행을 멈춘다.

아파트구구의 실거래 자동발행은 기본을 dry-run으로 두고 live=1을 명시해야 게시되게 했다. 공유 KV 중복방지가 없으면 실발행을 차단했다. 안전한 기본값이 자동화 사고를 줄인다.

이번 장의 Codex 실행 카드

목표: 예약발행·중복방지·실패복구를 구현한다. 먼저 발행 시간·중복 기준·공유 저장소·실패 정책을 확인해줘. 기본은 dry-run이며 영구 중복 저장소가 없으면 live를 막는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 스케줄·dedupe·재시도·rollback 테스트가 통과한다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 한국 시각과 하루 게시 수, 같은 글의 기준을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “실패하면 성공할 때까지 무한 재시도해”

직접 확인: 같은 항목이 두 번 게시되지 않는가; 오래된 데이터는 멈추는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 예약발행·중복방지·실패복구를 구현한다. 완료되면 스케줄·dedupe·재시도·rollback 테스트가 통과한다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

44장. Codex로 운영 자동화를 계속 개선하기

자동화는 한 번 만들고 끝나는 기능이 아니다. 게시 성공률, 중복, 클릭, 차단, 데이터 신선도를 보면서 개선한다.

자동화에서 먼저 설계할 것은 도구가 아니라 시나리오다. 무엇이 시작 신호인가 -> 어떤 데이터를 읽는가 -> 어떤 조건이면 멈추는가 -> 누가 승인하는가 -> 성공과 실패를 어디에 기록하는가 -> 실패 뒤 어떻게 복구하는가를 한 줄씩 적는다. GitHub Actions나 Cron, Telegram 같은 도구 이름은 그 뒤에 선택한다. 이 순서를 지키면 도구가 바뀌어도 운영 원칙은 남는다.

Codex에 다음 순서로 맡긴다.

- 1 최근 로그를 읽고 실패 유형을 분류한다.
- 2 재현 가능한 테스트를 만든다.
- 3 가장 작은 수정안을 구현한다.
- 4 dry-run으로 새 글과 스케줄을 확인한다.
- 5 한 건만 실행한다.
- 6 게시물 ID와 중복 키를 확인한다.
- 7 다음 자동 실행을 모니터링한다.

운영 분석 요청



최근 14일 Threads 자동발행 로그를 읽고 성공, 데이터 없음, 토큰 오류, rate limit, 중복 차단으로 분류해줘. 비밀 토큰은 가려줘. 가장 많이 발생한 실패의 원인과 작은 수정안을 제안하고, 실행 없이 mock과 dry-run으로 검증해줘.

콘텐츠 품질도 자동화한다. 같은 시작 문장이 반복되는지, 링크 없는 글이 이해되는지, 숫자 기준 시각이 빠지지 않았는지 검사하는 테스트를 둘 수 있다.

9부. 혼자 운영하고 성장시키기

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

이번 장의 Codex 실행 카드

목표: 최근 자동화 로그를 분석하고 가장 큰 실패 하나를 고친다. 먼저 최근 14일 로그와 성공 기준을 확인해줘. 토큰을 가리고 한 번에 한 원인만 수정한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 분류표·회귀 테스트·dry-run 비교가 남는다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 성공·실패로 보는 기준과 변경 가능한 범위를 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “로그는 지우고 봇을 처음부터 다시 만들어”

직접 확인: 실패율이 실제로 줄었는가; 콘텐츠 품질이 유지되는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 최근 자동화 로그를 분석하고 가장 큰 실패 하나를 고친다. 완료되면 분류표·회귀 테스트·dry-run 비교가 남는다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

45장. 로그·모니터링·백업·장애 대응

서비스가 공개된 순간부터 개발보다 운영 시간이 길어진다.

로그는 무슨 일이 일어났는지 기록한다. 모니터링은 오류와 성능을 계속 관찰한다. 백업은 데이터가 사라졌을 때 복구할 복사본이다. 세 가지는 서로 대신할 수 없다.

최소 운영 항목은 다음과 같다.

- 배포 성공과 실패 기록
- 서버 API 오류율과 응답 시간
- 로그인·결제·발행 실패
- 데이터베이스 백업과 복원 절차
- 외부 API 토큰 만료
- 자동 작업의 마지막 성공 시각
- 고객 문의와 재현 절차

마리에카드는 Sentry를 연결해 실제 Production 오류가 수집되는지 테스트했다. source map 토큰이 노출된 뒤에는 토큰을 교체하고 폐기했다. 도구를 연결하는 것 만큼 시험 오류를 발생시켜 수집되는지 확인하는 일이 중요하다.

장애가 나면 먼저 새 기능을 만들지 않는다.

- 1 영향 범위를 확인한다.
- 2 최근 배포와 환경변수 변경을 확인한다.
- 3 재현 절차와 시각을 기록한다.
- 4 화면, API, DB, 외부 서비스 상태를 분리한다.
- 5 안전한 임시 완화와 근본 수정을 구분한다.

6 수정 뒤 같은 절차로 확인한다.

이번 장의 Codex 실행 카드

목표: 출시 서비스의 로그·모니터링·백업·장애 절차를 만든다. 먼저 배포·API·DB·인증·결제·Cron 목록을 확인해줘. 민감정보를 로그에 남기지 않고 복원 테스트까지 계획한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 알림 조건·담당 행동·복원 순서가 문서화된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 고객에게 가장 치명적인 실패와 허용 복구 시간을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “모든 요청과 사용자 데이터를 로그에 남겨”

직접 확인: 시험 오류가 수집되는가; 백업에서 복원할 수 있는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 출시 서비스의 로그·모니터링·백업·장애 절차를 만든다. 완료되면 알림 조건·담당 행동·복원 순서가 문서화된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

46장. 비용이 갑자기 커지지 않게 하는 법

AI가 서비스를 빨리 만들수록 외부 API 호출과 저장 비용도 빨리 늘 수 있다.

비용은 다음 식으로 본다.

월 비용 = 사용자 수 × 사용자당 요청 수 × 요청당 단가

지도, AI, 공공데이터, 이미지 변환, 서버 함수, 데이터베이스와 스토리지마다 과금 기준이 다르다.

비용 방어 장치

- 공급자 대시보드에 예산 알림을 설정한다.
- API 키에 허용 도메인과 API 제한을 건다.
- 같은 요청은 캐시한다.
- 입력마다 호출하지 말고 제출 시 호출한다.
- 이미지 크기와 업로드 개수를 제한한다.
- 관리자·봇 API에 인증과 rate limit를 둔다.
- 무료 한도 초과 시 사용자에게 보여줄 동작을 정한다.

아파트구구는 사용자의 추천 요청마다 원천 공공 API를 호출하지 않고 서버가 주기적으로 데이터를 수집해 공유 캐시에 넣는 cache-first 구조로 바꿨다. 추천 응답은 기존 cold 13.7초에서 약 0.73~1.50초로 개선됐고 사용자 요청 중 원천 API 호출을 0건으로 만들었다. 성능 개선과 비용 방어가 같은 구조에서 나왔다.

이번 장의 Codex 실행 카드

목표: 월 비용을 추정하고 사용량 방어 장치를 넣는다. 먼저 예상 사용자·요청·파일·외부 API 단가를 확인해줘. 현재 공식 가격을 확인하고 모르는 사용량은 범위로 계산한다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 낮음·기준·높음 비용과 알림·캐시·rate limit 계획이 생긴다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 첫 달 사용자와 하루 행동 횟수를 추정해 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “무료 한도니까 비용 검사는 필요 없어”

직접 확인: 예산 알림과 API 키 제한이 켜져 있는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 월 비용을 추정하고 사용량 방어 장치를 넣는다. 완료되면 낮음·기준·높음 비용과 알림·캐시·rate limit 계획이 생긴다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

47장. 마리에카드를 만든 실제 순서

마리에카드는 처음부터 완성된 설계로 만들어지지 않았다. 142개의 커밋은 한 번의 거대한 프롬프트가 아니라 작은 구현과 수정의 기록이다.

실제 흐름을 초보자용 단계로 정리하면 다음과 같다.

- 1 공개 청첩장과 관리자 편집기라는 핵심 가치를 만든다.
- 2 모바일 미리보기와 공개 링크를 안정화한다.
- 3 Supabase 데이터 모델과 사용자별 접근 정책을 만든다.
- 4 Google 로그인과 대시보드를 연결한다.
- 5 이미지 업로드와 공개 상태를 정리한다.
- 6 테스트와 배포 체크를 만든다.
- 7 공개 랜딩, 다국어 페이지와 SEO를 확장한다.
- 8 발행 쿠폰과 관리자 감사 로그를 만든다.
- 9 로그인 전 제작하고 마지막에 로그인하는 흐름으로 전환한다.
- 10 한국어·영어 초대장을 같은 데이터 모델에서 편집하고 발행하도록 확장한다.
- 11 Production Supabase와 Sentry를 다시 검증한다.

출시 직전에는 **lint**, 132개의 자동 테스트, **build**, 13개의 Playwright 브라우저 테스트를 통과시켰다. 테스트 개수가 품질을 보장하는 것은 아니지만, 로그인 전 초안 복원, 첫 발행, 쿠폰 소비, 언어 전환처럼 돈과 데이터가 얹힌 핵심 흐름을 반복 확인할 수 있게 해준다.

중간에는 OAuth callback, 공개 상태, 쿠폰 조건, 모바일 미리보기와 환경변수 문제를 여러 번 고쳤다. Codex를 쓴다고 시행착오가 사라지지는 않는다. 대신 저장소 전체를 조사하고, 테스트와 문서를 남기고, 다음 작업자가 이어받을 수 있는 속도가 빨

라진다.

마리에카드형 서비스의 추천 스택

- 화면과 서버: Next.js
- 배포: Vercel
- 회원·DB·파일: Supabase
- 로그인: Supabase Auth + Google, 필요 시 Kakao
- 결제: 국내 PG 또는 Stripe를 상품 특성에 맞게 선택
- 테스트: Vitest + Playwright
- 오류 관찰: Sentry
- 업무 기록: GitHub + Jira + 저장소 문서

이번 장의 Codex 실행 카드

목표: 마리에카드형 회원·콘텐츠 서비스를 작은 출시 단계로 계획한다. 먼저 내 서비스 기획서와 마리에카드 사례를 확인해줘. 완성 구조를 첫날 복제하지 않고 핵심 가치부터 만든다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 MVP->로그인->저장->발행->SEO->운영 순서가 내 서비스에 맞게 조정된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 내 서비스에서 사용자가 만들고 공개할 결과물을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “마리에카드의 모든 기능을 한 번에 복사해”

직접 확인: 첫 주에 고객이 경험할 한 가지 가치가 있는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 **마리에카드형 회원·콘텐츠 서비스를 작은 출시 단계로 계획한다**. 완료되면 MVP->로그인->저장->발행->SEO->운영 순서가 내 서비스에 맞게 조정된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

48장. 아파트구구를 키운 실제 순서

아파트구구는 계산기에서 데이터 서비스와 iOS 앱, 자동 홍보 시스템으로 확장됐다. 702개의 커밋은 기능이 늘어날수록 운영 구조가 중요해진다는 증거다.

- 1 구매 가능 금액 계산과 결과 공유를 만든다.
- 2 모바일 UI와 결과 저장을 개선한다.
- 3 AdSense를 붙이되 정책 위반과 오클릭 위험을 고친다.
- 4 공공데이터와 추천 기능을 연결한다.
- 5 지도 위치 미리보기와 외부 지도 이동을 만든다.
- 6 Next.js 웹을 Capacitor로 감싸 iOS 앱으로 확장한다.
- 7 사용자 요청 중 원천 API 직접 호출을 없애고 cache-first로 전환한다.
- 8 별도 Mac 서버 수집기, Upstash Redis와 Cloudflare tunnel을 운영한다.
- 9 Threads와 Telegram에 정책·실거래·신고가를 자동발행한다.
- 10 freshness monitor, watchdog, 백업과 배포 guard를 추가한다.

처음 만드는 사람에게 이 모든 구조가 필요한 것은 아니다. 계산기 한 화면을 출시한 뒤 실제 느린 요청과 반복 업무가 생길 때 확장한다. 남의 완성된 아키텍처를 첫날 복사하지 않는다.

이번 장의 Codex 실행 카드

목표: 아파트구구형 정보·계산 서비스를 단계적으로 계획한다. 먼저 계산 정책·데이터 출처·갱신 주기를 확인해줘. 원천 API를 사용자 요청마다 무제한 호출하지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 계산 MVP->데이터->캐시->지도->광고->자동화 로드맵이 생긴다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 정확성을 책임질 정책과 데이터 출처를 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “데이터가 없으면 AI가 비슷한 숫자를 만들어”

직접 확인: 계산 회귀 테스트와 데이터 신선도 표시가 있는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 아파트구구형 정보·계산 서비스를 단계적으로 계획한다. 완료되면 계산 MVP->데이터->캐시->지도->광고->자동화 로드맵이 생긴다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

49장. 다니엘의 바이블로 배운 앱·구독·AI 운영

다니엘의 바이블은 “성경 본문을 보여주는 앱”으로 시작하지 않았다. 개인이 읽던 흐름과 목장 같은 공동체의 함께 읽기가 서로 끊겨 있다는 문제에서 시작했다. 그래서 읽던 위치 이어보기, 장·절 이동, 읽기 진행 기록, 그룹 생성과 참여, 초대 링크, 그룹별 읽기 계획과 멤버 진행 상황을 하나의 흐름으로 연결했다.

혼자 만든 서비스라도 제품은 여러 층으로 늘어났다.

- 1 Next.js와 React로 웹 읽기 경험을 만든다.
- 2 Node.js·Prisma·PostgreSQL로 사용자, 읽기 기록과 그룹 관계를 저장한다.
- 3 하이라이트, 북마크, 목상과 노트를 읽는 흐름 안에 넣는다.
- 4 Capacitor로 iOS 앱을 만들고 실제 기기와 TestFlight에서 확인한다.
- 5 구독 상품과 이용 권한을 연결하고 복원·만료·실패 상태를 처리한다.
- 6 스토어 심사에서 요구하는 계정, 결제, 개인정보와 설명을 맞춘다.
- 7 AI 해설을 기능 과시가 아니라 어려운 본문을 이해하고 다시 읽게 하는 보조 도구로 넣는다.

여기서 가장 오래 걸린 것은 첫 80%가 아니라 마지막 20%였다. 웹에서 잘 보이던 화면이 앱의 안전 영역에서 깨지고, 테스트 결제가 실제 심사 환경에서 다르게 동작하며, 구독 상태가 서버와 앱에서 어긋날 수 있었다. AI 답변도 한 번 나온다고 끝이 아니었다. 금지할 표현, 출처를 다루는 방식, 실패 시 안내와 재생성 조건을 정해야 했다.

Codex에는 “앱으로 만들어줘”라고 한 번에 맡기지 않는다. 다음처럼 단계와 증거를 분리한다.

현재 Next.js 웹을 먼저 조사해줘. 1단계에서는 Capacitor iOS 업데이트와 안전 영역만 만들고 기존 웹 동작은 바꾸지 마. 실제 기기에서 확인할 체크리스트를 작성해줘. 2단계에서는 구독 상태 모델과 복원·만료·실패 흐름을 문서로 설계하고 구현 전에 위험을 알려줘. 3단계에서는 샌드박스 결제를 붙이고 서버 영수증 확인, 중복 웹훅, 권한 회수를 테스트해줘. 각 단계가 끝날 때 변경 파일과 통과한 증거를 보고하고 다음 단계로 넘어가기 전에 멈춰줘.

AI 기능도 먼저 평가표를 만든다. 정확성, 사용자의 다음 행동, 위험한 단정, 빈 답변과 지연을 어떻게 판단할지 정한 뒤 대표 입력을 고정 테스트로 남긴다. AI가 생성했다는 사실보다 사용자가 안전하게 쓸 수 있는 품질 기준이 중요하다.

다니엘의 바이블 저장소에 쌓인 1,177개의 커밋은 혼자 만든다는 말이 혼자 모든 답을 안다는 뜻이 아님을 보여준다. 작은 변경을 기록하고, 실제 기기에서 틀린 가정을 발견하고, 심사와 사용자 피드백을 다시 제품 규칙으로 바꾸는 것이 혼자 운영하는 방식이다.

이번 장의 Codex 실행 카드

목표: 웹서비스를 앱·구독·AI 기능으로 안전하게 확장한다. 먼저 현재 웹 구조, 앱 심사 요구, 구독 상태와 AI 품질 기준을 확인해줘. 웹·앱·구독·AI를 한 번에 바꾸지 않고 단계별 증거를 남긴다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 실기기 테스트·구독 복원과 만료·AI 실패 상태까지 검증된다. 마지막에는 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 지원할 앱 플랫폼, 구독 상품, AI가 도울 한 가지 행동을 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “웹을 앱으로 바꾸고 구독과 AI도 한 번에 전부 붙여줘”

직접 확인: 실제 기기와 샌드박스에서 검증했는가; 사용자가 피해 볼 실패가 막히는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 웹서비스를 앱·구독·AI 기능으로 안전하게 확장한다. 완료되면 실기기 테스트·구독 복원과 만료·AI 실패 상태까지 검증된다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.

50장. 첫 30일 실행 계획

1주차: 문제와 로컬 MVP

- 고객 5명에게 현재 방법과 불편을 듣는다.
- 1페이지 기획서와 데이터 명사를 작성한다.
- 프로젝트 폴더와 Git 저장소를 만든다.
- Codex용 AGENTS.md를 만든다.
- Next.js 첫 화면과 핵심 입력·결과를 로컬에서 실행한다.

2주차: 제품과 데이터

- 핵심 흐름을 모바일에서 완성한다.
- 계산이나 정책 로직에 테스트를 만든다.
- 저장에 필요하면 Supabase 테스트 프로젝트를 연결한다.
- 로그인은 가치 경험 뒤로 미룰 수 있는지 검토한다.
- 사용자 3명에게 직접 사용하게 하고 막힌 지점을 기록한다.

3주차: GitHub와 배포

- 기능별 커밋과 브랜치를 사용한다.
- GitHub에 푸시한다.
- Vercel Preview를 만들고 환경변수를 구분한다.
- 실제 도메인을 연결한다.
- 공개·비공개 페이지의 SEO 정책을 만든다.

4주차: 첫 고객과 운영

- 결제 전이라도 구매 의사나 예약을 받는다.
- 결제가 핵심이면 테스트 결제부터 구현한다.
- Search Console과 Search Advisor를 연결한다.
- 로그와 오류 수집을 확인한다.
- 첫 30일 데이터를 보고 다음 기능 하나만 결정한다.

30일 목표는 완벽한 회사를 만드는 것이 아니다. 모르는 사람이 인터넷에서 서비스를 열고, 핵심 행동을 완료하고, 내가 그 결과를 관찰할 수 있는 상태를 만드는 것이다.

이번 장의 Codex 실행 카드

목표: 내 서비스의 실제 첫 30일 실행계획을 확정한다. 먼저 앞에서 만든 기획·기술·배포·운영 문서를 확인해줘. 한 주에 핵심 결과 하나만 두고 미완성 기능을 숨기지 않는다. 현재 상태와 계획을 쉬운 말로 설명한 뒤 필요한 작업을 진행해줘. 완료 조건은 날짜·결과물·검증 사용자·중단 기준이 있는 30일 표가 생긴다. 마지막으로 변경 파일, 실행한 검사, 완료 증거와 남은 위험을 구분해 알려줘.

Codex가 질문하면: 매주 확보 가능한 시간과 만날 고객 수를 답한다. 모르는 항목은 모름이라고 답하고 가장 단순하고 되돌리기 쉬운 선택을 추천받는다.

피할 요청: “한 달 안에 회원·결제·앱·자동홍보를 전부 완성해”

직접 확인: 30일 뒤 계속할지 판단할 실제 행동 데이터가 남는가.

연속 예제: 동네 클래스 예약 서비스라면 이번 단계에서 내 서비스의 실제 첫 30일 실행계획을 확정한다. 완료되면 날짜·결과물·검증 사용자·중단 기준이 있는 30일 표

가 생긴다는 증거가 남아야 한다.

작업 뒤에는 책 앞부분의 공통 검수 문장을 이어서 입력한다. 계정 비밀번호, API 비밀번호 키, 결제 키와 실제 사용자 개인정보는 대화에 붙여 넣지 않는다.



부록. 바로 복사해 쓰는 Codex 실전 도구

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

부록 1. 새 서비스 시작 프롬프트

나는 IT 용어를 거의 모르는 비개발자다. 이 폴더에서 [고객]의 [문제]를 해결하는 [서비스]를 만들고 싶다. 바로 코딩하지 말고 현재 폴더와 설치 환경을 조사해줘. 고객, 입력, 결과, 저장, 로그인, 결제, 운영에 관해 빠진 질문을 한 번에 하나씩 해줘. 답변이 끝나면 1페이지 기획서, 데이터 초안, MVP 범위, 30일 계획을 파일로 작성해줘. 모든 기술 용어는 처음 등장할 때 쉬운 말로 설명해줘.

부록 2. 기능 구현 프롬프트

목표: [원하는 결과]. 맥락: [관련 문서·화면·현재 동작]. 제약: [바꾸지 않을 것·정책·보안]. 완료 조건: [사용자 행동·테스트·화면].
먼저 관련 파일을 조사하고 계획을 세워줘. 구현 뒤 lint, test, build와 브라우저 확인을 하고, 변경 파일·검증 결과·남은 위험을 알려줘.

부록 3. 버그 수정 프롬프트

위치: [URL/화면]. 재현: [행동 순서]. 기대: [원래 결과]. 실제: [오류]. 시각과 환경: [Production/Preview/Local]. 운영 데이터는 수정하지 말고 읽기 전용 조사부터 해줘. 원인 가설을 세우고 로그와 코드로 반증해줘. 최소 수정 후 회귀 테스트와 같은 재현 절차로 확인해줘.

부록 4. 배포 전 프롬프트

현재 브랜치와 Git 상태를 확인해 이번 작업과 기존 사용자 변경을 구분해줘. .env, 토큰, 개인정보가 추적되지 않는지 검사해줘. 관련 테스트, lint, build를 실행하고 diff를 검토해줘. 실패가 없으면 관련 파일만 커밋하되 푸시와 Production 배포는 하지 말고 결과를 보고해줘.

부록 5. 출시 전 체크리스트

- ☐ 모바일과 데스크톱 핵심 흐름 완료
- ☐ 빈 값, 중복 클릭, 실패와 느린 네트워크 확인
- ☐ lint, test, build 통과
- ☐ Preview에서 실제 사용 흐름 확인
- ☐ .env와 비밀 키 Git 제외
- ☐ 사용자별 DB 권한 확인
- ☐ 운영 환경변수 설정 후 재배포
- ☐ 도메인과 HTTPS 확인
- ☐ robots.txt, sitemap.xml, canonical 확인
- ☐ 개인정보 처리방침·이용약관·문의 경로 확인
- ☐ 오류 수집과 백업 확인
- ☐ 결제는 테스트 키와 운영 키 분리
- ☐ 자동 게시 기능은 dry-run과 중복방지 확인

부록 6. 초보자 용어집

용어	쉬운 설명
코드	컴퓨터가 실행할 규칙을 글로 적은 것
프레임워크	서비스를 만드는 기본 구조와 도구 묶음
Next.js	React 기반 웹서비스를 만드는 프레임워크
로컬	인터넷 서버가 아니라 내 컴퓨터 환경
터미널	글자로 컴퓨터 명령을 실행하는 창
패키지	프로젝트가 가져다 쓰는 외부 코드 묶음
npm	JavaScript 패키지를 설치하고 명령을 실행하는 도구
build	개발 코드를 배포 가능한 결과물로 만드는 과정
deploy	결과물을 인터넷에서 실행되도록 올리는 일
domain	사용자가 입력하는 서비스 주소
DNS	도메인을 실제 서비스 위치와 연결하는 주소 체계
frontend	사용자가 보고 조작하는 화면
backend	화면 뒤에서 검증·저장·연동하는 로직
server	요청을 받아 코드를 실행하고 응답하는 환경
serverless	서버를 직접 관리하지 않고 필요한 코드를 실행하는 방식

용어	쉬운 설명
database	구조화된 데이터를 지속적으로 저장하는 곳
table	같은 종류의 데이터를 담는 표
storage	사진과 영상 같은 파일을 저장하는 곳
API	다른 프로그램과 대화하기 위한 약속된 창구
API key	API 요청 주체를 식별하는 키
token	일정 권한과 유효기간을 가진 디지털 출입증
environment variable	키와 설정을 코드 밖에 보관하는 값
authentication	사용자가 누구인지 확인하는 것
authorization	확인된 사용자가 무엇을 할 수 있는지 정하는 것
OAuth	외부 계정으로 안전하게 로그인 권한을 받는 방식
session	로그인 상태를 일정 기간 유지하는 정보
Git	파일 변경 역사를 관리하는 도구
GitHub	Git 저장소를 온라인에 보관·협업하는 플랫폼
repository	프로젝트 파일과 변경 이력을 담은 저장소
commit	선택한 변경을 역사에 저장한 스냅샷
push	로컬 커밋을 원격 저장소에 올리는 것
pull	원격 변경을 로컬로 가져오는 것
branch	기존 코드와 분리된 작업 줄
PR	브랜치 변경을 합치기 전 검토하는 제안

용어	쉬운 설명
Preview	운영 반영 전 인터넷에서 확인하는 배포
Production	실제 고객이 사용하는 운영 환경
Supabase	PostgreSQL DB, 인증, 파일 저장 등을 제공하는 서비스
Vercel	웹 애플리케이션을 빌드하고 배포하는 플랫폼
webhook	외부 서비스가 상태 변화를 우리 서버에 알려주는 요청
cron	정해진 시각에 작업을 실행하는 스케줄
cache	자주 쓰는 결과를 빠르게 다시 주는 임시 저장
SEO	검색엔진과 사용자가 페이지를 이해하도록 개선하는 일
sitemap	검색할 주요 페이지 주소 목록
robots.txt	검색로봇의 접근 경로 안내 파일
canonical	비슷한 페이지 중 대표로 삼을 주소
noindex	검색 결과에 넣지 말라는 페이지 지시
AdSense	Google의 웹사이트 광고 수익 서비스
Jira	할 일과 버그, 상태를 이슈로 관리하는 도구
log	프로그램에서 일어난 일을 남긴 기록
monitoring	오류와 성능, 상태를 계속 관찰하는 일
backup	복구를 위해 따로 보관한 데이터 복사본

부록 7. 공식 문서와 업데이트 원칙

이 책의 기술 절차는 2026년 7월 기준으로 다음 공식 자료와 실제 프로젝트 문서를 확인해 작성했다.

- OpenAI Codex Manual과 Best practices: <https://learn.chatgpt.com/guides/best-practices>
- Git과 GitHub 기본: <https://docs.github.com/en/get-started/using-git/about-git>
- GitHub Pages: <https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages>
- Vercel Git 배포: <https://vercel.com/docs/git>
- Vercel 환경: <https://vercel.com/docs/deployments/environments>
- Supabase Database: <https://supabase.com/docs/guides/database/overview>
- Google SEO Starter Guide: <https://developers.google.com/search/docs/fundamentals/seo-starter-guide>
- Naver Search Advisor: <https://searchadvisor.naver.com/guide/seo-help>
- Google Maps JavaScript API: <https://developers.google.com/maps/documentation/javascript/get-api-key>
- Naver Maps API: <https://navermaps.github.io/maps.js/en/docs/>
- Kakao Map과 카카오톡 공유: <https://developers.kakao.com/docs/en>
- Toss Payments 결제위젯: <https://docs.tosspayments.com/en/integration-widget>

- Google AdSense: <https://support.google.com/adsense/answer/10162>
- Meta Threads API: 구현 시 Meta for Developers의 최신 Threads 문서를 다시 확인한다.

도구의 화면, 가격, 무료 한도, 권한 이름, API 버전은 바뀔 수 있다. 이 책의 명령을 무조건 복사하기보다 Codex에 “현재 공식 문서를 확인하고 이 프로젝트 버전에 맞게 적용해줘”라고 요청한다. 결제, 개인정보, 세금, 부동산과 금융 정책은 특히 공식 기관과 전문가의 최신 기준을 별도로 확인한다.

부록 8. 하나의 예약 서비스로 보는 50 단계

동네 클래스 예약 서비스를 예로 들어 각 부의 결과가 어떻게 이어지는지 본다. 사용자는 수업을 찾고 날짜를 선택해 예약 요청을 보낸다. 운영자는 요청을 확인하고 일정을 확정한다. 첫 버전에는 결제와 채팅을 넣지 않는다.

단계	서비스에 생기는 변화	완료 증거
1-6장	고객·문제·웹서비스 구조와 필요한 도구를 정한다	1페이지 문제 정의와 구조도
7-13장	Codex 작업 규칙, 폴더, 요청과 오류 해결 방식을 만든다	프로젝트 지도와 AGENTS.md
14-19장	수업 목록·날짜 선택·예약 요청·완료 화면을 로컬에서 만든다	모바일 핵심 흐름과 테스트
20-25장	변경을 GitHub에 기록하고 Preview 주소를 만든다	커밋, 원격 저장소, Preview URL
26-31장	사용자와 예약 데이터를 저장하고 권한을 분리한다	A 사용자가 B의 예약을 볼 수 없는 테스트
32-35장	필요할 때만 테스트 결제·지도·공유·업무 관리를 붙인다	성공·취소·실패와 운영 체크리스트
36-40장	공개 페이지의 검색 정책과 콘텐츠를 정한다	sitemap, robots, canonical과 실제 응답
41-44장	빈자리·새 수업 알림 콘텐츠를 dry-run으로 자동 생성한다	중복 방지 기록과 발행 전 검수 화면

단계	서비스에 생기는 변화	완료 증거
45-50장	오류·비용·백업을 확인하고 첫 고객 행동을 기록한다	운영 대시보드와 다음 30일 결정

첫 버전의 화면 지도

화면	사용자의 핵심 행동	반드시 필요한 상태
수업 목록	원하는 수업을 고른다	로딩·빈 목록·오류
수업 상세	날짜와 내용을 확인한다	마감·신청 가능
예약 입력	이름과 연락 방법을 입력한다	필수값·잘못된 형식
예약 확인	입력과 날짜를 다시 본다	중복 제출 방지
완료	예약 요청 결과와 다음 행동을 본다	성공·실패·재시도
운영 목록	운영자가 새 요청을 확인한다	권한 없음·필터·상태 변경

기능을 추가하고 싶을 때마다 위 여섯 화면의 핵심 행동이 먼저 안정적인지 확인한다. 후기, 채팅, 쿠폰, 구독은 실제 고객이 필요성을 보여준 뒤 검토한다.

부록 9. 장별 완료 증거 기록표

장·작업	만들거나 바꾼 파일	실행한 검사	내가 직접 확인한 화면	남은 위험	다음 행동

완료 보고에 성공이라는 단어가 있어도 증거가 없으면 완료가 아니다. 명령의 종료 코드, 통과한 테스트 수, 확인한 URL과 화면을 함께 남긴다.

부록 10. 자주 만나는 오류 실습 8가지

증상	먼저 분리할 것	Codex에 줄 증거	완료 기준
화면이 열리지 않는다	개발 서버·주소·포트	실행 명령과 터미널 오류	같은 주소가 새로고침 뒤 열림
배포만 실패한다	로컬과 배포 환경 차이	build 로그와 배포 로그	Preview build 성공
환경변수를 못 읽는다	이름·환경·재배포 여부	값이 아닌 변수 이름과 설정 위치	비밀값 노출 없이 서버에서 읽힘
로그인 후 원래 화면이 사라진다	callback과 세션 복원	로그인 전후 URL과 재현 순서	로그인 뒤 원래 작업 복원
다른 사용자 데이터가 보인다	인증과 권한 정책	두 테스트 계정의 행동	교차 접근 차단
결제가 중복된다	주문 상태와 승인 재시도	주문 ID와 테스트 이벤트	같은 주문이 한 번만 승인
모바일에서 버튼이 겹친다	화면 폭과 고정 요소	기기 크기와 캡처	360px에서 핵심 행동 완료
자동 글이 두 번 올라간다	중복 기준과 영구 저장	콘텐츠 키와 실행 로그	재실행해도 같은 항목 미발행

부록 11. 판본과 업데이트 확인표

이 원고는 2026년 7월의 공식 문서와 프로젝트 상태를 기준으로 작성했다. 독자는 실행 전에 다음을 다시 확인한다.

- ☐ OpenAI Codex의 현재 설치·로그인·권한 방식
- ☐ Next.js와 Node.js의 지원 버전
- ☐ Vercel의 환경과 요금 정책
- ☐ Supabase의 인증·RLS·Storage 정책
- ☐ 결제사의 최신 연동·환불·웹훅 문서
- ☐ Google·Naver·Kakao의 API 키와 사용량 정책
- ☐ Search Console·Search Advisor·AdSense 등록 조건
- ☐ Threads API의 권한·발행 절차와 제한
- ☐ 개인정보·전자상거래·세금에 관한 최신 공식 기준

도구 화면이 달라졌다면 책의 오래된 버튼 위치를 억지로 찾지 않는다. 목표와 완료 조건을 Codex에 주고 현재 공식 문서를 기준으로 절차를 다시 설명하게 한다.



경험편. 아이디어를 결제로 바꾸는 법

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

이제부터는 Codex가 없던 시절 개발자와 함께 모바일 청첩장 서비스를 만들고, 고객을 얻고, 자동화하고, B2B로 확장하고, 결국 매각하기까지의 경험을 다룬다. 앞의 실전편이 '어떻게 만드는가'라면 경험편은 '무엇을 왜 만들어야 하는가'에 답한다.



프로로그

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

만들고 싶은 사람과 결제받는 사람의 차이

나는 개발자가 아니다. 대학에서 시각디자인을 전공했고, UX와 모바일 서비스 디자인을 배웠다. 화면을 설계하고 사용자의 행동을 관찰하는 일에는 익숙했지만 서버를 만들거나 결제 시스템을 개발할 수는 없었다.

그런데도 개발자 한 명과 함께 유료 웹서비스를 만들었다. 고객이 자신의 정보를 입력하고 결제하면 모바일 청첩장이 자동으로 만들어지는 서비스였다. 약 반년 동안 준비해 2020년 하반기에 출시했고, 약 1년간 운영했다. 서비스가 알려진 뒤에는 별도의 홍보를 거의 하지 않아도 하루에 결제가 발생했다. 직장 생활과 병행하기 어려워졌을 때는 서비스 매각을 선택했다.

결과만 놓고 보면 깔끔한 성공담처럼 보일 수 있다. 하지만 실제 과정은 그렇지 않았다. 무엇을 만들어야 하는지 확신할 수 없었고, 개발 범위를 정하는 일도 어려웠다. 고객이 정말 돈을 낼지 알 수 없었으며, 출시 후에는 예상하지 못한 문의와 수정 요청이 이어졌다. 자동화된 서비스라고 생각했지만 사람이 처리해야 하는 운영 업무도 계속 생겼다. 공동으로 만든 서비스를 누가 얼마나 관리하고 수익을 어떻게 나눌지도 현실적인 문제가 되었다.

이 책은 그 과정을 미화하지 않으려고 한다.

나는 대학생 때 처음 스타트업에 합류했다. 스마트워치를 활용한 호텔 관리 서비스를 만들며 미국 호텔에서 테스트를 진행했고 투자도 받았다. 그러나 제품의 핵심이었던 스마트워치 배터리 문제를 해결하지 못했다. 이후 해커톤에서 만난 사람들과 피트니스 서비스를 만들었고, 블록체인 보안 스타트업에서도 일했다. 여러 회사를 거치며 서비스가 실패하거나 성장하는 장면을 가까이서 봤다.

그 경험을 통해 알게 된 사실이 하나 있다. 좋은 아이디어만으로는 아무 일도 일어나지 않는다는 것이다.

아이디어가 제품이 되려면 누군가의 구체적인 불편과 연결되어야 한다. 제품이 사업이 되려면 사용자의 행동이 결제로 이어져야 한다. 그리고 결제가 반복되려면 창업자가 계속 손으로 처리하지 않아도 되는 운영 구조가 필요하다.

내가 만든 모바일 청첩장 서비스도 처음부터 거대한 사업을 목표로 시작한 것은 아니었다. 이미 존재하는 상품을 사용하다가 작은 불편을 발견했고, 그 불편을 조금 더 매끄럽게 해결하는 방법을 생각했다. 시장을 새로 만든 것이 아니라 기존 고객이 불편하게 하고 있던 일을 더 쉽게 바꿨다.

이 책을 읽는 사람 중에는 메모장에 여러 개의 아이디어를 적어둔 사람이 있을 것이다. 개발자를 구하지 못해 멈춘 사람도 있을 것이고, 일단 만들었지만 아무도 결제하지 않아 낙담한 사람도 있을 것이다.

그렇다면 첫 질문은 “무엇을 만들까?”가 아니다.

“누가 지금 어떤 불편을 해결하기 위해 이미 시간이나 돈을 쓰고 있는가?”

이 질문에서 시작하면 아이디어는 막연한 상상이 아니라 검증할 수 있는 가설이 된다. 이 책은 그 가설을 첫 결제로 바꾸는 과정을 순서대로 보여주려 한다.



1부. 돈을 지불할 문제 찾기

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

1장. 사이드 프로젝트는 아이디어가 아니라 불편에서 시작한다



사이드 프로젝트를 시작하려는 사람은 대개 아이디어부터 찾는다.

“요즘 무엇이 유행하지?”

“어떤 앱을 만들면 사람들이 많이 사용할까?”

“AI를 붙이면 새로운 서비스가 되지 않을까?”

나도 새로운 산업과 기술에 관심이 많았다. 스마트워치가 등장했을 때는 손목 위에서 동작하는 서비스를 만들었고, 암호화폐가 주목받을 때는 블록체인 산업으로 옮겨갔다. 새로운 기술은 분명 기회를 만든다. 하지만 기술 자체가 고객의 결제를 보장하지는 않는다.

모바일 청첩장 서비스는 기술에서 출발하지 않았다. 이미 사람들이 사용하고 있는 모바일 청첩장을 보면서 느낀 불편에서 시작했다.

당시 모바일 청첩장은 낯선 상품이 아니었다. 예비부부는 종이 청첩장과 함께 모바일 청첩장을 준비하고 있었다. 그러나 상당수 서비스의 화면은 최신 스마트폰 환경에

맞지 않았고, 제작 과정도 매끄럽지 않았다. 고객이 정보를 전달하면 판매자가 내용을 확인해 수동으로 제작하는 방식이 많았다. 수정이 생길 때마다 다시 요청하고 결과를 기다려야 했다.

결혼식 인원 제한과 비대면 소통이 늘면서 또 다른 요구도 생겼다. 예비부부와 가족들은 모바일 청첩장에 계좌번호를 넣고 싶어 했지만, 계좌번호가 화면에 그대로 노출되는 방식은 부담스럽게 느꼈다. 정보를 제공해야 하지만 노골적으로 보이고 싶지는 않은 미묘한 문제였다.

이런 문제는 세상을 바꾸는 거대한 문제처럼 보이지 않는다. 하지만 좋은 사업의 출발점이 반드시 거대할 필요는 없다. 중요한 것은 다음 세 가지다.

- 1 실제로 그 일을 하는 사람이 존재하는가.
- 2 현재 해결 방식에 반복되는 불편이 있는가.
- 3 그 불편을 해결하기 위해 이미 돈을 쓰고 있는가.

모바일 청첩장에는 세 조건이 모두 있었다. 결혼을 준비하는 사람은 계속 생겼고, 모바일 청첩장은 이미 구매하는 상품이었다. 우리는 고객에게 새로운 소비 습관을 가르칠 필요가 없었다. 기존 구매 과정의 불편을 줄이고 결과물을 더 낮게 만들면 됐다.

이 차이는 매우 중요하다.

처음 서비스를 만드는 사람은 누구도 하지 않은 아이디어를 찾으려고 한다. 하지만 완전히 새로운 상품은 고객에게 상품의 필요성부터 설명해야 한다. 반대로 이미 돈이 오가는 시장에서는 고객이 무엇을 원하는지, 어디에서 불편을 느끼는지 관찰할 수 있다.

첫 사이드 프로젝트라면 혁신의 크기보다 검증 가능성을 선택하는 편이 낫다.

문제를 찾을 때 적어야 할 다섯 문장

나는 새로운 아이디어가 떠오르면 다음 질문부터 적는다.

- 이 문제를 겪는 사람은 누구인가?
- 그 사람은 지금 이 문제를 어떻게 해결하는가?

- 현재 방식에서 가장 번거로운 단계는 무엇인가?
- 이미 이 문제를 해결하기 위해 돈을 쓰고 있는가?
- 한 단계만 제거한다면 무엇을 없앨 수 있는가?

모바일 청첩장에 대입하면 다음과 같았다.

- 고객은 결혼을 준비하는 예비부부다.
- 고객은 온라인 제작 업체를 찾아 정보를 전달하고 결과물을 기다린다.
- 정보 전달과 수정 요청이 반복되고, 완성될 때까지 기다려야 한다.
- 고객은 이미 모바일 청첩장 제작비를 지불한다.
- 판매자의 수동 제작 단계를 없애고 고객이 직접 수정할 수 있게 한다.

마지막 문장이 서비스의 핵심이 되었다.

우리가 만들려던 것은 단순히 더 예쁜 모바일 청첩장이 아니었다. 고객이 직접 정보를 입력하고, 결과를 바로 확인하고, 필요하면 수정하고, 결제 후 곧바로 사용할 수 있는 제작 시스템이었다.

예쁜 화면은 경쟁력이 될 수 있다. 하지만 구매 과정을 바꾸는 것은 더 큰 경쟁력이 된다.

실행 노트

오늘 하루 동안 자신이 돈을 지불했던 서비스 세 개를 적어보자. 각 서비스에서 기다렸던 순간, 반복해서 입력한 정보, 누군가에게 요청해야 했던 작업을 찾아본다. 새로운 시장보다 이미 결제하고 있는 과정의 마찰을 찾는 것이 첫 유료 서비스에 더 가까운 출발점이다.

2장. 경쟁 서비스를 직접 구매해 보라

시장조사는 검색 결과를 정리하는 일로 끝나기 쉽다. 경쟁사의 가격, 기능, 화면을 표로 만들고 나면 시장을 이해했다고 생각한다. 하지만 고객이 실제로 겪는 경험은 소개 페이지에 적혀 있지 않다.

나는 모바일 청첩장 제작 과정을 알아보기 위해 결혼 준비 커뮤니티에 가입하고 여러 업체의 상품을 직접 구매했다.

직접 돈을 내는 순간 관찰자의 위치가 달라진다. 무료 체험에서는 넘어갈 수 있는 불편도 돈을 낸 고객의 입장에서는 선명하게 보인다. 결제 전에는 무엇이 불안한지, 결제 후에는 무엇을 기다려야 하는지, 수정을 요청할 때는 어떤 설명을 반복해야 하는지 알 수 있다.

경쟁 상품을 구매하며 확인해야 할 것은 기능의 개수가 아니다. 고객이 목표를 달성할 때까지 거치는 전체 과정이다.

결제 전

- 어떤 정보를 보고 상품을 신뢰하게 되는가?
- 가격과 결과물을 쉽게 비교할 수 있는가?
- 샘플을 충분히 확인할 수 있는가?
- 결제 전에 고객이 가장 걱정하는 것은 무엇인가?

결제 직후

- 고객이 무엇을 전달해야 하는가?
- 같은 정보를 여러 번 입력하는가?
- 완성까지 얼마나 기다려야 하는가?

- 현재 진행 상태를 알 수 있는가?

결과물을 받은 뒤

- 고객이 직접 수정할 수 있는가?
- 수정 요청을 어떤 방식으로 전달하는가?
- 작은 오타 하나를 고치는 데도 판매자가 필요한가?
- 다른 사람에게 공유하기까지 추가 작업이 필요한가?

이 과정을 따라가며 중요한 문제를 발견했다. 기존 서비스에서는 제작자와 고객이 계속 메시지를 주고받아야 했다. 고객에게도 번거롭지만 판매자에게도 노동집약적인 구조였다. 주문이 늘면 매출과 함께 제작 업무도 거의 같은 비율로 늘어날 수밖에 없었다.

우리는 이 문제를 고객 경험과 운영 효율이라는 두 방향에서 바라봤다.

고객은 기다리지 않고 직접 만들 수 있어야 했다. 운영자는 주문이 늘어도 매번 같은 내용을 복사하고 수정하지 않아야 했다. 한 번의 자동화로 양쪽의 문제를 함께 줄일 수 있었다.

여기에서 첫 번째 제품 원칙이 나왔다.

고객이 판매자에게 요청해야 하는 일을 가능한 한 고객 스스로 끝낼 수 있게 한다.

이 원칙은 모바일 청첩장에만 적용되는 것이 아니다. 예약, 견적, 보고서, 이미지 제작, 문서 생성처럼 고객의 정보를 받아 일정한 결과물을 제공하는 서비스라면 같은 방식으로 살펴볼 수 있다.

경쟁 서비스 분석표

단계	고객 행동	불편	자동화 가능성	우리가 제공할 변화
탐색	업체와 샘플 비교	결과 예측이 어려움	중간	완성본에 가까운 샘플 제공
주문	정보 전달	양식이 복잡하고 누락 발생	높음	단계별 입력 화면 제공
제작	결과 대기	진행 상황을 알 수 없음	높음	입력 즉시 미리보기 생성
수정	판매자에게 요청	응답을 기다려야 함	높음	고객 직접 수정 기능
사용	링크 전달	기능과 사용법이 불명확	중간	공유 중심의 완료 화면

경쟁사를 분석할 때 “우리에게 없는 기능”만 찾으면 기능 경쟁에 빠진다. 대신 “고객이 기다리거나 요청해야 하는 순간”을 찾으면 구조를 개선할 수 있다.

실행 노트

만들고 싶은 분야에서 유료 상품을 최소 두 개 구매해 보자. 결제부터 결과물을 받는 순간까지 화면을 기록하고, 기다린 시간과 판매자에게 질문한 내용을 적는다. 불편을 평가하는 데 쓴 구매 비용은 가장 저렴한 사용자 조사비가 될 수 있다.

3장. 고객의 말보다 고객이 이미 하고 있는 행동을 본다

사람들은 좋은 아이디어라고 쉽게 말한다. 지인에게 서비스 아이디어를 설명하면 대부분 응원해 준다. 하지만 “좋다”는 반응과 “돈을 내겠다”는 행동 사이에는 큰 간격이 있다.

그래서 초기에는 고객의 의견보다 고객이 이미 하고 있는 행동을 관찰하는 편이 낫다.

모바일 청첩장 고객은 이미 다음과 같은 행동을 하고 있었다.

- 결혼 준비 커뮤니티에서 제작 업체를 검색했다.
- 여러 샘플과 가격을 비교했다.
- 마음에 드는 업체에 자신의 정보를 전달했다.
- 수정 요청을 위해 판매자와 메시지를 주고받았다.
- 계좌번호 노출 방식과 송금 기능에 관심을 보였다.

이 행동은 시장이 존재한다는 신호였다. 우리가 확인해야 했던 것은 모바일 청첩장이 필요한지 여부가 아니었다. 기존 방식보다 직접 제작하고 수정하는 방식을 선택할 것인지, 그리고 그 편리함에 돈을 낼 것인지였다.

초기 서비스 검증에서 가장 좋은 질문은 “이 서비스가 있으면 사용하시겠어요?”가 아니다. 사람들은 미래의 행동을 정확히 예측하지 못한다. 다음처럼 과거와 현재의 행동을 물어야 한다.

- 마지막으로 이 문제를 겪은 때는 언제인가?
- 그때 어떤 방법으로 해결했는가?
- 해결하는 데 얼마나 많은 시간과 돈을 썼는가?

- 과정에서 가장 불편했던 순간은 무엇인가?
- 다른 서비스를 비교하거나 포기한 적이 있는가?

답변이 구체적일수록 문제가 실제로 존재할 가능성이 높다. 반대로 “있으면 좋을 것 같다”는 말만 반복되고, 현재 아무런 해결 행동도 하지 않는다면 결제로 이어질 가능성은 낮다.

기능 요청을 그대로 만들지 않는다

고객은 종종 자신이 원하는 기능을 말한다. 하지만 창업자가 들어야 하는 것은 기능 이름보다 그 기능을 요청한 이유다.

예를 들어 고객이 “계좌번호를 숨길 수 있으면 좋겠다”고 말했다면 단순히 숨김 버튼을 추가하는 데서 끝낼 수 있다. 그러나 그 뒤에는 “축하를 요청하는 것처럼 보이고 싶지 않다”는 감정이 있었다. 그렇다면 제품은 정보를 감추는 것뿐 아니라 필요할 때 자연스럽게 확인하도록 경험을 설계해야 한다.

우리는 계좌번호를 화면에 계속 노출하는 대신 사용자가 버튼을 눌렀을 때 확인할 수 있는 방식을 적용했다. 카카오톡 송금과 연결되는 기능도 함께 고려했다. 기능 하나가 아니라 고객이 느끼는 어색함을 줄이는 흐름을 만든 것이다.

이 경험을 통해 기능 요구를 해석하는 세 단계가 생겼다.

- 1 고객이 요청한 기능을 적는다.
- 2 그 기능이 필요한 상황과 감정을 묻는다.
- 3 같은 문제를 더 단순하게 해결할 방법을 찾는다.

고객의 말은 중요한 단서다. 그러나 고객이 말한 화면을 그대로 만드는 것이 UX는 아니다. 고객이 이루려는 목적을 이해하고 가장 적은 단계로 해결하는 것이 제품 설계다.

첫 검증의 기준은 칭찬이 아니라 작은 약속이다

아직 제품이 없다면 결제 대신 더 작은 행동을 요청할 수 있다.

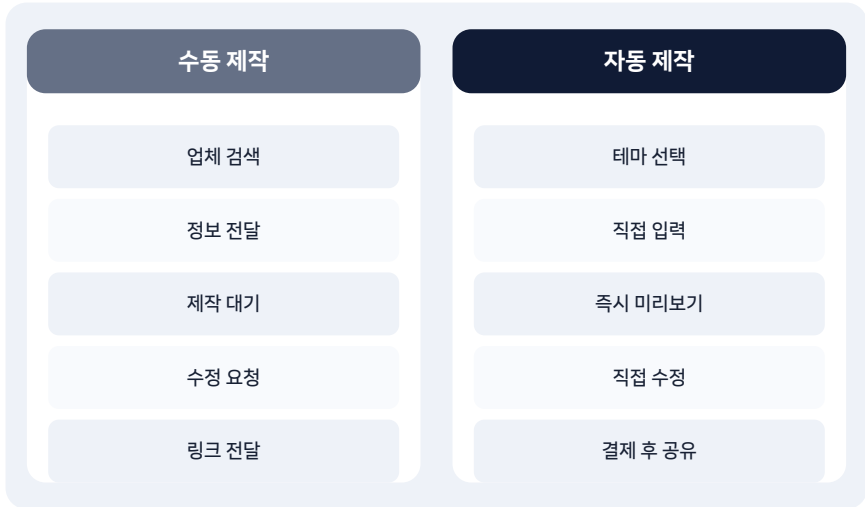
- 출시 알림을 받기 위해 연락처를 남기는가?
- 자신의 실제 자료를 제공하고 샘플 제작을 요청하는가?
- 일정 금액을 예약금으로 지불하는가?
- 지인에게 서비스를 소개하는가?
- 다시 사용하기 위해 저장하거나 재방문하는가?

시간, 개인정보, 평판, 돈처럼 고객이 무엇인가를 내놓는 행동은 단순한 칭찬보다 강한 신호다.

실행 노트

아이디어를 다섯 명에게 설명하고 의견을 묻는 대신, 해당 문제를 최근에 겪은 세 명을 찾는다. 그들이 실제로 했던 행동과 지불한 비용을 기록한다. 인터뷰가 끝날 때는 출시 알림 신청, 샘플 요청 또는 예약처럼 작은 행동 하나를 제안한다.

4장. 완전히 새로운 아이디어가 아니어도 되는 이유



처음 창업을 꿈꿀 때는 누구도 생각하지 못한 아이디어를 가져야 한다고 믿기 쉽다. 이미 비슷한 서비스가 있으면 늦었다고 생각하고, 검색 결과가 나오지 않으면 비로소 기회라고 느낀다.

하지만 경쟁 서비스가 없다는 사실은 두 가지 의미를 가질 수 있다. 아무도 발견하지 못한 기회일 수도 있지만, 고객이 돈을 낼 만큼 중요하지 않은 문제일 수도 있다.

첫 서비스를 만드는 사람에게는 이미 거래가 일어나는 시장이 오히려 안전하다. 고객이 누구인지 찾을 수 있고, 가격 범위를 알 수 있으며, 구매 전후의 행동을 관찰할 수 있기 때문이다. 모바일 청첩장 시장에도 이미 여러 제작 업체가 있었다. 우리의 출발점은 새로운 상품의 발명이 아니라 기존 제작 방식의 재설계였다.

기존 방식은 대체로 다음과 같았다.

- 1 고객이 판매자를 찾는다.
- 2 샘플을 보고 상품을 구매한다.
- 3 제작에 필요한 정보를 전달한다.
- 4 판매자가 수동으로 결과물을 만든다.
- 5 고객이 확인하고 수정 사항을 보낸다.
- 6 판매자가 수정한 뒤 다시 전달한다.

우리는 이 과정에서 판매자가 반복하던 작업을 시스템으로 옮기려고 했다.

- 1 고객이 테마를 선택한다.
- 2 필요한 정보를 직접 입력한다.
- 3 결과물을 바로 확인한다.
- 4 자신이 원하는 만큼 수정한다.
- 5 결제 후 링크를 공유한다.

상품은 같아 보이지만 사업 구조는 달라진다. 수동 제작에서는 주문이 늘수록 작업량도 늘어난다. 자동 제작에서는 초기 개발 비용이 크지만 주문 한 건이 추가될 때 증가하는 작업량은 상대적으로 작다.

이것이 우리가 만들고 싶었던 차별점이었다.

차별화는 기능의 수가 아니라 구조에서 나온다

경쟁사를 이기기 위해 기능을 더 많이 넣어야 한다고 생각할 수 있다. 그러나 고객이 구매하는 이유는 기능표의 길이가 아니다. 기존보다 더 빠르고, 더 쉽고, 덜 불안하게 목표를 달성할 수 있기 때문이다.

우리의 핵심 차별화는 다음 세 문장으로 정리할 수 있었다.

- 기다리지 않고 직접 만든다.
- 판매자에게 요청하지 않고 직접 수정한다.

- 입력한 결과를 바로 확인하고 사용한다.

명확한 차별화는 개발 범위를 정하는 기준도 된다. 새 기능을 제안받을 때 이 세 문장에 도움이 되는지 물었다. 도움이 되지 않는 기능은 첫 버전에서 제외할 수 있었다.

개선형 아이디어를 찾는 네 가지 관점

완전히 새로운 아이디어 대신 다음과 같은 변화를 찾아보자.

- 대기 제거: 사람이 처리할 때까지 기다리는 단계를 없앨 수 있는가?
- 요청 제거: 고객이 담당자에게 부탁해야 하는 일을 직접 할 수 있게 만들 수 있는가?
- 반복 제거: 판매자가 주문마다 반복하는 일을 자동화할 수 있는가?
- 불안 제거: 고객이 결과나 진행 상태를 알 수 없어 느끼는 불안을 줄일 수 있는가?

좋은 사이드 프로젝트는 세상을 처음부터 다시 만들지 않는다. 이미 돈이 오가는 과정에서 한두 개의 큰 마찰을 제거한다.

실행 노트

경쟁 서비스 세 개의 구매 과정을 나란히 적어본다. 모든 서비스에서 공통으로 기다리거나 요청해야 하는 단계를 표시한다. 그중 하나를 제거했을 때 고객과 판매자 모두에게 이익이 생기는지 확인한다.



2부. 비개발자가 서비스를 만드는 방법

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

5장. 개발자보다 먼저 찾아야 할 것

비개발자가 서비스를 만들려고 하면 가장 먼저 “개발자를 어디에서 구해야 할까?”를 고민한다. 나 역시 개발자가 없으면 아무것도 시작할 수 없다고 생각한 적이 있다.

그러나 개발자를 찾기 전에 해야 할 일이 있다. 무엇을 만들지, 누구를 위해 만들지, 고객이 왜 돈을 낼지를 설명할 수 있어야 한다.

아이디어가 정리되지 않은 상태에서 개발자를 만나면 대화는 기능 이야기로 흐른다.

“회원가입이 필요해요.”

“결제도 있어야 해요.”

“관리자 페이지도 만들어야 하고요.”

“테마를 여러 개 고를 수 있으면 좋겠어요.”

이 목록만으로는 우선순위를 정할 수 없다. 개발자는 요구받은 기능을 구현할 수 있지만 어떤 기능이 고객의 결제를 만드는지는 대신 결정해 줄 수 없다.

개발자를 찾기 전에 최소한 다음 다섯 가지를 한 장으로 정리해야 한다.

- 1 고객: 누가 사용하는가?
- 2 상황: 언제 이 서비스를 찾는가?
- 3 문제: 현재 방식에서 무엇이 가장 불편한가?
- 4 핵심 행동: 고객이 서비스 안에서 반드시 끝내야 하는 일은 무엇인가?
- 5 결제 이유: 무료가 아니라 돈을 낼 이유는 무엇인가?

모바일 청첩장 서비스는 다음처럼 정리할 수 있었다.

- 고객은 결혼을 준비하는 예비부부다.

- 기존 제작 업체를 찾고 정보를 전달하는 시점에 서비스를 발견한다.
- 수동 제작과 반복되는 수정 요청이 번거롭다.
- 고객은 정보를 입력해 공유 가능한 모바일 청첩장을 완성해야 한다.
- 기다리지 않고 직접 제작·수정할 수 있으며 디자인 품질도 확보할 수 있기 때문에 결제한다.

이 문서가 있으면 개발자와 기능을 논의할 때 기준점이 생긴다. 고객의 핵심 행동을 완성하는 기능은 먼저 만들고, 그렇지 않은 기능은 미룬다.

화면보다 먼저 그릴 것

디자이너는 화면부터 만들기 쉽다. 보기 좋은 홈 화면과 여러 테마를 먼저 디자인하면 서비스가 빠르게 구체화되는 느낌을 받는다. 그러나 초기에는 화면보다 고객의 이동 경로를 먼저 그리는 편이 좋다.

모바일 청첩장의 핵심 흐름은 단순했다.

샘플 확인 -> 테마 선택 -> 정보 입력 -> 미리보기 -> 수정 -> 결제 -> 공유

각 단계에서 고객이 다음으로 이동하지 못하는 이유를 적었다. 샘플이 부족하면 테마를 고르지 못하고, 입력 항목이 복잡하면 중간에 포기하며, 결제 전에 결과를 볼 수 없으면 불안해진다. 이탈 이유가 보이면 화면의 목적도 명확해진다.

실행 노트

개발자를 찾기 전에 서비스 설명을 한 장으로 작성한다. 기능 목록은 열 개를 넘기지 않는다. 그중 고객이 결제하기 전에 반드시 필요한 기능 세 개에 표시한다. 나머지는 개발자와 만나기 전에 한 번 더 삭제한다.

6장. 개발자 한 명과 반년 동안 MVP 만들기

나는 과거 스타트업에서 함께 일했던 개발자와 모바일 청첩장 서비스를 만들었다. 서로의 업무 방식을 어느 정도 알고 있다는 점은 큰 장점이었다. 하지만 친분이 있다고 해서 협업이 저절로 잘되는 것은 아니었다.

둘 다 직장인이었기 때문에 본업이 끝난 뒤에야 사이드 프로젝트를 진행할 수 있었다. 짧은 시간 안에 기획, 디자인, 개발, 테스트를 나눠야 했다. 약 반년의 준비 기간이 필요했다.

초기 역할은 비교적 분명했다. 나는 고객 조사, 서비스 기획, UX와 UI 디자인, 상품 구성과 판매 준비를 담당했다. 개발자는 데이터 구조, 사용자 기능, 결제와 서비스 운영에 필요한 기술 구현을 맡았다.

역할이 다르더라도 결정은 계속 함께 내려야 했다. 디자이너가 생각한 간단한 수정이 개발에서는 큰 구조 변경일 수 있었고, 개발자가 빠르게 만들 수 있다고 생각한 방식이 고객에게는 복잡할 수 있었다.

좋은 협업은 완벽한 기획서를 넘기는 일이 아니다

비개발자는 모든 화면을 완성한 뒤 개발자에게 전달해야 한다고 생각하기 쉽다. 반대로 개발자는 요구사항이 완전히 정리되기를 기다릴 수 있다. 그러나 초기 제품은 만들면서 배우는 일이 많다.

우리가 필요했던 것은 완벽한 문서보다 결정의 기준이었다.

- 이 기능이 없으면 고객이 결과물을 완성할 수 없는가?
- 고객이 결제하기 전에 반드시 확인해야 하는가?
- 사람이 임시로 처리해도 되는가?

- 첫 고객의 반응을 본 뒤 만들어도 되는가?

예를 들어 관리자 기능을 처음부터 완벽하게 만들려고 하면 출시가 늦어진다. 주문 수가 적을 때는 일부 운영을 사람이 처리하고, 반복되는 업무가 확인되면 그때 자동화할 수 있다.

사이드 프로젝트의 일정은 기능이 아니라 시간으로 정한다

본업이 있는 팀은 매일 같은 시간을 투입하기 어렵다. 그래서 “모든 기능이 완성되면 출시한다”는 계획은 계속 밀린다. 출시일을 먼저 정하고 그때까지 완성할 범위를 줄이는 편이 현실적이다.

우리는 제한된 시간 안에 고객이 모바일 청첩장을 완성하고 사용할 수 있는 한 줄의 흐름에 집중했다. 멋진 아이디어가 생겨도 핵심 흐름과 무관하면 뒤로 미뤘다.

MVP는 조잡한 제품이라는 뜻이 아니다. 고객에게 약속한 핵심 가치만큼은 제대로 작동하되, 아직 검증되지 않은 주변 기능을 줄인 제품이다.

실행 노트

팀원이 일주일에 실제로 사용할 수 있는 시간을 서로 적는다. 희망 시간이 아니라 최근 한 달 동안 확보할 수 있었던 시간을 기준으로 한다. 그 시간으로 네 주 안에 완성할 수 있는 범위만 첫 개발 목록에 남긴다.

7장. 기능을 더하는 것보다 빼는 것이 어려웠다

A	결제를 만드는 기능	없으면 고객 목표가 완성되지 않는다
B	구매 불안을 줄이는 기능	샘플-미리보기-명확한 안내
C	구매 후 만족 기능	첫 결제 검증 뒤 순차적으로 확장

서비스를 만들기 시작하면 아이디어가 계속 늘어난다. 모바일 청첩장에도 넣고 싶은 기능이 많았다. 다양한 테마, 사진과 영상, 방명록, 참석 의사 확인, 계좌번호 확인, 간편 송금, 관리자 기능, 제휴 업체 기능까지 모두 유용해 보였다.

문제는 각각의 기능이 디자인과 개발로 끝나지 않는다는 것이다. 테스트, 오류 대응, 고객 안내, 유지보수까지 따라온다. 기능 하나는 화면 하나가 아니라 운영할 약속 하나다.

초기 기능을 결정할 때 다음 세 등급으로 나눌 수 있다.

A. 결제를 만드는 기능

이 기능이 없으면 고객이 돈을 낼 이유가 사라진다. 모바일 청첩장에서는 정보 입력, 결과 확인, 수정, 사용 가능한 링크 생성이 여기에 해당했다.

B. 구매 불안을 줄이는 기능

고객이 결제 전에 품질과 결과를 판단하도록 돕는다. 샘플, 미리보기, 명확한 제작 방식 안내가 여기에 해당한다.

C. 구매 후 만족을 높이는 기능

있으면 좋지만 첫 결제를 검증한 뒤 만들어도 된다. 부가적인 소통 기능과 여러 확장 기능이 여기에 해당할 수 있다.

첫 버전에서는 A를 완성하고 B를 필요한 만큼 제공해야 한다. C를 계속 넣기 시작하면 고객을 만나기도 전에 시간과 비용을 소진한다.

가장 위험한 문장

사이드 프로젝트에서 가장 위험한 문장은 “이왕 만드는 김에”다.

이왕 만드는 김에 회원 기능을 더하고, 이왕 만드는 김에 관리자 통계를 만들고, 이왕 만드는 김에 모바일 앱도 준비한다. 각각은 합리적으로 들리지만 모두 합치면 출시할 수 없는 제품이 된다.

기능을 미루는 것은 포기가 아니다. 고객이 실제로 원하는지 확인할 때까지 투자를 보류하는 일이다.

실행 노트

현재 기능 목록 옆에 A, B, C를 표시한다. C 기능은 모두 다음 버전으로 옮긴다. A 기능만으로 고객이 처음부터 끝까지 목표를 달성할 수 있는지 직접 시뮬레이션한다.

8장. 고객이 스스로 입력하고 결제하게 만들기

우리 서비스의 핵심은 예쁜 청첩장 테마가 아니었다. 고객이 판매자의 도움 없이 제작을 끝내는 구조였다.

자동화 서비스라고 하면 복잡한 기술부터 떠올리기 쉽다. 하지만 고객 입장에서 자동화는 단순하다. 필요한 정보를 한 번 입력하고, 결과를 즉시 확인하며, 막히는 순간 없이 다음 단계로 이동하는 것이다.

입력 화면은 제작 양식이 아니라 대화다

판매자가 고객에게 직접 질문할 때는 이해하지 못한 부분을 다시 설명할 수 있다. 온라인 입력 화면에서는 그 역할을 문장과 순서가 대신해야 한다.

입력 항목을 설계할 때는 다음을 확인해야 한다.

- 고객이 이 정보가 왜 필요한지 아는가?
- 어떤 형식으로 입력해야 하는지 예시가 있는가?
- 나중에 수정할 수 있다는 사실을 아는가?
- 한 화면에서 너무 많은 정보를 요구하지 않는가?
- 아직 준비되지 않은 항목을 건너뛸 수 있는가?

결혼 준비 과정에서는 모든 정보가 한 번에 확정되지 않는다. 사진이 준비되지 않았거나, 부모님 계좌번호를 나중에 입력해야 할 수 있다. 고객이 모든 자료를 준비할 때까지 시작조차 할 수 없다면 이탈한다. 저장과 수정의 자유가 중요한 이유다.

결제 전에 결과를 보여준다

디지털 결과물은 구매 전에 품질을 판단하기 어렵다. 고객이 정보를 모두 입력했는데 결제 후에야 결과를 볼 수 있다면 불안이 커진다.

그래서 미리보기는 단순한 편의 기능이 아니라 결제를 돕는 기능이다. 고객이 자신의 이름, 사진, 일정이 들어간 결과를 보는 순간 상품은 남의 샘플에서 자신의 결과물로 바뀐다.

결제는 그다음에 일어난다.

자동화의 목표는 무인 운영이 아니다

고객이 직접 제작한다고 해서 문의가 사라지는 것은 아니다. 어떤 문구를 써야 할지, 사진이 왜 잘리는지, 송금 기능은 어떻게 동작하는지 질문이 생긴다.

자동화의 목표를 “사람이 아무것도 하지 않는 것”으로 잡으면 현실과 충돌한다. 더 좋은 목표는 반복적인 제작 업무를 줄이고, 사람이 고객의 예외적인 문제에 집중하게 만드는 것이다.

첫 버전 이후 실제로 확장된 기능

운영 중에는 테마 선택 폭이 늘어 최종적으로 13개 테마를 안내했고, 카카오페이 송금, 방명록, 참석 의사 확인, 유튜브 라이브 연결 같은 기능도 제공했다. 고객 리뷰에서는 자유로운 수정과 함께 이 부가 기능들이 자주 언급됐다.

중요한 점은 이 기능들을 모두 첫날의 필수 기능으로 착각하지 않는 것이다. 핵심은 여전히 고객이 자신의 정보를 입력하고 바로 사용 가능한 청첩장을 완성하는 과정이었다. 부가 기능은 핵심 흐름이 작동한 뒤 고객의 실제 사용 장면에 맞춰 확장할 수 있었다.

실행 노트

서비스의 입력 과정을 처음 보는 사람에게 맡긴다. 설명하지 말고 관찰한다. 질문한 항목, 멈춘 화면, 잘못 입력한 값을 기록한다. 사용자를 교육하려 하지 말고 화면이 설명하도록 수정한다.

9장. 디자인과 개발 사이에서 반드시 문서로 남길 것

두 명이 만드는 작은 서비스에서는 문서가 필요 없다고 생각하기 쉽다. 바로 이야기 하면 되고, 메신저에 기록이 남는다고 생각한다. 하지만 시간이 지나면 서로 다른 결정을 기억하고, 같은 문제를 반복해서 논의하게 된다.

거대한 기획서가 필요한 것은 아니다. 다음 다섯 가지는 짧게라도 남겨야 한다.

1. 핵심 사용자 흐름

고객이 처음 들어와 결과물을 사용하기까지의 순서다. 흐름이 바뀌면 어떤 화면과 데이터가 함께 바뀌는지 판단할 수 있다.

2. 기능의 완료 조건

“사진 업로드 기능”이라고만 쓰지 않는다. 지원 형식, 용량, 자르기 방식, 실패했을 때 안내처럼 완료를 판단할 기준을 적는다.

3. 예외 상황

결제가 실패하거나, 입력 도중 창을 닫거나, 링크를 잃어버리거나, 같은 정보를 중복 제출할 수 있다. 정상 흐름보다 예외 상황에서 고객 문의가 더 많이 발생한다.

4. 결정과 이유

무엇을 선택했는지만 남기면 나중에 같은 논의를 반복한다. 왜 그 결정을 했는지 한 줄을 함께 적는다.

5. 출시 후 목록

첫 버전에서 제외한 기능을 삭제하지 말고 별도 목록으로 옮긴다. 고객 반응이 확인되면 다시 우선순위를 정한다.

말로 합의한 수익 배분도 문서가 필요하다

제품 문서만큼 중요한 것이 협업 문서다. 친한 사이일수록 역할, 비용, 수익, 소유권을 말로만 합의하기 쉽다. 그러나 서비스가 예상보다 잘되거나 한 사람의 참여 시간이 줄어들면 처음의 합의가 문제로 돌아온다.

최소한 다음 내용은 시작 전에 적어야 한다.

- 각자 맡은 역할과 예상 투입 시간
- 서버비·광고비·외주비 등 비용 부담 방식
- 매출이 아니라 비용을 제외한 이익의 계산 방식
- 수익 배분 비율과 정산 주기
- 소스코드, 디자인, 상표, 도메인의 소유 관계
- 한 사람이 프로젝트를 중단할 때 처리 방법
- 서비스 매각 제안이 왔을 때 의사결정 방식

문서는 상대를 의심해서 만드는 것이 아니다. 서로 다른 기대를 미리 발견하기 위해 만든다.

실행 노트

프로젝트의 핵심 흐름, 역할, 비용, 수익 배분을 각각 한 페이지로 작성한다. 팀원과 함께 읽으며 같은 단어를 다르게 이해하는 부분에 표시한다. 합의가 끝난 문서는 날짜와 함께 보관한다.

3부. 첫 고객과 첫 결제 만들기

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

10장. 출시했다고 고객이 오지는 않는다

제품을 만드는 동안에는 출시가 결승선처럼 느껴진다. 기능을 완성하고 결제를 연결하면 이제 고객이 찾아올 것이라고 기대한다.

하지만 출시 버튼은 사업의 시작을 알릴 뿐 고객을 데려오지 않는다.

초기 서비스에는 검색 기록도, 리뷰도, 브랜드 인지도도 없다. 고객 입장에서는 처음 보는 업체에 결혼식 정보를 맡기고 돈을 지불해야 한다. 서비스가 실제로 작동하는지, 문제가 생겼을 때 대응해 줄지 알 수 없다.

첫 고객을 얻으려면 제품과 함께 신뢰를 만들어야 한다.

첫 판매 전에 준비할 네 가지

빈 템플릿보다 이름, 사진, 일정이 채워진 결과물이 이해하기 쉽다.

1 실제 사용 장면에 가까운 샘플

고객이 무엇을 입력하고 언제 결과를 받는지 단계별로 보여준다.

1 제작 과정을 보여주는 설명

초기 고객은 기능보다 운영자를 믿고 구매하는 경우가 많다.

1 문제가 생겼을 때 연락할 방법

무엇이 포함되고 고객이 어디까지 직접 수정할 수 있는지 알려준다.

1 명확한 가격과 수정 범위

첫 고객을 모으는 일은 대규모 광고보다 작은 집단에서 시작하는 편이 낫다. 고객이 모여 있고 문제에 관해 이미 대화하는 곳을 찾아야 한다.

모바일 청첩장의 경우 결혼 준비 커뮤니티가 그런 장소였다. 예비부부는 예식장, 사진, 청첩장, 답례품을 비교하고 실제 경험을 공유하고 있었다. 서비스의 타깃을 새로 모을 필요 없이 이미 모여 있는 곳으로 가면 됐다.

출시와 유통을 동시에 설계한다

제품 기획 단계에서 다음 질문을 함께 해야 한다.

- 고객은 어디에서 이 문제를 검색하는가?
- 구매 전에 누구의 후기를 믿는가?
- 어떤 검색어로 경쟁 상품을 찾는가?
- 제품을 한 문장으로 소개하면 무엇이라고 말할까?

이 질문에 답하지 못하면 제품이 완성된 뒤 마케팅 채널을 처음부터 찾아야 한다. 반대로 유통 경로가 보이면 상품명과 설명, 샘플 구성도 고객의 언어에 맞춰 설계할 수 있다.

실행 노트

출시 전에 잠재 고객이 모인 커뮤니티 다섯 곳을 찾는다. 각 공간에서 자주 등장하는 질문과 사용자가 실제로 쓰는 표현을 기록한다. 광고 글을 올리기 전에 먼저 질문과 답변의 흐름을 이해한다.

11장. 결혼 준비 커뮤니티에서 고객을 만나다

결혼 준비는 정보 탐색이 많은 과정이다. 예비부부는 여러 업체의 가격과 결과물을 비교하고, 먼저 경험한 사람의 후기를 찾는다. 모바일 청첩장도 마찬가지였다.

우리는 결혼 준비 카페와 블로그, 소셜미디어에서 서비스를 알렸다. 단순히 링크를 반복해서 올리는 것보다 고객이 궁금해하는 지점에 답하는 콘텐츠가 필요했다.

- 계좌번호를 부담스럽지 않게 보여주는 방법
- 모바일 청첩장을 직접 수정할 수 있는지
- 종이 청첩장과 다른 사진을 사용할 수 있는지
- 방명록과 참석 의사 확인은 어떻게 사용하는지
- 제작에 얼마나 시간이 걸리는지

이런 질문은 상품 설명의 재료가 되었다. 고객 문의에 한 번 답하고 끝내지 않고 상세 페이지와 안내 문구에 반영하면 다음 고객의 불안을 미리 줄일 수 있었다.

커뮤니티에서는 판매자보다 참여자가 되어야 한다

사람들이 정보를 나누는 공간에 들어가 상품만 홍보하면 거부감을 만든다. 먼저 기존 질문을 읽고, 제품 구매 여부와 상관없이 도움이 되는 정보를 제공해야 한다.

초기에는 다음 순서가 유효하다.

- 1 반복되는 질문을 수집한다.
- 2 질문에 답하는 짧은 콘텐츠를 만든다.
- 3 실제 결과물과 사용 방법을 보여준다.
- 4 체험할 사람을 제한적으로 모집한다.

5 사용 중 막힌 부분을 직접 듣는다.

이 과정의 목적은 단순한 노출이 아니다. 우리가 예상한 문제가 실제 고객에게도 중요한지 확인하는 일이다.

첫 고객은 매출과 동시에 제품을 준다

초기 고객은 완성된 제품을 소비하는 사람만이 아니다. 설명이 부족한 화면, 예상하지 못한 입력 방식, 중요하게 생각하지 않았던 기능을 발견하게 해준다.

모바일 청첩장 리뷰에는 우리가 강조하지 않았던 가치가 반복해서 나타났다. 고객은 디자인이 깔끔하다는 점뿐 아니라 직접 수정할 수 있다는 점, 계좌번호를 필요한 때에 확인할 수 있다는 점, 빠르게 완성할 수 있다는 점을 높게 평가했다.

창업자가 생각한 장점과 고객이 구매한 이유가 다를 수 있다. 고객의 표현을 모으면 다음 상세페이지의 제목이 보인다.

실행 노트

첫 고객 열 명의 문의와 후기를 문장 단위로 분리한다. 반복되는 표현에 표시하고, 가장 많이 등장한 세 문장을 상품 설명의 상단에 배치한다. 창업자의 전문용어보다 고객의 표현을 우선한다.

12장. 후기 한 줄이 광고보다 강했던 이유

결혼식은 다시 되돌리기 어려운 행사다. 고객은 가격이 저렴하다는 이유만으로 처음 보는 서비스를 선택하지 않는다. 다른 사람이 실제로 사용했고 문제가 없었다는 증거가 필요하다.

초기 리뷰에서 반복된 반응은 구체적이었다.

- 직접 수정할 수 있어 편했다.
- 계좌번호가 계속 노출되지 않아 좋았다.
- 예상보다 빠르게 만들 수 있었다.
- 받은 사람들이 깔끔하고 예쁘다고 말했다.
- 방명록과 참석 의사 확인 기능이 유용했다.

이런 후기는 “최고의 모바일 청첩장”이라는 광고 문구보다 강하다. 고객이 걱정하는 상황과 사용 후의 변화를 함께 보여주기 때문이다.

좋은 후기는 감탄보다 맥락이 있다

판매자는 별점과 칭찬의 수를 늘리고 싶어 한다. 그러나 다음 고객에게 도움이 되는 후기는 세 가지를 포함한다.

- 1 구매 전 어떤 문제가 있었는가?
- 2 어떤 기능이나 과정이 문제를 해결했는가?
- 3 사용 후 무엇이 달라졌는가?

후기를 요청할 때도 “좋은 리뷰를 써주세요”보다 다음과 같이 묻는 편이 낫다.

- 구매 전에 가장 걱정했던 점은 무엇인가요?
- 직접 사용하며 가장 편했던 기능은 무엇인가요?
- 다른 예비부부에게 알려주고 싶은 점이 있나요?

체험단과 후기 이벤트의 원칙

초기에는 체험 기회를 제공하고 사용 경험을 부탁할 수 있다. 다만 보상의 조건을 긍정적인 평가나 높은 별점으로 만들면 신뢰를 훼손한다. 체험 사실과 보상 여부를 투명하게 알리고, 솔직한 의견을 요청해야 한다.

후기는 제품을 포장하는 도구가 아니라 제품을 개선하는 데이터다. 불편하다는 리뷰를 숨기기보다 같은 문제가 반복되는지 확인하고 수정해야 한다.

우리가 실제로 했던 방식과 지금 다시 한다면 바꿀 점

초기에는 스마트스토어에서 상품을 구매하고 리뷰를 작성하면 구매 금액을 돌려주는 체험단을 운영했다. 블로그 체험단에는 제작 과정과 주요 기능을 소개해 달라고 요청했고, 스마트스토어 체험단에도 방명록, 참석 의사 확인, 송금, 자유로운 수정 기능을 살펴봐 달라고 안내했다.

이 방식은 짧은 시간에 사용 사례와 후기를 확보하는 데 도움이 됐다. 동시에 분명한 한계가 있었다. 판매자가 강조할 기능을 미리 제시하면 체험자의 솔직한 언어보다 우리가 원하는 메시지가 리뷰에 반복될 수 있다. 구매액을 전액 돌려주는 방식도 일반 구매자의 반응과 같은 데이터로 보면 안 된다.

지금 다시 한다면 다음처럼 운영할 것이다.

- 체험 또는 보상 제공 사실을 명확히 표시한다.
- 높은 별점이나 긍정적인 표현을 조건으로 삼지 않는다.
- 강조할 장점을 지정하지 않고 불편한 점도 함께 묻는다.
- 체험 리뷰와 실제 유료 구매 리뷰를 분리해 분석한다.
- 해당 플랫폼의 최신 리뷰·광고 표시 정책을 먼저 확인한다.

초기 성장을 위해 한 행동을 성공 공식으로 포장하기보다, 어떤 데이터 왜곡이 생겼는지까지 설명하는 편이 다음 창업자에게 더 유용하다.

실행 노트

후기 요청 문구를 세 질문으로 바꾼다. 문제, 사용한 기능, 결과를 각각 묻는다. 수집한 후기에는 체험 제공 여부를 구분해 기록하고, 공개 시 관련 플랫폼의 표시 기준을 확인한다.

13장. 고객 문의에서 다음 기능을 찾는 법

고객 문의는 운영자를 지치게 하지만 동시에 가장 값싼 제품 리서치다. 같은 질문이 반복된다면 고객이 설명을 읽지 않은 것이 아니라 제품이 충분히 설명하지 못했을 가능성이 크다.

문의는 네 종류로 나눌 수 있다.

1. 사용 방법을 찾지 못한 문의

기능은 있지만 위치나 표현이 이해되지 않는 경우다. 화면의 버튼명, 안내 순서, 입력 예시를 수정하면 줄일 수 있다.

2. 오류와 예외 상황 문의

사진 업로드 실패, 결제 오류, 링크 문제처럼 정상 흐름 밖에서 발생한다. 자주 발생하면 시스템 개선이 필요하고, 드문 경우라면 빠른 대응 절차가 필요하다.

3. 아직 없는 기능을 요청하는 문의

한 명의 요청만으로 바로 만들지 않는다. 같은 상황이 반복되는지, 실제 결제에 영향을 주는지 확인한다.

4. 감정과 예절에 관한 문의

결혼식처럼 가족과 관계가 얽힌 상품에서는 기능만으로 해결되지 않는 문제가 있다. 계좌번호를 어떻게 보여줄지, 어떤 문구가 자연스러운지 같은 질문이다. 이런 문의는 템플릿과 예시 콘텐츠로 해결할 수 있다.

문의를 기능으로 바꾸기 전 세 번 묻는다

- 이 문제가 몇 명에게 반복되었는가?
- 해결하지 않으면 구매나 사용을 포기하는가?
- 기능 개발 대신 문구나 운영으로 먼저 해결할 수 있는가?

모든 문의를 기능으로 만들면 제품은 복잡해진다. 반대로 반복되는 문제를 계속 사람의 응대로만 처리하면 운영이 무너진다. 빈도와 영향도를 함께 봐야 한다.

문의 기록표

문의	빈도	결제 영향	현재 대응	제품 개선
입력 방법을 모르겠음	높음	중간	개별 안내	입력 예시 추가
결제 실패	낮음	높음	수동 확인	실패 안내·재시도 개선
계좌번호 표현 문의	높음	높음	문구 추천	선택형 문구 제공
특수 기능 요청	낮음	낮음	미지원 안내	보류

실행 노트

최근 문의 30건을 네 유형으로 분류한다. 빈도와 결제 영향을 각각 상·중·하로 표시한다. 빈도와 영향이 모두 높은 항목부터 제품 또는 안내 문구를 수정한다.

14장. 자체 결제에서 스마트스토어로 이동한 이유



처음에는 웹서비스 안에서 결제가 이루어지도록 결제대행사를 연결했다. 고객이 정보를 입력하고 결제한 뒤 바로 사용하는 흐름은 자연스러웠다.

하지만 좋은 결제 경험만으로 고객이 유입되는 것은 아니었다. 자체 사이트는 검색 노출과 신뢰를 처음부터 만들어야 했다. 반면 네이버 스마트스토어는 고객이 이미 익숙한 결제 환경과 리뷰, 검색 노출 구조를 갖고 있었다.

우리는 운영 중 결제 채널을 스마트스토어 중심으로 바꾸었다. 서비스의 핵심 기능은 자체 웹에서 제공하되, 고객이 상품을 발견하고 결제하는 일부 과정을 기존 플랫폼에 맡기는 방식이었다.

플랫폼을 쓰면 마진만 줄어드는가

플랫폼에는 수수료와 정책 제약이 있다. 고객 정보와 구매 흐름을 완전히 통제하기도 어렵다. 그럼에도 초기 사업자가 플랫폼을 사용할 이유가 있다.

- 고객이 결제 방식을 이미 알고 있다.
- 구매와 환불에 대한 기본적인 신뢰가 있다.

- 리뷰가 한곳에 쌓인다.
- 검색과 쇼핑 영역에서 발견될 가능성이 커진다.
- 여러 결제 수단을 직접 관리하는 부담이 줄어든다.

수수료만 비교하면 자체 결제가 유리해 보일 수 있다. 하지만 고객 획득 비용과 신뢰 구축 비용까지 포함하면 결과가 달라진다.

자체 채널과 플랫폼을 대립시키지 않는다

자체 웹사이트는 제품 경험과 고객 데이터를 설계하기 좋다. 플랫폼은 발견과 결제의 장벽을 낮춘다. 두 채널의 역할을 나눌 수 있다.

- 플랫폼: 검색, 리뷰, 결제 신뢰
- 자체 서비스: 정보 입력, 제작, 수정, 결과물 사용

실제 연결 방식은 쿠폰이었다. 고객이 스마트스토어에서 결제를 완료하면 네이버 특통을 통해 자체 웹서비스에서 사용할 제작 쿠폰을 전달했다. 고객은 쿠폰을 입력한 뒤 테마를 선택하고 모바일 청첩장을 완성했다.

이 방식은 플랫폼의 결제 신뢰와 자체 서비스의 제작 경험을 연결했지만 운영 업무도 만들었다. 쿠폰 생성과 전달, 사용 여부 확인, 결제자와 실제 사용자 연결, 환불 시 쿠폰 상태 처리까지 관리해야 했다. 채널을 연결할 때는 고객의 편리함뿐 아니라 운영자가 처리할 예외 상황도 함께 설계해야 한다.

중요한 것은 유행하는 채널을 선택하는 것이 아니라 고객이 어디에서 이탈하는지 보는 일이다. 유입이 부족한지, 결제에서 망설이는지, 결제 후 사용이 어려운지에 따라 해결책이 달라진다.

실행 노트

현재 판매 흐름을 발견 -> 비교 -> 신뢰 -> 결제 -> 사용 -> 재추천으로 나눈다. 자체 채널과 외부 플랫폼 중 각 단계에서 더 강한 쪽을 표시한다. 모든 단계를 한 채널에 억지로 넣지 말고 연결 비용을 계산한다.



4부. 작은 서비스를 사업으로 바꾸기

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

15장. B2C에서 웨딩플래너용 B2B로 확장하기



처음 서비스의 고객은 예비부부였다. 한 명의 고객이 자신의 모바일 청첩장을 만들고 결제하는 B2C 구조였다.

운영하면서 다른 사용자가 보이기 시작했다. 여러 예비부부를 지속적으로 만나는 웨딩플래너와 웨딩 관련 업체였다. 이들은 모바일 청첩장을 직접 사용하는 고객은 아니지만, 자신의 고객에게 부가 서비스로 제공할 수 있었다.

한 번 결혼하는 개인보다 반복해서 고객을 만나는 사업자와 연결하면 판매 구조가 달라진다. 매번 새로운 개인 고객을 광고로 데려오는 대신 한 명의 파트너가 여러 건의 사용을 만들 수 있다.

우리는 제휴 회원이 할인된 제작 쿠폰을 구매해 자신의 고객에게 제공하는 방식을 준비했다. 플래너는 고객에게 추가 혜택을 제공할 수 있고, 우리는 반복적인 판매 채널을 얻는 구조였다.

웨딩플래너뿐 아니라 웨딩사진 스튜디오에도 제휴 제안을 준비했다. 촬영 고객에게 모바일 청첩장 쿠폰을 함께 제공하면 스튜디오는 부가 혜택을 만들고, 우리는 예비부

부가 이미 신뢰하는 채널을 통해 판매할 수 있었다. 당시 기업회원에게는 구매량에 따라 큰 폭의 할인안을 제시했지만, 지금 다시 설계한다면 할인율보다 파트너별 재구매율과 지원 비용을 먼저 계산할 것이다.

B2B 확장은 기능 추가가 아니라 고객 변경이다

개인 고객에게 좋은 제품이 사업자에게도 그대로 좋은 것은 아니다. 사업자는 다음과 같은 질문을 한다.

- 여러 고객의 이용권을 한꺼번에 관리할 수 있는가?
- 고객에게 전달하는 과정이 간단한가?
- 할인과 정산 기준이 명확한가?
- 문제가 생겼을 때 빠르게 지원받을 수 있는가?
- 자신의 서비스처럼 자연스럽게 제공할 수 있는가?

그래서 B2B 확장에는 제휴용 페이지와 쿠폰 관리, 안내 자료가 필요했다. 개인 고객의 제작 화면을 그대로 보여주는 것만으로는 충분하지 않았다.

가장 가까운 반복 구매자를 찾는다

B2B 기회는 멀리 있는 대기업 제휴에서만 나오지 않는다. 현재 고객이 구매 결정을 내리기 전에 만나는 사람을 살펴보면 된다.

- 고객에게 상품을 추천하는 사람
- 같은 문제를 여러 번 대신 처리하는 사람
- 고객 경험을 높이기 위해 부가 서비스를 찾는 사람
- 여러 고객을 관리하는 대행자와 전문가

모바일 청첩장에는 웨딩플래너가 있었다. 다른 서비스라면 세무사, 강사, 컨설턴트, 중개인, 에이전시가 될 수 있다.

실행 노트

현재 고객이 구매 전후에 만나는 전문가와 업체를 다섯 개 적는다. 그중 같은 문제를 한 달에 두 번 이상 접하는 대상을 찾는다. 그들이 고객에게 서비스를 대신 제공할 때 필요한 관리 기능과 가격 구조를 인터뷰한다.

16장. 관리자 페이지가 매출보다 먼저 해결한 것

초기에는 주문이 많지 않아 수동으로 확인해도 큰 문제가 없다. 창업자는 관리자 기능보다 고객에게 보이는 기능을 먼저 만들고 싶어 한다. 그러나 주문과 문의가 늘면 운영 화면이 제품의 성장 속도를 결정한다.

관리자 페이지의 목적은 멋진 그래프를 보여주는 것이 아니다. 고객 문제를 빠르게 찾고 해결하게 만드는 것이다.

우리에게 필요했던 정보는 다음과 같았다.

- 누가 결제했고 어떤 상태인가?
- 고객이 제작을 완료했는가?
- 오류가 발생한 단계는 어디인가?
- 쿠폰이 정상적으로 사용되었는가?
- 문의가 왔을 때 해당 고객의 정보를 찾을 수 있는가?

이 정보가 흩어져 있으면 문의 하나를 해결하기 위해 결제 내역, 사용자 정보, 제작 결과를 각각 찾아야 한다. 자동화 서비스가 성장해도 운영자는 계속 여러 화면을 오가며 수작업을 하게 된다.

운영자의 시간도 사용자 경험이다

고객이 오류를 겪었을 때 운영자가 상황을 바로 확인하지 못하면 답변이 늦어진다. 고객 입장에서는 제품 오류와 느린 응대가 하나의 나쁜 경험이다.

관리자 기능은 내부 도구지만 고객 경험에 직접 영향을 준다.

무엇을 자동화할지 운영 기록이 알려준다

처음부터 완벽한 관리자 시스템을 설계하기는 어렵다. 일정 기간 수동으로 운영하면서 다음 내용을 기록한다.

- 하루에 반복한 작업
- 한 건을 처리하는 데 걸린 시간
- 실수가 자주 발생한 단계
- 고객에게 같은 답을 보낸 횟수
- 문제 해결을 위해 확인한 데이터

반복 횟수와 소요 시간을 곱하면 자동화 우선순위가 보인다. 하루 한 번 20분이 걸리는 작업은 한 달이면 상당한 시간이 된다. 반대로 한 달에 한 번 발생하는 예외 상황은 자동화 비용이 더 클 수 있다.

실행 노트

일주일 동안 운영 업무를 10분 단위로 기록한다. 빈도 × 처리시간 × 오류위험이 높은 세 가지를 고른다. 기능 개발, 안내 개선, 외부 도구 연결 중 가장 작은 비용으로 줄일 방법을 찾는다.

17장. 자동화해도 사람이 해야 하는 일은 남는다

고객이 정보를 입력하고 결제하면 결과물이 자동으로 만들어졌다. 재고를 매입하거나 포장하고 택배를 보낼 필요도 없었다. 실물 상품에 비해 추가 주문을 처리하는 비용이 낮았다.

그래서 자동화 서비스는 운영이 거의 필요 없을 것이라고 생각하기 쉽다. 실제로 반복 제작 업무는 크게 줄었다. 하지만 사람이 해야 하는 일은 다른 형태로 남았다.

- 입력 방법과 기능에 대한 고객 문의
- 결제와 쿠폰 오류 확인
- 사진과 문구가 예상대로 보이지 않는 예외 상황
- 환불과 이용 정책 안내
- 새로운 스마트폰과 브라우저에서의 품질 점검
- 제휴 업체와의 소통
- 경쟁 서비스와 고객 요구의 변화 확인

자동화는 노동을 없애기보다 노동의 종류를 바꾼다. 제작 노동은 줄지만 제품 관리와 고객 대응의 중요성은 커진다.

자동수익이라는 표현이 감추는 것

온라인에서는 잠자는 동안에도 돈이 들어오는 구조가 자주 강조된다. 시스템이 결제를 처리하는 동안 운영자가 자고 있을 수는 있다. 하지만 시스템을 만들고 유지하고 개선하는 시간까지 사라지는 것은 아니다.

고객이 늘면 작은 오류의 영향도 커진다. 한 명에게 발생하던 문제가 하루 열 명에게 반복될 수 있다. 자동화된 판매는 운영을 안 해도 된다는 뜻이 아니라, 한 번의 개선이 더 많은 고객에게 적용된다는 뜻에 가깝다.

운영 수준을 세 단계로 나눈다

- 1 반복 업무: 정해진 절차로 처리할 수 있다. 자동화하거나 문서화한다.
- 2 예외 업무: 빈도는 낮지만 판단이 필요하다. 대응 기준을 만든다.
- 3 개선 업무: 고객 행동과 시장을 보며 제품을 바꾼다. 창업자가 계속 관여해야 한다.

반복 업무를 줄여야 개선 업무에 시간을 쓸 수 있다. 자동화의 목적은 창업자를 제품에서 완전히 떼어놓는 것이 아니라 더 중요한 판단에 집중하게 하는 것이다.

실행 노트

현재 하는 업무를 반복, 예외, 개선으로 분류한다. 반복 업무에는 자동화 또는 매뉴얼을 붙이고, 예외 업무에는 처리 기준과 책임자를 정한다. 매주 최소 한 시간은 개선 업무를 위해 비워둔다.

18장. 직장과 서비스를 동시에 운영하는 현실

사이드 프로젝트는 본업이 끝난 뒤 시작된다. 퇴근 후와 주말을 사용하면 된다고 생각하지만, 서비스가 고객을 만나기 시작하면 시간을 원하는 대로 배치하기 어렵다.

고객 문의는 내가 한가할 때만 오지 않는다. 결제 오류와 긴급한 수정 요청도 업무시간을 피해 발생하지 않는다. 결혼식을 앞둔 고객에게는 작은 문제도 긴급하다.

우리 팀은 두 명 모두 직장인이었다. 서비스에 더 집중해 광고하고 고객을 대응하면 성장 가능성이 있었지만, 본업과 함께 운영하면서 충분한 시간을 쓰지 못했다. 최근 이직까지 겹치며 관리할 수 있는 시간은 더 줄었다.

사이드 프로젝트가 사업이 되는 순간

아이디어를 만드는 동안에는 시간을 선택할 수 있다. 고객이 돈을 지불한 뒤에는 일정 수준의 책임이 생긴다. 다음 질문에 답해야 한다.

- 평일 업무시간에 문의가 오면 누가 답하는가?
- 장애가 생기면 몇 시간 안에 확인할 것인가?
- 휴가와 출장 중에는 어떻게 운영하는가?
- 한 명이 바쁠 때 다른 사람이 대신할 수 있는가?
- 매주 개선에 사용할 고정 시간이 있는가?

이 질문에 답하지 못하면 매출이 늘수록 부담도 커진다.

시간보다 에너지의 문제가 크다

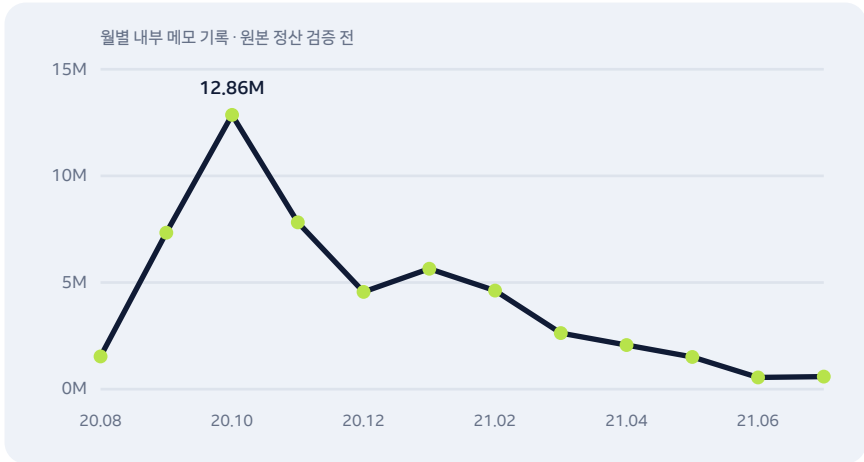
퇴근 후 두 시간이 있다고 해서 항상 집중할 수 있는 것은 아니다. 본업에서 의사결정을 많이 한 날에는 작은 운영 업무도 무겁게 느껴진다. 그래서 사이드 프로젝트 계획은 남은 시간이 아니라 반복 가능한 에너지를 기준으로 세워야 한다.

- 즉시 대응이 필요한 업무를 줄인다.
- 문의 답변과 점검 시간을 정해 묶어서 처리한다.
- 팀원만 아는 운영 방법을 문서화한다.
- 주간 유지 시간의 상한을 정한다.
- 상한을 넘으면 자동화, 외주, 가격 인상을 검토한다.

실행 노트

최근 네 주 동안 서비스에 쓴 시간을 제작, 고객대응, 오류처리, 마케팅, 개선으로 나눈다. 본업과 병행 가능한 주간 상한을 정하고 이를 넘긴 원인을 찾는다. 성장보다 먼저 운영 지속 가능성을 계산한다.

19장. 아무것도 하지 않아도 팔릴 때 더 위험하다



서비스가 입소문을 타면서 별도의 홍보를 하지 않아도 결제가 발생하는 시기가 왔다. 만들 때 꿈꾸던 장면이었다. 고객이 스스로 찾아와 결제하고 결과물을 사용하는 구조가 작동하고 있었다.

하지만 역설적으로 그때 서비스에 덜 집중하게 되었다.

매일 판매를 위해 애쓰지 않아도 결제가 생기니 긴장감이 낮아졌다. 본업이 바쁘다는 이유로 마케팅과 제품 개선을 미뤘다. 서비스가 잘 작동하는 것과 사업이 성장하는 것을 같은 의미로 받아들이었다.

안정적인 매출은 정체를 가릴 수 있다

현재 매출이 유지된다고 해서 경쟁력이 강해지고 있는 것은 아니다.

- 신규 고객 유입 경로가 한 채널에 의존할 수 있다.

- 경쟁사가 같은 기능을 빠르게 따라올 수 있다.
- 고객 문의가 쌓이지만 제품 개선은 멈출 수 있다.
- 특정한 사회적 상황으로 생긴 수요가 줄어들 수 있다.
- 운영자가 제품의 핵심 데이터를 보지 않을 수 있다.

특히 결혼과 비대면 소통 환경의 변화는 당시 수요에 영향을 주었다. 일시적인 바람을 제품의 영구적인 성장으로 착각하면 안 된다.

자동화된 사업에도 주간 계기판이 필요하다

매주 다음 숫자를 확인하면 정체를 빠르게 발견할 수 있다.

- 방문자와 유입 채널
- 샘플 확인에서 입력 시작으로 넘어간 비율
- 입력 시작에서 결제까지의 비율
- 결제 실패와 환불 건수
- 고객 문의의 종류와 해결 시간
- 제휴 파트너의 재구매
- 운영에 투입한 시간

매출만 보면 고객이 어디에서 늘거나 줄었는지 알 수 없다. 과정의 숫자를 함께 봐야 다음 행동을 정할 수 있다.

월별 기록이 보여준 진짜 흐름

내부 메모에 남은 월별 결제 기록을 보면 출시 직후의 변화가 선명하다. 2020년 8월 약 153만 원에서 9월 약 733만 원, 10월 약 1,286만 원으로 빠르게 증가했다. 이후 11월 약 782만 원, 12월 약 456만 원으로 내려왔고, 이듬해에는 대체로 하락 흐름이 이어졌다. 이 숫자는 공개 전 결제·정산 자료와 다시 대조해야 한다.

이 기록은 두 가지를 동시에 보여준다.

첫째, 고객의 문제가 실제 결제로 이어졌다. 짧은 기간에 매출이 빠르게 증가한 것은 상품과 시점이 맞았다는 신호였다.

둘째, 초기 반응이 지속적인 성장으로 자동 연결되지는 않았다. 출시 효과와 당시의 비대면 수요, 커뮤니티 홍보가 약해지면서 매출도 내려갔다. 제품이 자동으로 판매된다는 사실에 안심하고 신규 유입과 재구매 구조를 충분히 만들지 못했다.

“한 달에 얼마를 벌었다”는 최고점만 보여주면 이 경험의 절반을 감추게 된다. 더 중요한 교훈은 최고 매출 다음 달에 무엇을 했고, 무엇을 하지 않았는가에 있다.

실행 노트

매주 30분 동안 확인할 지표를 여섯 개 이하로 정한다. 숫자마다 기준선을 만들고, 기준보다 떨어졌을 때 실행할 행동을 한 가지씩 연결한다. 계기판은 보는 데서 끝나지 않고 행동을 바꾸어야 한다.



5부. 계속할 것인가, 팔 것인가

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

20장. 잘 팔리던 서비스를 매각한 이유



서비스가 실패해서 매각한 것은 아니었다. 고객이 있었고 결제가 발생했으며, 운영에 더 집중하면 성장시킬 가능성도 보였다.

문제는 가능성과 우리의 상황이 맞지 않았다는 것이다.

함께 만든 두 사람 모두 직장인이었다. 제품을 개선하고 광고를 집행하며 제휴 업무를 늘리려면 더 많은 시간이 필요했다. 하지만 본업과 이직, 각자의 생활 속에서 서비스가 요구하는 집중도를 확보하기 어려웠다.

아무런 홍보를 하지 않아도 판매가 일어난다는 사실은 좋은 신호였지만 동시에 아쉬운 신호였다. 고객이 있다는 것이 확인됐는데도 기회를 충분히 활용하지 못하고 있었다. 서비스를 계속 보유하는 것보다 더 집중할 수 있는 운영자에게 넘기는 선택을 고민하게 됐다.

매각은 실패도, 자동적인 성공도 아니다

서비스를 팔았다고 하면 큰 성공을 떠올릴 수 있다. 반대로 계속 운영하지 못했다는 점만 보면 포기라고 생각할 수도 있다. 실제로는 제한된 자원 속에서 내리는 사업 결정이다.

다음 네 가지 선택지를 비교해야 한다.

- 1 현 상태 유지: 최소 운영만 하며 매출을 유지한다.
- 2 집중 투자: 본업 또는 다른 일을 줄이고 서비스 성장에 집중한다.
- 3 운영 위임: 고객 대응과 마케팅을 맡을 사람을 채용하거나 외주화한다.
- 4 매각: 제품과 운영 자산을 다른 주체에게 이전한다.

각 선택에는 비용이 있다. 현 상태를 유지하면 시장 기회를 잃을 수 있고, 집중 투자에는 안정적인 소득을 포기하는 위험이 있다. 운영 위임은 관리 역량과 고정비가 필요하다. 매각하면 미래의 성장 가능성을 넘기는 대신 현재의 시간과 자원을 회수할 수 있다.

팔기 전에 답해야 할 질문

- 이 사업을 앞으로 2년 더 운영하고 싶은가?
- 성장시키기 위해 매주 몇 시간이 필요한가?
- 그 시간을 확보하려면 무엇을 포기해야 하는가?
- 다른 사람이 운영하면 더 성장할 가능성이 있는가?
- 현재 수익보다 미래 가능성을 더 중요하게 보는가?
- 매각 후 나는 어떤 일에 집중할 것인가?

매각은 지쳤을 때 급하게 결정하면 협상력이 떨어진다. 제품이 안정적으로 작동하고 지표와 운영 자료가 정리되어 있을 때 선택지로 준비하는 편이 좋다.

실행 노트

유지, 집중, 위임, 매각의 네 선택지를 표로 만든다. 각 선택에 필요한 주간 시간, 추가 비용, 예상 위험, 1년 뒤 기대 상태를 적는다. 감정이 아니라 자원 배분의 관점에서 비교한다.

21장. 공동창업자와 역할·수익을 나눌 때 생기는 문제

두 사람이 함께 만들면 혼자서는 할 수 없는 일을 빠르게 시작할 수 있다. 나는 제품과 고객 경험, 판매를 맡고 개발자는 기술 구현을 담당했다. 서로 다른 능력이 결합됐기 때문에 서비스가 실제로 출시될 수 있었다.

하지만 제품을 만드는 기여와 계속 운영하는 기여는 시간이 지나며 달라질 수 있다.

초기에는 두 사람 모두 개발과 출시를 위해 많은 시간을 쓴다. 출시 후에는 고객 대응, 마케팅, 정산, 오류 수정처럼 새로운 업무가 생긴다. 한 사람은 본업이 바빠지고 다른 사람은 운영을 더 맡을 수 있다. 처음 정한 동일 배분이 계속 공정한지 의문이 생길 수 있다.

지분, 수익, 노동비는 서로 다르다

공동 프로젝트에서는 세 개를 구분해야 한다.

- 소유권: 서비스와 자산을 누가 얼마나 소유하는가?
- 수익 배분: 비용을 제외한 이익을 어떤 비율로 나누는가?
- 운영 보상: 고객 대응과 마케팅 같은 지속 노동에 얼마를 지급하는가?

세 가지를 하나의 비율로 처리하면 역할이 변할 때 갈등이 생긴다. 초기 제작의 기여는 소유권에 반영하고, 매달 반복되는 운영은 별도의 보상 기준을 둘 수 있다.

비용을 빼기 전 돈을 나누지 않는다

결제액이 곧 이익은 아니다. 플랫폼과 결제 수수료, 서버비, 광고비, 세금, 외주비, 환불을 반영해야 한다. 두 사람이 각자 쓴 비용도 기록해야 한다.

정산식은 단순하고 공개적이어야 한다.

**총결제액 - 환불 - 결제·플랫폼 수수료 - 광고비 - 서버·도구비 - 외주비 - 세금 총당액
= 배분 가능한 이익**

운영 노동에 별도의 보상을 정했다면 배분 전후 어느 단계에서 지급하는지도 합의한다.

관계를 지키기 위해 불편한 대화를 먼저 한다

친한 사람과 시작하면 계약 이야기를 꺼내기 어렵다. 하지만 돈이 생기기 전에 대화하는 편이 훨씬 쉽다.

- 한 달 동안 아무 일도 하지 못하면 어떻게 하는가?
- 한 사람이 프로젝트를 그만두고 싶으면 어떻게 하는가?
- 추가 개발 비용은 누가 부담하는가?
- 매각 가격과 조건은 누가 결정하는가?
- 의견이 같지 않을 때 최종 결정권은 누구에게 있는가?

답이 완벽할 필요는 없다. 상황이 바뀌면 다시 합의할 수 있다. 중요한 것은 기대의 차이를 침묵 속에 쌓아두지 않는 것이다.

실행 노트

공동창업자와 분기마다 역할과 기여를 검토한다. 소유권은 자주 바꾸지 않더라도 운영 업무와 보상은 현실에 맞게 조정한다. 합의 내용과 적용 날짜를 문서로 남긴다.

22장. 서비스의 가치를 설명하는 숫자들

서비스를 매각하려면 “좋은 아이디어”라는 설명만으로 부족하다. 인수자는 제품을 넘겨받았을 때 얼마나 안정적으로 운영할 수 있고 어떤 성장 가능성이 있는지 알고 싶어 한다.

우리의 기록에는 사용자, 세션, 결제액, 판매 건수, 일별 결제, 월 최고 매출과 같은 숫자가 있었다. 다만 숫자가 많다고 가치가 명확해지는 것은 아니다. 집계 기간과 정의가 다르면 오히려 신뢰를 떨어뜨린다.

매각 자료에 필요한 네 종류의 지표

1. 수요

- 월별 방문자와 유입 채널
- 신규 고객 수
- 검색과 직접 유입 비중
- 광고를 중단한 뒤에도 발생한 방문과 결제

2. 전환

- 샘플 확인에서 제작 시작으로 이동한 비율
- 제작 시작에서 결제까지의 비율
- 채널별 구매 전환율
- 결제 실패와 환불 비율

3. 수익성

- 월별 총결제액과 회계상 매출
- 결제·플랫폼 수수료
- 광고비와 고객 획득 비용
- 서버와 외부 도구 비용
- 운영 인건비를 포함한 실제 이익

4. 운영 난이도

- 주간 운영 시간
- 문의 건수와 유형
- 장애 빈도
- 반복 업무의 자동화 수준
- 특정 개인에게 의존하는 업무

많은 서비스가 매출만 보여주고 운영자의 시간을 제외한다. 인수자에게는 매출 1원이 아니라 그 매출을 유지하기 위해 필요한 노동이 중요하다.

숫자의 정의를 통일한다

사용자와 방문자, 결제액과 매출, 주문과 실제 사용 건수는 다르다. 판매페이지와 매각 자료에서 용어를 섞으면 안 된다.

- 총결제액: 고객이 결제한 전체 금액
- 매출: 회계 기준에 따라 인식한 금액
- 이익: 모든 비용과 총당액을 반영한 뒤 남은 금액
- 사용자: 분석 도구의 정의에 따른 고유 사용자
- 판매 건수: 결제가 완료되고 취소되지 않은 주문

집계 기간과 데이터 출처를 숫자 옆에 적는다. 기억에 의존한 숫자는 공개하지 않는다.

실행 노트

최근 12개월의 손익과 운영 지표를 한 표에 정리한다. 각 숫자에 정의, 집계 기간, 원본 자료 위치를 붙인다. 증빙할 수 없는 숫자는 추정치로 명확히 표시하거나 외부 공개에서 제외한다.

23장. 매각 전에 정리해야 할 운영 자산

디지털 서비스는 소스코드만 넘긴다고 운영할 수 있는 것이 아니다. 고객이 상품을 발견하고 결제하고 문제를 해결하는 전체 시스템이 함께 이전되어야 한다.

제품 자산

- 소스코드와 배포 방법
- 데이터베이스 구조와 백업 방법
- 디자인 원본과 사용한 폰트·이미지의 라이선스
- 도메인과 서버 계정
- 외부 API와 결제 연동 정보
- 알려진 오류와 향후 개발 목록

판매 자산

- 브랜드명과 상표 관련 자료
- 상품 설명과 상세페이지
- 검색·소셜미디어 계정
- 리뷰와 고객 사례
- 제휴 파트너 목록과 계약
- 광고 소재와 과거 성과

운영 자산

- 고객 문의 답변 템플릿

- 환불과 이용 정책
- 일·주·월 운영 체크리스트
- 장애 발생 시 대응 방법
- 정산과 세금 처리 흐름
- 외주업체와 연락처

데이터와 개인정보

고객 데이터는 마음대로 넘길 수 있는 자산이 아니다. 개인정보 처리방침, 이용약관, 고객 동의 범위와 관련 법률을 확인해야 한다. 필요한 경우 법률·세무 전문가의 검토를 받는다.

결제 계정과 플랫폼 계정도 양도가 가능한지 각각 확인해야 한다. 개인 명의로 만든 서비스라면 사업자 변경 과정에서 새로운 계약과 고객 안내가 필요할 수 있다.

인수자가 없어도 정리할 가치가 있다

운영 문서는 매각을 위해서만 만드는 것이 아니다. 창업자 자신이 휴가를 가거나 새로운 팀원이 들어왔을 때도 필요하다. 매각 준비가 잘된 서비스는 대체로 운영도 잘 정리된 서비스다.

실행 노트

제품, 판매, 운영, 법무·데이터의 네 폴더를 만든다. 각 항목에 담당자, 계정 소유자, 갱신일, 이전 가능 여부를 기록한다. 비밀번호 자체를 일반 문서에 적지 말고 안전한 비밀번호 관리 도구를 사용한다.

24장. 첫 서비스가 다음 도전에 남긴 것

나는 모바일 청첩장 서비스를 만들기 전에도 여러 제품을 경험했다.

대학생 때는 스마트워치로 호텔 직원의 업무를 개선하는 서비스를 만들었다. 미국 호텔에서 테스트하고 투자도 받았지만, 핵심 사용 환경에서 배터리가 너무 빨리 소모되는 문제를 해결하지 못했다. 좋은 시장과 화려한 기회가 있어도 기술적 제약을 잘못 판단하면 제품은 지속될 수 없었다.

이후 해커톤에서 만난 사람들과 피트니스 서비스를 만들었다. 현장의 트레이너와 사용자를 관찰하며 고객 관리와 운동검사 문제를 제품으로 바꾸는 과정을 경험했다. 블록체인 보안 회사에서는 복잡한 데이터를 사용자가 이해할 수 있는 제품으로 설계했다.

모바일 청첩장 서비스는 앞선 경험이 작은 형태로 모인 프로젝트였다.

- 사용자 행동에서 문제를 찾았다.
- 기존 시장과 경쟁 제품을 조사했다.
- 디자이너와 개발자가 역할을 나눴다.
- 고객이 직접 사용하는 흐름을 설계했다.
- 실제 결제를 통해 가설을 검증했다.
- 운영의 병목을 자동화했다.
- 계속 보유하는 대신 매각을 선택했다.

이 경험은 이후 혼자 제품을 만드는 방식에도 남았다. 당시에는 개발자와 역할을 나누고 화면과 요구사항을 건네며 제품을 완성했다. AI 코딩 도구를 쓰기 시작한 뒤에는 그 협업 원리가 사라진 것이 아니라 더 세밀해졌다. 문제를 한 문장으로 정의하고, 화면과 데이터 흐름을 나누고, 작은 단위로 구현하고, 실제 환경에서 검수하는 순서가 그대로 필요했다.

아파트 구매 판단에 흠어진 정보를 연결한 서비스에서는 계산과 공공데이터, 추천, 운영 자동화를 하나의 흐름으로 묶었다. 개인 읽기와 공동체 읽기를 잇는 성경 앱에서는 기록, 그룹, 묵상과 AI 해설, 앱 출시의 관계를 설계했다. 회사 업무에서는 Figma 시안을 반응형 코드로 옮겨 실제 페이지 배포까지 이어갔다. 첫 유료 서비스에서 배운 고객·운영·완료의 기준이 있었기 때문에, 새 도구를 단순히 화면을 빨리 만드는 수단으로만 보지 않을 수 있었다.

도구는 달라졌지만 질문은 같았다. 누가 왜 쓰는가, 어디에서 멈추는가, 무엇을 저장해야 하는가, 실패했을 때 어떻게 돌아오는가, 출시 뒤 누가 운영하는가. 첫 서비스가 남긴 가장 긴 자산은 특정 기능이나 매출 숫자가 아니라 이 질문들을 끝까지 놓치지 않는 습관이었다.

첫 서비스의 목적은 평생 운영할 회사를 찾는 것만이 아니다

첫 제품이 반드시 큰 회사가 될 필요는 없다. 작은 유료 서비스를 끝까지 만들어보면 다음 도전에서 무엇을 확인해야 하는지 알게 된다.

아이디어를 설명하는 것과 고객에게 판매하는 것은 다르다. 화면을 디자인하는 것과 운영 가능한 제품을 만드는 것도 다르다. 매출이 발생하는 것과 지속 가능한 사업을 운영하는 것도 다르다.

이 차이를 책으로만 완전히 배울 수는 없다. 직접 작은 제품을 출시하고 고객의 질문과 결제를 만나야 한다.

당신의 첫 서비스는 더 작아도 된다

처음부터 결제 시스템과 복잡한 웹서비스를 만들 필요는 없다. 입력 폼과 수동 제작으로 시작할 수 있고, 한 명의 고객에게 결과물을 제공하며 검증할 수도 있다. 반복되는 과정이 확인되면 그때 자동화한다.

중요한 것은 아이디어의 크기가 아니라 한 사람의 문제를 끝까지 해결하고 대가를 받아보는 경험이다.

이제 메모장에 적어둔 아이디어 중 하나를 고르자. 누가 이미 돈을 쓰고 있는지 확인하고, 경쟁 상품을 구매하고, 핵심 흐름을 한 장으로 그린다. 네 주 안에 한 명에게 판

매할 수 있는 가장 작은 형태를 만든다.

첫 결제는 사업의 완성이 아니다. 하지만 상상을 현실로 바꾸는 가장 분명한 경계선이다.

마지막 실행 노트

오늘부터 30일 뒤 첫 유료 고객을 만나는 날짜를 정한다. 완벽한 제품 대신 한 사람의 문제를 해결할 최소한의 결과물을 약속한다. 매주 조사, 제안, 제작, 판매의 네 단계 중 하나를 완료한다.



부록

경험을 실행 가능한 원칙으로 바꾸는 실전 파트

부록 1. 문제 발견 기록지

아래 질문에 한 문장씩 답한다.

- 1 문제를 겪는 사람은 누구인가?
- 2 문제가 발생하는 구체적인 상황은 언제인가?
- 3 현재 어떤 방법으로 해결하고 있는가?
- 4 현재 해결법에 쓰는 시간과 비용은 얼마인가?
- 5 가장 자주 기다리거나 반복하는 단계는 무엇인가?
- 6 문제를 해결하지 않으면 어떤 손해가 생기는가?
- 7 그 단계를 하나 제거한다면 무엇이 달라지는가?
- 8 30일 안에 검증할 수 있는 가장 작은 가설은 무엇인가?

부록 2. 경쟁 서비스 분석표

항목	경쟁 서비스 A	경쟁 서비스 B	경쟁 서비스 C	우리의 가설
주요 고객				
가격				
구매 전 불안				
정보 입력 방식				
결과 대기시간				
수정 방식				
고객 후기의 장점				
반복되는 불만				
우리가 제거할 단계				

가능하면 조사만 하지 말고 최소 한 개는 직접 구매한다.

부록 3. 고객 인터뷰 질문지

미래의 의향보다 최근의 행동을 묻는다.

- 1 마지막으로 이 문제를 겪은 때를 설명해 주세요.
- 2 당시 가장 먼저 무엇을 했나요?
- 3 어떤 서비스나 사람의 도움을 받았나요?
- 4 해결하는 데 얼마나 걸렸나요?
- 5 얼마를 지불했나요?
- 6 가장 불편하거나 불안했던 순간은 언제였나요?
- 7 중간에 포기하거나 다른 방법으로 바꾼 적이 있나요?
- 8 결과에서 가장 만족한 점은 무엇인가요?
- 9 다시 같은 문제가 생긴다면 무엇을 다르게 하겠나요?
- 10 샘플을 만들면 자신의 실제 자료로 시험해볼 의향이 있나요?

“이 서비스가 있으면 쓰겠어요?”라는 질문은 마지막까지 하지 않는다.

부록 4. 1페이지 MVP 기획서

고객

[구체적인 상황]에 있는 [구체적인 사람]

문제

현재 [기존 방법]을 사용하면서 [반복되는 불편]을 겪는다.

해결

우리 서비스는 [제거할 단계]를 없애고 [완료할 결과]를 제공한다.

핵심 흐름

발견 -> 입력 -> 결과 확인 -> 결제 -> 사용

첫 버전의 필수 기능 세 개

1. 2. 3.

만들지 않을 기능

1. 2. 3.

결제 가설

고객은 [구체적인 가치] 때문에 [가격]을 지불한다.

30일 검증 기준

인터뷰 __명 / 샘플 사용 __명 / 실제 결제 __건

부록 5. 개발자 협업 체크리스트

- ☐ 고객과 문제를 한 문장으로 합의했다.
- ☐ 핵심 사용자 흐름을 함께 확인했다.
- ☐ 첫 버전에서 제외할 기능을 정했다.
- ☐ 화면별 완료 조건을 적었다.
- ☐ 결제 실패와 입력 중단 등 예외 상황을 정리했다.
- ☐ 개발·디자인·운영 담당자를 정했다.
- ☐ 각자의 주간 투입 가능 시간을 확인했다.
- ☐ 비용 부담과 수익 배분 방식을 문서화했다.
- ☐ 코드, 디자인, 도메인, 데이터의 소유 관계를 정했다.
- ☐ 한 사람이 중단할 때 처리 방법을 정했다.
- ☐ 출시일과 출시 조건을 정했다.
- ☐ 출시 후 문의 대응 책임자를 정했다.

부록 6. 출시 전 점검표

상품

- ☐ 고객이 결과물을 미리 이해할 샘플이 있다.
- ☐ 가격과 포함 범위가 명확하다.
- ☐ 고객이 입력할 자료 목록을 안내했다.
- ☐ 수정과 환불 범위를 안내했다.

제품

- ☐ 처음부터 결제·사용까지 전체 흐름을 시험했다.
- ☐ 모바일 환경에서 확인했다.
- ☐ 결제 실패와 중복 결제를 시험했다.
- ☐ 고객이 입력 도중 나갔다 돌아오는 상황을 확인했다.
- ☐ 오류 발생 시 연락 방법이 보인다.

운영

- ☐ 문의 답변 템플릿을 준비했다.
- ☐ 주문과 결제 상태를 확인할 수 있다.
- ☐ 환불 절차를 정했다.
- ☐ 개인정보 처리방침과 이용약관을 확인했다.
- ☐ 매출·비용 기록 방법을 정했다.

부록 7. 첫 30일 고객 확보 계획



1주차 - 문제 확인

- 최근 문제를 겪은 고객 5명 인터뷰
- 경쟁 상품 2개 구매 분석
- 가장 큰 불편 1개 선택

2주차 - 제안 만들기

- 한 문장 가치 제안 작성
- 실제 결과에 가까운 샘플 제작
- 가격 가설과 판매 설명 작성

3주차 - 작은 판매

- 고객이 모인 채널 3곳에서 질문과 표현 조사
- 잠재 고객 10명에게 개별 제안

- 첫 유료 고객 또는 예약 고객 확보

4주차 - 개선

- 사용 과정 관찰
- 문의와 이탈 지점 기록
- 가장 큰 문제 1개 수정
- 다음 30일의 판매 목표 결정

부록 8. 서비스 운영·매각 체크리스트

운영

- ☐ 월별 방문, 주문, 결제, 환불을 기록한다.
- ☐ 수수료와 광고비, 서버비, 세금을 구분한다.
- ☐ 주간 운영 시간을 기록한다.
- ☐ 반복 문의와 오류를 분류한다.
- ☐ 계정과 도메인의 소유자를 기록한다.

매각 준비

- ☐ 최근 12개월 손익 자료가 있다.
- ☐ 숫자의 정의와 출처가 명확하다.
- ☐ 소스코드와 배포 방법이 정리되어 있다.
- ☐ 디자인과 콘텐츠의 라이선스를 확인했다.
- ☐ 운영 매뉴얼과 고객 대응 문서가 있다.
- ☐ 외부 서비스 계약의 이전 가능성을 확인했다.
- ☐ 개인정보 이전에 필요한 절차를 검토했다.
- ☐ 공동 소유자 모두 매각 조건에 합의했다.
- ☐ 세무·법률 전문가에게 필요한 검토를 받았다.



개발자가 아니어도 서비스의 책임자가 될 수 있습니다.

문제를 고르고, Codex에 맥락을 주고, 결과를 직접 검증하고, 작은 단위로 배포하세요.

첫 서비스의 목적은 완벽한 코드를 쓰는 것이 아니라 한 사람의 문제를 실제로 해결하는 것입니다.

필립